

Spring DI Interview Guide

09/03/2026 12:23:41

Dependency Injection / IoC What is IoC? What is Dependency Injection? Why do...

Absolutely. For a 5-year Java/Spring Boot interview, don't just memorize...

What is a Spring Bean? How does Spring create a bean? Explain the complete...

Yes. This is a very important 5-year Spring Boot interview area because...

. Bean Scopes What is singleton scope? Is Spring singleton the same as Singleto...

Spring Bean Scopes — 5-Year Interview Guide Bean scope answers one...

@Component vs @Service vs @Repository. Is @Service technically different fro...

Absolutely. This is another high-value 5-year Spring Boot interview topic. The ke...

What is Spring Boot? Spring Framework vs Spring Boot. Why was Spring Boot...

For a 5-year Java/Spring Boot interview, don't stop at "Spring Boot reduces...

What is auto-configuration? How does Spring Boot decide what to configure? Wh...

1. What is Auto-Configuration? Auto-configuration is a Spring Boot feature that...

application.properties vs application.yml. How do you define custom properties?...

1. application.properties vs application.yml Both are used for externalized...

Controllers @Controller vs @RestController. @RequestMapping. @GetMapping....

@Controller — Marks a Spring MVC class that typically returns a view/page such...

GET vs POST. PUT vs PATCH. PUT vs POST. What is idempotency? Which HTTP...

GET vs POST — GET retrieves data and should not modify server state; POST...

How do you handle exceptions globally? @ExceptionHandler. @ControllerAdvice....

1. How do you handle exceptions globally? Use @RestControllerAdvice +...

What is Bean Validation? @Valid vs @Validated. @NotNull. @NotBlank....

1. What is Bean Validation? Bean Validation is a standard Java mechanism for...

What is AOP? Why do we need AOP? Cross-cutting concerns. Aspect. Join point....

1. What is AOP? AOP (Aspect-Oriented Programming) is a programming approac...

What is @Transactional? How does @Transactional work internally? Transaction...

1. What is @Transactional? @Transactional tells Spring to execute a method with...

JPA vs Hibernate. What is ORM? Entity. @Entity. @Id. @GeneratedValue. Entity...

1. JPA vs Hibernate JPA — A Java specification/API that defines how Java...

What is Spring Data JPA? JpaRepository vs CrudRepository. How does Spring...

1. What is Spring Data JPA? Spring Data JPA is a Spring module that simplifies...

Authentication vs authorization. What is Spring Security? Security filter chain....

1. Authentication vs Authorization Authentication — Verifies who the user is....

What is Actuator? Health endpoint. Metrics. Info. Environment. Beans. Loggers....

1. What is Actuator? Spring Boot Actuator provides production-ready endpoints...

SLF4J vs Logback. Why use SLF4J? Log levels. DEBUG vs INFO vs WARN vs ERRO...

1. SLF4J vs Logback SLF4J — Logging abstraction/API that lets application code u...

What is Spring Cache abstraction? @Cacheable. @CachePut. @CacheEvict....

1. What is Spring Cache Abstraction? Spring Cache abstraction provides a...

Monolith vs microservices. When should you choose microservices?...

1. Monolith vs Microservices Monolith — The application is built and deployed as...

What is API Gateway? Why do we need an API Gateway? Gateway vs Load...

1. What is API Gateway? API Gateway is a single entry point for clients that rout...

Unit test vs integration test. JUnit 5. Mockito. @Mock. @InjectMocks. @MockBea...

1. Unit Test vs Integration Test Unit Test — Tests one class/component in isolatio...



Dependency Injection / IoC

What is IoC?

What is Dependency Injection?

Why do we need Dependency Injection?

Constructor injection vs setter injection vs field injection.

Why is constructor injection preferred?

What is the Spring IoC container?

BeanFactory vs ApplicationContext.

How does Spring create a bean?

How does Spring manage dependencies between beans?

What happens when two beans have the same type?

@Autowired vs @Qualifier.

@Primary vs @Qualifier.

1. Can Spring inject private fields?
2. Can Spring inject a bean through a constructor?
3. What happens if a dependency is missing?
4. What happens if multiple beans match the same interface?
5. How would you conditionally create a bean?



Absolutely. For a **5-year Java/Spring Boot interview**, don't just memorize definitions. Interviewers usually go one level deeper: **why, how Spring does it internally, what happens in edge cases, and which approach you'd choose in production.**

Dependency Injection / IoC — Interview Guide

1. What is IoC?

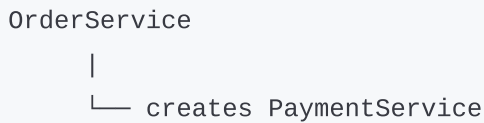
IoC (Inversion of Control) means that the responsibility of creating objects and managing their dependencies is transferred from the application code to a framework/container.

Without IoC

```
public class OrderService {  
  
    private PaymentService paymentService = new PaymentService();  
  
}
```

OrderService directly creates PaymentService .

So:



This creates tight coupling.

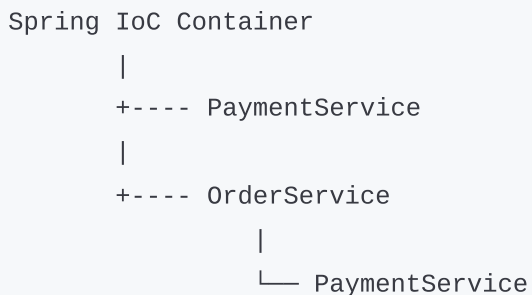
With Spring IoC

```
@Service
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Spring creates both objects and provides PaymentService to OrderService .



Interview answer

IoC means transferring control of object creation, configuration, and dependency management from application code to a container such as the Spring IoC container.

2. What is Dependency Injection?

Dependency Injection is one way of implementing IoC.

Suppose:

```
class OrderService {  
    private PaymentService paymentService;  
}
```

OrderService depends on PaymentService .

Instead of creating it:

```
PaymentService paymentService = new PaymentService();
```

we let Spring provide it:

```
@Service  
class OrderService {  
  
    private final PaymentService paymentService;  
  
    public OrderService(PaymentService paymentService) {  
        this.paymentService = paymentService;  
    }  
}
```

So:

```
Dependency  
↓  
PaymentService  
  
Injected into  
↓  
OrderService
```

Important distinction

```
IoC  
↓  
Broad principle  
  
Dependency Injection  
↓  
Technique used to achieve IoC
```

A strong interview answer:

IoC is the principle where control of object creation and dependency management is transferred to the container, while Dependency Injection is the mechanism through which the container supplies required dependencies to objects.

3. Why do we need Dependency Injection?

The biggest reason is **loose coupling**.

Suppose:

```
class OrderService {  
  
    private PaymentService paymentService = new PaymentService();  
  
}
```

Now `OrderService` is tightly coupled to a concrete implementation.

If we change:

```
PaymentService  
    ↓  
UPIPaymentService
```

we have to modify `OrderService`.

Instead:

```
interface PaymentService {  
    void pay();  
}
```

```
@Service  
class UPIPaymentService implements PaymentService {  
    public void pay() {}  
}
```

```
@Service  
class OrderService {  
  
    private final PaymentService paymentService;  
  
    public OrderService(PaymentService paymentService) {  
        this.paymentService = paymentService;  
    }  
}
```

```
}  
}
```

Now `OrderService` depends on an abstraction.

Benefits

- Loose coupling
- Easier testing
- Easier replacement of implementations
- Better maintainability
- Centralized object configuration
- Supports SOLID principles
- Easier mocking

For example:

```
PaymentService mockPaymentService
```

can be injected during testing.

4. Constructor Injection vs Setter Injection vs Field Injection

Constructor Injection

```
@Service  
public class OrderService {  
  
    private final PaymentService paymentService;  
  
    public OrderService(PaymentService paymentService) {  
        this.paymentService = paymentService;  
    }  
}
```

Dependency is supplied during object creation.

Setter Injection

```

@Service
public class OrderService {

    private PaymentService paymentService;

    @Autowired
    public void setPaymentService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}

```

Dependency is provided after object creation.

Useful when dependency is **optional or can be changed later**.

Field Injection

```

@Service
public class OrderService {

    @Autowired
    private PaymentService paymentService;
}

```

Spring uses reflection to inject the dependency into the private field.

5. Why is Constructor Injection preferred?

This is a **very common 5-year interview question**.

Reason 1 — Dependency is mandatory

```

public OrderService(PaymentService paymentService)

```

You cannot construct the service without its required dependency.

Reason 2 — Supports immutability

```

private final PaymentService paymentService;

```

Once assigned, it cannot be reassigned.

Reason 3 — Easier unit testing

You can simply do:

```
PaymentService mock = mock(PaymentService.class);

OrderService service = new OrderService(mock);
```

No Spring context required.

Reason 4 — Makes dependencies explicit

Look at:

```
public OrderService(
    PaymentService paymentService,
    InventoryService inventoryService,
    OrderRepository repository) {
}
```

You immediately know what `OrderService` requires.

With field injection, dependencies are hidden inside the class.

Reason 5 — Fail fast

If a mandatory dependency is missing, application startup fails instead of discovering the problem later.

Interview answer

Constructor injection is preferred because it makes dependencies explicit and mandatory, supports final fields and immutability, improves testability, and allows Spring to fail fast when a required dependency cannot be resolved.

6. What is the Spring IoC Container?

The Spring IoC container is responsible for:

```
Create beans
  ↓
Configure beans
  ↓
Resolve dependencies
```

↓
Inject dependencies
↓
Manage bean lifecycle
↓
Destroy beans when appropriate

Common containers:

BeanFactory
ApplicationContext

In Spring Boot applications, you normally interact with:

ApplicationContext

rather than directly working with `BeanFactory`.

7. BeanFactory vs ApplicationContext

BeanFactory	ApplicationContext
Basic IoC container	More feature-rich container
Lazy bean creation generally	Supports eager singleton initialization by default
Basic dependency injection	DI + events + i18n + resource loading
Lightweight	More commonly used
Lower-level	Higher-level

Interview answer

BeanFactory provides the basic IoC functionality, while ApplicationContext extends it with additional enterprise features such as application events, internationalization, resource loading, and easier integration with Spring infrastructure.

In Spring Boot, `ApplicationContext` is the normal choice.

8. How does Spring create a bean?

Suppose:

```
@Service
public class PaymentService {
}
```

During startup, Spring scans configured packages.

It finds:

```
@Service
```

and registers a **BeanDefinition**.

Conceptually:

```
@Component scanning
    ↓
Find @Service
    ↓
Create BeanDefinition
    ↓
Register in BeanFactory
    ↓
Instantiate bean
    ↓
Resolve dependencies
    ↓
Inject dependencies
    ↓
Initialize bean
    ↓
Bean ready
```

For example:

```
@Service
class OrderService {

    private final PaymentService paymentService;

    OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Spring recognizes that:

```
OrderService
  ↓
requires PaymentService
```

So it finds the `PaymentService` bean and passes it into the constructor.

9. How does Spring manage dependencies between beans?

Suppose:

```
@Service
class OrderService {

    private final PaymentService paymentService;

    OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Spring maintains a registry of beans.

Conceptually:

```
Bean Container

PaymentService → object
OrderService   → object
```

When creating `orderService`, Spring sees:

```
OrderService requires PaymentService
```

It resolves:

```
PaymentService
  ↓
inject
  ↓
OrderService
```

If `PaymentService` itself has dependencies, Spring resolves those recursively.

```
OrderService
  |
  ↓
PaymentService
  |
  ↓
PaymentRepository
```

Spring builds the dependency graph.

10. What happens when two beans have the same type?

This is extremely important.

Suppose:

```
interface PaymentService {
    void pay();
}
```

You have:

```
@Service
class UpiPaymentService implements PaymentService {
}
```

and:

```
@Service
class CardPaymentService implements PaymentService {
}
```

Now:

```
@Service
class OrderService {

    private final PaymentService paymentService;

    OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Spring sees:

```
PaymentService
|
+----- UpiPaymentService
|
+----- CardPaymentService
```

It cannot determine which one to inject.

You generally get:

```
NoUniqueBeanDefinitionException
```

11. @Autowired vs @Qualifier

`@Autowired` tells Spring:

Find a suitable bean for this dependency.

Example:

```
@Autowired
private PaymentService paymentService;
```

If there is only one implementation, it works.

But if there are multiple implementations, use:

```
@Autowired
@Qualifier("upiPaymentService")
private PaymentService paymentService;
```

Or preferably constructor injection:

```
public OrderService(
    @Qualifier("upiPaymentService")
    PaymentService paymentService) {

    this.paymentService = paymentService;
}
```

12. @Primary vs @Qualifier

Suppose:

```
@Service
@Primary
class UpiPaymentService implements PaymentService {
}
```

and:

```
@Service
class CardPaymentService implements PaymentService {
}
```

Now:

```
OrderService(PaymentService paymentService)
```

will receive:

```
UpiPaymentService
```

because it is marked `@Primary`.

What if I want CardPaymentService?

Use:

```
OrderService(
    @Qualifier("cardPaymentService")
    PaymentService paymentService
)
```

Key difference

`@Primary`:

Default choice when multiple candidates exist.

`@Qualifier`:

Explicitly select this particular bean.

Priority

Think:

```
@Qualifier
  ↓
explicit choice

@Primary
  ↓
default choice
```

If a qualifier identifies a bean, it takes precedence over simply choosing the primary candidate.

13. Can Spring inject private fields?

Yes.

Example:

```
@Service
public class OrderService {

    @Autowired
    private PaymentService paymentService;
}
```

Spring can perform field injection using reflection.

It doesn't require the field to be public.

But should you use it?

Generally, **no for mandatory dependencies**.

Prefer:

```
private final PaymentService paymentService;

public OrderService(PaymentService paymentService) {
    this.paymentService = paymentService;
}
```

Interview answer

Yes, Spring can inject private fields through reflection, but constructor injection is generally preferred because it makes dependencies explicit, supports immutability, and improves

14. Can Spring inject a bean through a constructor?

Yes. This is the preferred approach.

```
@Service
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

In modern Spring, if the class has **a single constructor**, you generally don't need `@Autowired`.

So this is enough:

```
public OrderService(PaymentService paymentService) {
    this.paymentService = paymentService;
}
```

You don't need:

```
@Autowired
public OrderService(...)
```

for the single-constructor case.

15. What happens if a dependency is missing?

Suppose:

```
@Service
class OrderService {

    OrderService(PaymentService paymentService) {
    }
}
```

But Spring cannot find a bean of type:

```
PaymentService
```

Then application startup normally fails with an error such as:

```
NoSuchBeanDefinitionException
```

or, depending on the resolution path/context, a bean-creation/dependency-resolution exception wrapping that problem.

The important interview point:

```
Required dependency missing
      ↓
Spring cannot create dependent bean
      ↓
Application context fails to start
```

This is one reason constructor injection is useful: **fail fast**.

16. What happens if multiple beans match the same interface?

Example:

```
interface NotificationService {
}
```

```
@Service
class EmailNotificationService implements NotificationService {
}
```

```
@Service
class SmsNotificationService implements NotificationService {
}
```

Then:

```
NotificationService service
```

has two candidates.

Spring cannot choose automatically.

You need one of:

@Qualifier

```
public OrderService(  
    @Qualifier("emailNotificationService")  
    NotificationService service) {  
}
```

@Primary

```
@Service  
@Primary  
class EmailNotificationService  
    implements NotificationService {  
}
```

Then:

```
NotificationService service
```

will use the primary bean.

17. How would you conditionally create a bean?

This is a **good 5-year Spring Boot question**.

Spring Boot provides several mechanisms.

@ConditionalOnProperty

Very common.

```
@Configuration  
public class PaymentConfig {  
  
    @Bean  
    @ConditionalOnProperty(  
        name = "payment.enabled",  
        havingValue = "true"  
    )  
    public PaymentService paymentService() {  
        return new PaymentService();  
    }  
}
```

Configuration:

```
payment.enabled=true
```

Then bean is created.

If:

```
payment.enabled=false
```

the bean isn't created.

@Profile

You can create beans for specific environments.

```
@Bean
@Profile("dev")
public PaymentService devPaymentService() {
    return new MockPaymentService();
}
```

And:

```
@Bean
@Profile("prod")
public PaymentService prodPaymentService() {
    return new RealPaymentService();
}
```

So:

```
dev → MockPaymentService
```

```
prod → RealPaymentService
```

@ConditionalOnMissingBean

Very common when building reusable Spring Boot configuration.

```
@Bean
@ConditionalOnMissingBean
public PaymentService paymentService() {
    return new DefaultPaymentService();
}
```

Meaning:

Create this bean only if another bean of this type hasn't already been provided.

This is heavily used in Spring Boot auto-configuration.

5-Year Interview Scenario

Interviewer:

We have three implementations of `PaymentService`: UPI, Card, and Wallet. The application should use UPI by default, but certain services should use Card. How would you design it?

Good answer:

```
@Service
@Primary
class UpiPaymentService implements PaymentService {
}
```

Then the normal injection:

```
public PaymentProcessor(PaymentService paymentService) {
    this.paymentService = paymentService;
}
```

gets UPI.

For a service requiring Card:

```
public CardOrderService(
    @Qualifier("cardPaymentService")
    PaymentService paymentService) {

    this.paymentService = paymentService;
}
```

So:

```
Default
  ↓
@Primary
  ↓
UPI

Specific requirement
```

↓
@Qualifier
↓
Card

🔥 The Interview Mental Model

Remember this entire topic as:

```

      IoC
      |
  Spring controls
      |
  Object creation
      |
  Dependency management
      |
      ↓
  Dependency Injection
      |
+-----+-----+
  |         |         |
Constructor Setter  Field
  |
  ↓
Preferred
  |
+----- explicit dependency
+----- final fields
+----- immutable
+----- testable
+----- fail fast
```

And for multiple beans:

```

One interface
  |
+----- Bean A
  |
+----- Bean B
  |
  ↓
Multiple candidates
  |
```

```
+---- @Qualifier → explicit bean
|
+---- @Primary   → default bean
```

And conditional beans:

```
Should bean exist?
|
+---- @Profile
|
+---- @ConditionalOnProperty
|
+---- @ConditionalOnMissingBean
```

★ Questions you should be ready for next

For a **5-year Spring Boot interview**, the interviewer can easily continue with:

1. What exactly happens during Spring bean lifecycle?
2. What is a BeanDefinition?
3. What is component scanning?
4. `@Component` vs `@Service` vs `@Repository` ?
5. What happens internally when `@Autowired` is used?
6. How does Spring resolve constructor dependencies?
7. What is circular dependency?
8. Can constructor injection create circular dependency?
9. What is `@Lazy` and when would you use it?
10. Singleton vs prototype bean scope?
11. What happens when a singleton bean depends on a prototype bean?
12. How does Spring handle circular dependencies?
13. `@Bean` vs `@Component` ?
14. `@Configuration` and proxy behavior?
15. What is Spring BeanPostProcessor?
16. What is `BeanFactoryPostProcessor` ?
17. What is `ApplicationContext` startup sequence?
18. How does Spring Boot auto-configuration work?

19. How does `@ConditionalOnMissingBean` work internally?
20. What happens if two `@Configuration` classes define beans with the same name?



What is a Spring Bean?
How does Spring create a bean?
Explain the complete bean lifecycle.
What is dependency injection during bean creation?
@PostConstruct.
@PreDestroy.
InitializingBean.
DisposableBean.
init-method.
destroy-method.
Which happens first: constructor or @PostConstruct?
When can a bean be used after initialization?
What happens during application shutdown?
What is BeanPostProcessor?
How does Spring use BeanPostProcessor internally?
Explain what happens from the moment Spring Boot starts until your controller receives the first request



Yes. This is a **very important 5-year Spring Boot interview area** because interviewers often start with “What is a bean?” and then keep drilling into the **actual lifecycle and startup sequence**.

1. What is a Spring Bean?

A **Spring Bean** is an object that is created, configured, and managed by the Spring IoC container.

For example:

```
@Service
public class OrderService {
}
```

Spring sees `@Service`, creates an instance of `OrderService`, and manages its lifecycle.

You can also explicitly create a bean:

```
@Configuration
public class AppConfig {

    @Bean
```

```
public PaymentService paymentService() {  
    return new PaymentService();  
}  
}
```

Here, the object returned by `paymentService()` becomes a Spring Bean.

Interview answer

A Spring Bean is an object whose creation, dependency injection, initialization, lifecycle, and destruction are managed by the Spring IoC container.

2. How does Spring create a bean?

Consider:

```
@Service  
public class OrderService {  
  
    private final PaymentService paymentService;  
  
    public OrderService(PaymentService paymentService) {  
        this.paymentService = paymentService;  
    }  
}
```

During application startup, Spring roughly does:

```
Spring Boot starts  
↓  
ApplicationContext created  
↓  
Component scanning  
↓  
@Service discovered  
↓  
BeanDefinition created  
↓  
BeanDefinition registered  
↓  
Spring decides bean needs to be created  
↓  
Resolve dependencies  
↓
```

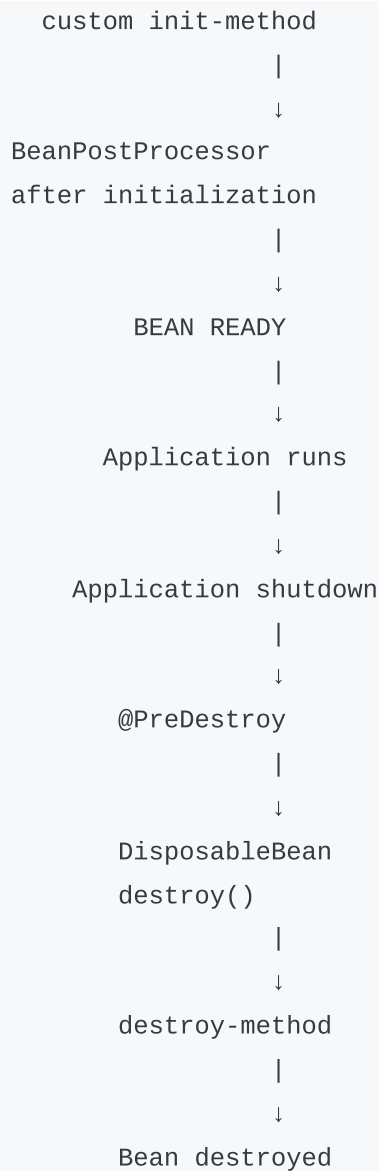
```
Create object using constructor
↓
Inject dependencies
↓
Initialize bean
↓
BeanPostProcessor processing
↓
Bean becomes ready
```

The exact internal path is more complex, but this is the correct interview-level mental model.

3. Complete Spring Bean Lifecycle

This is one of the most important diagrams to remember.

```
Spring Container
|
↓
BeanDefinition
|
↓
Instantiate Bean
|
↓
Constructor executes
|
↓
Dependency Injection
|
↓
Aware callbacks (if applicable)
|
↓
BeanPostProcessor
before initialization
|
↓
@PostConstruct
|
↓
InitializingBean
afterPropertiesSet()
|
↓
```



There are some additional callback interfaces such as `*Aware` interfaces, which you should know exist but don't usually need to implement in normal application code.

4. What is dependency injection during bean creation?

Suppose:

```
@Service
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Spring cannot simply do:

```
new OrderService();
```

because the constructor requires:

```
PaymentService
```

Spring first resolves the dependency:

```
OrderService
  |
  ↓
requires PaymentService
  |
  ↓
Spring finds PaymentService bean
  |
  ↓
creates/gets PaymentService
  |
  ↓
passes it to OrderService constructor
```

Conceptually:

```
PaymentService payment = container.getBean(PaymentService.class);

OrderService order =
    new OrderService(payment);
```

The actual Spring internals are more sophisticated, but this is the right conceptual explanation.

5. Which happens first: Constructor or `@PostConstruct` ?

Constructor happens first.

Example:

```
@Component
public class MyBean {

    public MyBean() {
        System.out.println("Constructor");
    }
}
```

```

@PostConstruct
public void init() {
    System.out.println("PostConstruct");
}
}

```

Output:

```

Constructor
PostConstruct

```

Why?

Because Spring must first create the object.

```

Constructor
  ↓
Object exists
  ↓
Dependencies populated
  ↓
@PostConstruct

```

6. What is `@PostConstruct` ?

`@PostConstruct` is used for initialization logic that should execute **after dependency injection has completed** and before the bean is ready for normal use.

Example:

```

@Component
public class CacheService {

    private final ConfigService configService;

    public CacheService(ConfigService configService) {
        this.configService = configService;
    }

    @PostConstruct
    public void initialize() {
        System.out.println("Loading cache...");
    }
}

```

```
}  
}
```

At this point:

```
constructor  
  ↓  
dependency injected  
  ↓  
@PostConstruct
```

So `@PostConstruct` is useful when initialization needs already-injected dependencies.

Important interview point

Don't put logic requiring injected dependencies inside the constructor if that logic is conceptually initialization work that belongs after dependency injection.

7. What is `InitializingBean` ?

Spring provides:

```
InitializingBean
```

with:

```
afterPropertiesSet()
```

Example:

```
@Component  
public class CacheService implements InitializingBean {  
  
    @Override  
    public void afterPropertiesSet() {  
        System.out.println("Initializing...");  
    }  
}
```

It is another lifecycle callback.

Conceptually:

```
@PostConstruct  
  ↓
```

```
afterPropertiesSet()
```

when considering these standard initialization callbacks.

Which should you prefer?

Usually:

```
@PostConstruct
```

because it keeps your application class less coupled to Spring-specific lifecycle interfaces.

8. What is `init-method`?

You can configure an initialization method explicitly.

```
@Configuration
public class AppConfig {

    @Bean(initMethod = "initialize")
    public PaymentService paymentService() {
        return new PaymentService();
    }
}
```

Class:

```
public class PaymentService {

    public void initialize() {
        System.out.println("Initializing payment service");
    }
}
```

Spring calls:

```
initialize()
```

during bean initialization.

This can be useful when you're configuring a class you don't want to modify, such as a third-party class.

9. Initialization order

For the commonly discussed callbacks, remember:

```
Constructor
  ↓
Dependency Injection
  ↓
@PostConstruct
  ↓
InitializingBean.afterPropertiesSet()
  ↓
init-method
  ↓
Bean ready
```

But there is an important detail:

BeanPostProcessor callbacks surround initialization.

More accurately:

```
Constructor
  ↓
Dependency injection
  ↓
BeanPostProcessor
postProcessBeforeInitialization()
  ↓
@PostConstruct
  ↓
InitializingBean.afterPropertiesSet()
  ↓
init-method
  ↓
BeanPostProcessor
postProcessAfterInitialization()
  ↓
Bean ready
```

This is the better diagram to give in a senior interview.

10. When can a bean be used after initialization?

After Spring has completed its bean creation and initialization processing.

Conceptually:

```
Bean instantiated
  ↓
```

```
Dependencies injected
  ↓
Initialization callbacks
  ↓
BeanPostProcessor processing
  ↓
READY
```

After that, Spring can expose the bean to other components and the application can use it. For singleton beans, most of them are created during application context startup by default, although lazy initialization can change when a bean is instantiated.

11. What is `@PreDestroy` ?

`@PreDestroy` is called before Spring destroys a bean.

Example:

```
@Component
public class CacheService {

    @PreDestroy
    public void cleanup() {
        System.out.println("Cleaning cache...");
    }
}
```

It is commonly used for cleanup operations.

For example:

```
close resources
stop background processing
flush buffers
cleanup state
```

Don't use it for something that must survive a forced JVM termination; normal Spring shutdown is the lifecycle in which these callbacks are expected.

12. What is `DisposableBean` ?

Similar to `InitializingBean`, Spring provides a destruction callback:

```
DisposableBean
```

with:

```
destroy()
```

Example:

```
@Component
public class CacheService implements DisposableBean {

    @Override
    public void destroy() {
        System.out.println("Destroying...");
    }
}
```

Conceptually:

```
Bean ready
  ↓
Application running
  ↓
Shutdown
  ↓
destroy callbacks
```

Again, application code usually prefers annotation/configuration approaches over coupling classes to Spring lifecycle interfaces.

13. What is `destroy-method`?

You can configure a custom destruction method:

```
@Configuration
public class AppConfig {

    @Bean(destroyMethod = "cleanup")
    public PaymentService paymentService() {
        return new PaymentService();
    }
}
```

Class:

```
public class PaymentService {

    public void cleanup() {
        System.out.println("Cleanup");
    }
}
```

Spring calls:

```
cleanup()
```

when the context is closed.

14. Destruction order

For the commonly discussed callbacks:

```
Application shutdown
    ↓
@PreDestroy
    ↓
DisposableBean.destroy()
    ↓
destroy-method
    ↓
Bean destroyed
```

So the overall lifecycle you can give in an interview is:

```
CREATE
    |
    ↓
Constructor
    |
    ↓
Dependency Injection
    |
    ↓
postProcessBeforeInitialization()
    |
    ↓
@PostConstruct
    |
    ↓
```



15. What is `BeanPostProcessor` ?

This is a **very important senior-level Spring concept**.

`BeanPostProcessor` allows Spring or your application to execute logic **before and after bean initialization**.

Interface:

```
public interface BeanPostProcessor {  
  
    Object postProcessBeforeInitialization(  
        Object bean,  
        String beanName);  
  
    Object postProcessAfterInitialization(  

```

```
        Object bean,  
        String beanName);  
    }
```

For example:

```
@Component  
public class MyBeanPostProcessor  
    implements BeanPostProcessor {  
  
    @Override  
    public Object postProcessBeforeInitialization(  
        Object bean,  
        String beanName) {  
  
        System.out.println("Before: " + beanName);  
        return bean;  
    }  
  
    @Override  
    public Object postProcessAfterInitialization(  
        Object bean,  
        String beanName) {  
  
        System.out.println("After: " + beanName);  
        return bean;  
    }  
}
```

16. Why does Spring use BeanPostProcessor?

Because many Spring features require Spring to inspect, modify, wrap, or enhance beans.

Examples include infrastructure around:

- `@Autowired`
- `@PostConstruct`
- AOP
- transactions
- security
- proxies
- other annotation-driven features

A key interview point:

`BeanPostProcessor` is one of the mechanisms that makes Spring's annotation-driven and proxy-based behavior possible.

17. How does Spring use BeanPostProcessor internally?

This is where interviewers may go deeper.

Suppose:

```
@Service
public class OrderService {

    @Transactional
    public void createOrder() {
    }
}
```

Spring may need to create a proxy around `OrderService`.

Conceptually:

```
Original OrderService
    |
    ↓
BeanPostProcessor
    |
    ↓
Proxy created
    |
    ↓
Application gets proxy
```

So when your application calls:

```
orderService.createOrder();
```

it may actually be:

```
Application
    ↓
OrderService proxy
    ↓
transaction handling
```

↓
real OrderService

This is a major reason `BeanPostProcessor` is important.

18. `BeanPostProcessor` VS `BeanFactoryPostProcessor`

Another common follow-up.

BeanPostProcessor

Works primarily with **bean instances**.

BeanDefinition
↓
Bean instance
↓
BeanPostProcessor

BeanFactoryPostProcessor

Works with **BeanDefinitions / container metadata before beans are instantiated**.

BeanDefinition
↓
BeanFactoryPostProcessor
↓
Modified metadata
↓
Bean creation

Remember:

BeanFactoryPostProcessor
↓
Bean definitions

BeanPostProcessor
↓
Bean instances

19. What happens when Spring Boot starts?

Now let's answer the big question.

Explain what happens from the moment Spring Boot starts until your controller receives the first request.

This is an excellent 5-year interview question.

Step 1 — JVM starts

You run:

```
java -jar application.jar
```

The JVM starts and loads your application classes.

Step 2 — `main()` executes

Typical Spring Boot application:

```
@SpringBootApplication
public class Application {

    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

Execution reaches:

```
SpringApplication.run(...)
```

Step 3 — SpringApplication is initialized

Spring Boot determines things such as:

- Application configuration
- Environment
- Profiles
- Application listeners
- Initializers
- ApplicationContext type

For a typical web application, Spring Boot creates an appropriate web application context.

Step 4 — Environment and configuration are prepared

Spring Boot loads configuration from sources such as:

```
application.properties
application.yml
environment variables
command-line arguments
profiles
```

For example:

```
server:
  port: 8080

spring:
  datasource:
    url: ...
```

These values become part of Spring's `Environment`.

Step 5 — ApplicationContext is created

Spring creates the IoC container.

Conceptually:

```
ApplicationContext
|
+--- BeanFactory
+--- Environment
+--- Bean definitions
+--- Event infrastructure
```

Step 6 — Component scanning happens

Because you have:

```
@SpringBootApplication
```

which includes component-scanning behavior, Spring scans the relevant packages.

It discovers:

```
@Component
@Service
@Repository
@Controller
@RestController
@Configuration
```

and other registered configuration sources.

Step 7 — BeanDefinitions are registered

Spring doesn't immediately just create every object when it sees a class.

It first creates metadata describing beans.

Conceptually:

```
OrderService
-----
Bean name: orderService
Class: OrderService
Scope: singleton
Dependencies: PaymentService
...
```

This metadata is a **BeanDefinition**.

Step 8 — Spring processes configuration

Spring processes configuration and registers additional beans.

For example:

```
@Configuration
public class AppConfig {

    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }
}
```

The resulting bean is registered in the container.

Spring Boot also processes auto-configuration.

For example, depending on your dependencies and configuration, Boot can automatically configure things such as:

```
DataSource
Tomcat
Jackson
MVC infrastructure
Transaction infrastructure
etc.
```

Step 9 — Bean creation begins

For non-lazy singleton beans, Spring generally creates them during context startup.

Suppose:

```
@RestController
public class OrderController {

    private final OrderService orderService;

    public OrderController(OrderService orderService) {
        this.orderService = orderService;
    }
}
```

Spring needs:

```
OrderController
    ↓
OrderService
```

So it resolves the dependency graph.

Maybe:

```
OrderController
    ↓
OrderService
    ↓
PaymentService
    ↓
PaymentRepository
```

Spring creates dependencies as necessary.

Step 10 — Constructor executes

For:

```
public OrderService(PaymentService paymentService)
```

Spring resolves `PaymentService` and calls the constructor.

Conceptually:

```
new OrderService(paymentService);
```

Step 11 — Dependencies are injected

For constructor injection, dependencies are already passed through the constructor.

For field/setter injection, Spring performs the relevant injection afterward.

Step 12 — BeanPostProcessor executes

Spring runs:

```
postProcessBeforeInitialization()
```

Then initialization callbacks:

```
@PostConstruct  
afterPropertiesSet()  
init-method
```

Then:

```
postProcessAfterInitialization()
```

Some processors can return a different object, such as a proxy.

For example:

```
Real Service  
  ↓  
Post Processor  
  ↓  
Transactional Proxy
```

↓
Container

Step 13 — Web infrastructure starts

For a Spring Boot MVC application, the embedded web server starts.

Typically:

```
Spring Boot
  ↓
Embedded Servlet Container
  ↓
Tomcat
  ↓
HTTP port 8080
```

Spring MVC infrastructure is initialized, including the components responsible for mapping incoming HTTP requests to controllers.

Step 14 — Application becomes ready

Once startup completes, Spring publishes application lifecycle events and the application is considered ready to receive traffic.

You can observe this with mechanisms such as:

ApplicationReadyEvent

For example:

```
@Component
public class StartupListener {

    @EventListener
    public void onReady(ApplicationReadyEvent event) {
        System.out.println("Application is ready");
    }
}
```

Step 15 — Client sends first HTTP request

Suppose:

```
GET /orders/100
```

Request reaches:

```
Client
  ↓
Load Balancer
  ↓
Application server
  ↓
Embedded Tomcat
```

Then Spring MVC processes it.

Step 16 — Request reaches DispatcherServlet

A very important Spring MVC component is:

```
DispatcherServlet
```

Conceptually:

```
HTTP Request
  ↓
Tomcat
  ↓
DispatcherServlet
```

`DispatcherServlet` acts as the front controller for Spring MVC.

Step 17 — DispatcherServlet finds the controller

Suppose:

```
@RestController
@RequestMapping("/orders")
public class OrderController {

    @GetMapping("/{id}")
    public Order getOrder(@PathVariable Long id) {
        return orderService.getOrder(id);
    }
}
```

Spring's request-mapping infrastructure determines:

```
GET /orders/100
  ↓
OrderController.getOrder(100)
```

Step 18 — Controller calls service

```
DispatcherServlet
  ↓
OrderController
  ↓
OrderService
  ↓
Repository
  ↓
Database
```

If the service is proxied by Spring AOP:

```
OrderController
  ↓
OrderService Proxy
  ↓
Transaction / AOP logic
  ↓
Real OrderService
```

Step 19 — Controller returns response

Suppose the controller returns:

```
return order;
```

Because it's a REST controller, Spring MVC uses message conversion, commonly Jackson for JSON, to serialize the object.

Conceptually:

```
Order Java object
  ↓
Jackson
  ↓
```

JSON



HTTP Response

Example:

```
{  
  "id": 100,  
  "status": "CONFIRMED"  
}
```



Complete Startup + First Request Flow

This is the diagram I'd recommend memorizing for a 5-year interview:

```
JVM starts  
↓  
main()  
↓  
SpringApplication.run()  
↓  
Create Environment  
↓  
Load application.properties / YAML / profiles  
↓  
Create ApplicationContext  
↓  
Component scanning  
↓  
Configuration processing  
↓  
BeanDefinitions registered  
↓  
Auto-configuration  
↓  
Create singleton beans  
↓  
Resolve dependencies  
↓  
Constructor  
↓  
Dependency Injection  
↓  
BeanPostProcessor
```



before initialization



@PostConstruct



InitializingBean



init-method



BeanPostProcessor

after initialization



Possible proxy creation



Start embedded server



Initialize Spring MVC



ApplicationReadyEvent



Application READY



FIRST HTTP REQUEST



Client



Load Balancer



Tomcat



DispatcherServlet



HandlerMapping



Controller



Service



Repository



Database



Service



```
Controller
  ↓
HttpMessageConverter / Jackson
  ↓
HTTP Response
  ↓
Client
```

🔥 What happens during application shutdown?

When the Spring application context is closed, Spring performs destruction processing for beans that it manages.

Conceptually:

```
SIGTERM / graceful shutdown
  ↓
Spring ApplicationContext closes
  ↓
@PreDestroy
  ↓
DisposableBean.destroy()
  ↓
destroy-method
  ↓
Beans destroyed
  ↓
Web server shuts down
  ↓
Application exits
```

For production systems, this matters because you may need to gracefully:

```
finish in-flight requests
stop accepting new requests
close database connections
shutdown executors
flush resources
close clients
```

★ Most important interview distinction

If the interviewer asks:

What is the order of Spring bean initialization?

Answer:

```
Constructor
  ↓
Dependency Injection
  ↓
BeanPostProcessor.before
  ↓
@PostConstruct
  ↓
InitializingBean.afterPropertiesSet()
  ↓
init-method
  ↓
BeanPostProcessor.after
  ↓
Bean ready
```

And destruction:

```
@PreDestroy
  ↓
DisposableBean.destroy()
  ↓
destroy-method
```

Follow-up questions interviewer can ask

Once you answer this topic, a 5-year interviewer will very likely move to:

Bean lifecycle

- What is the difference between `@PostConstruct` and constructor?
- Can you access an injected dependency inside the constructor?
- What happens if `@PostConstruct` throws an exception?
- Does Spring call `@PreDestroy` if the JVM is killed forcefully?
- Does Spring manage the lifecycle of prototype beans?
- What is the lifecycle of a request-scoped bean?

BeanPostProcessor

- How does `@Autowired` work internally?
- How does `@Transactional` create a proxy?
- What is the difference between JDK dynamic proxy and CGLIB?
- What is `BeanFactoryPostProcessor` ?
- What happens if a `BeanPostProcessor` returns a different object?

Spring Boot startup

- What exactly does `@SpringBootApplication` contain?
- How does component scanning work?
- How does Spring Boot auto-configuration work?
- What is `spring.factories` vs `AutoConfiguration.imports` in modern Spring Boot?
- How does Spring resolve circular dependencies?
- Why can constructor injection fail with circular dependencies?
- When are singleton beans created?
- What changes when `@Lazy` is used?
- How does Spring MVC map `/orders/{id}` to a controller method?

Those are the questions that take this topic from **basic Spring knowledge to 5-year interview level**.



. Bean Scopes

What is singleton scope?

Is Spring singleton the same as Singleton Design Pattern?

Prototype scope.

Request scope.

Session scope.

Application scope.

WebSocket scope.

What happens when a singleton bean depends on a prototype bean?

How can you inject a prototype bean into a singleton correctly?

ObjectProvider / Provider



Spring Bean Scopes — 5-Year Interview Guide

Bean scope answers one fundamental question:

How many objects of a particular Spring Bean should Spring create, and how long should those objects live?

The most important scopes for a 5-year Spring Boot interview are:

```
singleton
prototype
request
session
application
websocket
```

1. What is Singleton Scope?

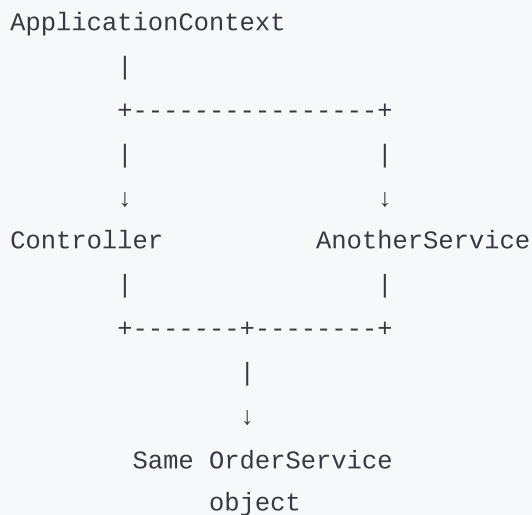
`singleton` is the **default Spring bean scope**.

```
@Service
public class OrderService {
}
```

is effectively:

```
@Service
@Scope("singleton")
public class OrderService {
}
```

For a given Spring `ApplicationContext`, Spring normally creates **one bean instance** and reuses it wherever that bean is injected.



Example:

```
@Service
public class PaymentService {
}
```

Then:

```
@Autowired
private PaymentService paymentService;
```

in multiple beans normally points to the same Spring-managed instance.

Important

Singleton does **not** mean:

Only one object of this class can ever exist in the JVM.

It means:

Spring maintains one instance of that bean within a particular `ApplicationContext`.

2. Is Spring Singleton the same as Singleton Design Pattern?

No.

This is a very common interview trap.

Singleton Design Pattern

The class itself controls object creation.

For example:

```
public class MySingleton {  
  
    private static final MySingleton INSTANCE =  
        new MySingleton();  
  
    private MySingleton() {  
    }  
  
    public static MySingleton getInstance() {  
        return INSTANCE;  
    }  
}
```

The class guarantees the singleton.

Spring Singleton

Spring controls the lifecycle:

```
@Service  
public class PaymentService {  
}
```

Spring's container manages the instance.

You don't write:

```
PaymentService.getInstance();
```

Instead:

```
@Service  
public class OrderService {
```

```

private final PaymentService paymentService;

public OrderService(PaymentService paymentService) {
    this.paymentService = paymentService;
}
}

```

Key difference

Spring Singleton	Singleton Design Pattern
Container-managed	Class-managed
One instance per ApplicationContext	Usually one instance per JVM/classloader design
No private constructor required	Usually private constructor
Dependency injection supported	Must implement dependency management yourself
Scope mechanism	Creational design pattern

Excellent interview answer

Spring singleton scope and the Singleton Design Pattern are different concepts. Spring singleton means one bean instance per Spring ApplicationContext, whereas the Singleton Design Pattern puts the responsibility of enforcing a single instance inside the class itself.

3. Prototype Scope

With prototype scope:

```

@Component
@Scope("prototype")
public class ReportGenerator {
}

```

Spring creates a **new instance whenever the bean is requested from the container**.

For example:

```

ReportGenerator r1 =
    applicationContext.getBean(ReportGenerator.class);

ReportGenerator r2 =
    applicationContext.getBean(ReportGenerator.class);

```

Conceptually:

```
getBean()  
  ↓  
New object  
  
getBean()  
  ↓  
Another new object
```

So:

```
r1 != r2
```

4. Singleton vs Prototype

Singleton:

```
getBean()  
  ↓  
Object A  
  
getBean()  
  ↓  
Object A  
  
getBean()  
  ↓  
Object A
```

Prototype:

```
getBean()  
  ↓  
Object A  
  
getBean()  
  ↓  
Object B  
  
getBean()  
  ↓  
Object C
```

Important interview point

Prototype doesn't mean:

Spring creates a new object every time you call a normal method.

It means Spring creates a new instance when the prototype bean is **requested from the container**.

5. Request Scope

Request scope is used in web applications.

```
@Component
@RequestScope
public class RequestContext {
}
```

Spring creates one instance **for each HTTP request**.

Suppose:

```
Request 1
  ↓
RequestContext A

Request 2
  ↓
RequestContext B

Request 3
  ↓
RequestContext C
```

So:

```
one HTTP request
  ↓
one bean instance
```

Useful for request-specific information such as:

```
request metadata
correlation ID
```

request-specific state

Important

Request scope only makes sense when a web request context exists.

6. Session Scope

Session-scoped bean:

```
@Component
@SessionScope
public class UserSessionData {
}
```

Spring maintains one instance for a particular HTTP session.

Conceptually:

```
User A / Session A
    ↓
UserSessionData A

User B / Session B
    ↓
UserSessionData B
```

The same user/session gets the same bean instance during that session.

7. Application Scope

Application scope is associated with the **ServletContext**.

Example:

```
@Component
@ApplicationScope
public class ApplicationData {
}
```

Conceptually:

```
ServletContext
|
```

↓
ApplicationData

It lives for the lifecycle of the web application's `ServletContext`.

Important interview distinction

Don't simply say:

Application scope = singleton.

They can behave similarly in a simple Spring Boot application, but they are **different scope concepts**.

- `singleton` → one instance per Spring `ApplicationContext`
- `application` → one instance per Servlet `ServletContext`

8. WebSocket Scope

WebSocket scope is associated with a WebSocket session.

Conceptually:

```
WebSocket Session A
    ↓
Bean A

WebSocket Session B
    ↓
Bean B
```

The bean lives for the lifecycle of that WebSocket session.

This is less commonly used than singleton/request/session scopes, but it can come up in senior interviews.

9. Scope Summary

Scope	Instance
Singleton	One per Spring <code>ApplicationContext</code>
Prototype	New instance when requested

Scope	Instance
Request	One per HTTP request
Session	One per HTTP session
Application	One per Servlet <code>ServletContext</code>
WebSocket	One per WebSocket session

Remember:

```

singleton → application context
prototype → every container request
request   → HTTP request
session   → HTTP session
application → ServletContext
websocket → WebSocket session

```

10. What happens when a Singleton depends on a Prototype?

🔥 This is one of the most important questions in Spring scopes.

Suppose:

```

@Component
@Scope("prototype")
public class ReportGenerator {
}

```

and:

```

@Service
public class ReportService {

    private final ReportGenerator generator;

    public ReportService(ReportGenerator generator) {
        this.generator = generator;
    }
}

```

`ReportService` is singleton.

`ReportGenerator` is prototype.

What happens?

Many people incorrectly answer:

A new `ReportGenerator` is created every time `ReportService` is used.

That's wrong.

Spring creates the singleton `ReportService` once.

During its creation, Spring needs `ReportGenerator`.

So it gets **one prototype instance** and injects it.

```
Application startup  
  
Create ReportService  
    ↓  
Need ReportGenerator  
    ↓  
Create Prototype A  
    ↓  
Inject A  
    ↓  
ReportService keeps A
```

Later:

```
Request 1 → ReportService → Prototype A  
  
Request 2 → ReportService → Prototype A  
  
Request 3 → ReportService → Prototype A
```

So the prototype dependency effectively behaves like a singleton **from the perspective of that singleton's injected field**.

11. Why does this happen?

Because constructor injection happens **when the singleton is created**.

Spring doesn't automatically re-run the constructor every time you use:

```
reportService.generate();
```

Therefore:

```
private final ReportGenerator generator;
```

continues pointing to the same object.

12. How can you correctly inject a Prototype into a Singleton?

If you actually need a **new prototype instance every time you need one**, don't directly inject the prototype instance.

Instead, inject something that can ask Spring for the prototype.

There are several approaches.

13. ObjectProvider

This is one of the best Spring-native solutions.

```
@Component
@Scope("prototype")
public class ReportGenerator {
}
```

Singleton:

```
@Service
public class ReportService {

    private final ObjectProvider<ReportGenerator> provider;

    public ReportService(ObjectProvider<ReportGenerator> provider) {
        this.provider = provider;
    }

    public void generate() {

        ReportGenerator generator =
            provider.getObject();

        // use generator
    }
}
```

Now:

```
provider.getObject()
```

asks Spring for a prototype bean.

So:

```
Request 1
  ↓
provider.getObject()
  ↓
Prototype A

Request 2
  ↓
provider.getObject()
  ↓
Prototype B

Request 3
  ↓
provider.getObject()
  ↓
Prototype C
```

assuming the prototype is requested each time.

14. Why does **ObjectProvider** solve the problem?

Instead of:

```
Singleton
  |
  ↓
Prototype instance
```

you have:

```
Singleton
  |
  ↓
ObjectProvider
  |
  ↓
Spring Container
  |
```

```
+----- Prototype A
+----- Prototype B
+----- Prototype C
```

The singleton doesn't hold one prototype instance permanently.

It holds a **provider capable of obtaining prototype instances**.

15. `javax.inject.Provider` / `jakarta.inject.Provider`

You may also encounter:

```
Provider<ReportGenerator>
```

depending on the dependency/API version being used.

Example:

```
@Service
public class ReportService {

    private final Provider<ReportGenerator> provider;

    public ReportService(Provider<ReportGenerator> provider) {
        this.provider = provider;
    }

    public void generate() {
        ReportGenerator generator =
            provider.get();
    }
}
```

The idea is the same:

```
Provider
  ↓
ask container
  ↓
new prototype instance
```

ObjectProvider vs Provider

`ObjectProvider` is Spring-specific and provides additional convenience methods such as optional lookup capabilities.

`Provider` is based on the Jakarta/JSR-style dependency injection API and is less tied to Spring.

16. Another solution — `@Lookup`

Spring also provides `@Lookup`.

Example:

```
@Component
@Scope("prototype")
public class ReportGenerator {
}
```

Then:

```
@Service
public abstract class ReportService {

    public void generate() {

        ReportGenerator generator =
            getReportGenerator();

        // use generator
    }

    @Lookup
    protected abstract ReportGenerator getReportGenerator();
}
```

Spring overrides/intercepts the lookup method so that the container can provide the appropriate bean.

Conceptually:

```
generate()
  ↓
getReportGenerator()
  ↓
Spring Container
  ↓
new prototype instance
```

This works, but `ObjectProvider` is often easier to understand and test.

17. What about `ApplicationContext.getBean()`?

You might see:

```
@Service
public class ReportService {

    private final ApplicationContext context;

    public ReportService(ApplicationContext context) {
        this.context = context;
    }

    public void generate() {

        ReportGenerator generator =
            context.getBean(ReportGenerator.class);
    }
}
```

This can work.

But it's generally less desirable because your business class becomes tightly coupled to the Spring container.

Compare:

```
context.getBean(...)
```

with:

```
ObjectProvider<ReportGenerator>
```

`ObjectProvider` communicates the dependency more clearly and is generally preferable.

18. Which solution would I choose?

For a 5-year interview:

If a singleton needs a fresh prototype instance on demand, I would typically use `ObjectProvider<PrototypeBean>` or `Provider<PrototypeBean>`. `@Lookup` is another Spring-supported option. I would avoid directly injecting the prototype instance because that instance is resolved when the singleton is created and then reused.

That's a **strong answer**.

19. Important: Prototype destruction

Here's another interview trap.

Spring manages creation and initialization of prototype beans, but **does not fully manage their destruction lifecycle like singleton beans**.

If Spring creates:

```
Prototype A
```

and gives it to your application, Spring generally doesn't keep tracking it for automatic destruction callbacks when the context shuts down.

Therefore, if a prototype bean owns resources requiring cleanup, you need to design that lifecycle carefully.

20. Real-world scenario

Interviewer:

Suppose I have a singleton `OrderService` and every order processing operation requires a fresh `OrderContext`. `OrderContext` is prototype scoped. How would you implement it?

You could do:

```
@Component
@Scope(ConfigurableBeanFactory.SCOPE_PROTOTYPE)
public class OrderContext {

    private String orderId;

    public void setOrderId(String orderId) {
        this.orderId = orderId;
    }
}
```

Then:

```
@Service
public class OrderService {

    private final ObjectProvider<OrderContext> contextProvider;

    public OrderService(
```

```

        ObjectProvider<OrderContext> contextProvider) {
    this.contextProvider = contextProvider;
}

public void process(String orderId) {

    OrderContext context =
        contextProvider.getObject();

    context.setOrderId(orderId);

    // process order
}
}

```

Now:

```

Order 101
  ↓
getObject()
  ↓
OrderContext A

Order 102
  ↓
getObject()
  ↓
OrderContext B

```

The singleton stays singleton, while the context is freshly created.



21. Very Important Interview Scenario

Interviewer:

What if a request-scoped bean is injected into a singleton?

You shouldn't answer simply:

It won't work.

Spring can handle this using **scoped proxies** or appropriate provider/proxy mechanisms.

For example:



```

@Component
@RequestScope
public class RequestContext {
}

```

and:

```

@Service
public class OrderService {

    private final RequestContext requestContext;

    public OrderService(RequestContext requestContext) {
        this.requestContext = requestContext;
    }
}

```

The singleton can hold a proxy that resolves the actual request-scoped object for the current request.

Conceptually:

```

Singleton OrderService
    |
    ↓
RequestContext Proxy
    |
    +---- Request 1 → Context A
    |
    +---- Request 2 → Context B
    |
    +---- Request 3 → Context C

```

This is a more advanced Spring concept.

22. Scoped Proxy

You may explicitly see:

```

@Component
@Scope(
    value = WebApplicationContext.SCOPE_REQUEST,
    proxyMode = ScopedProxyMode.TARGET_CLASS
)

```

```
public class RequestContext {  
}
```

Now Spring creates a proxy.

```
OrderService  
  |  
  ↓  
Proxy  
  |  
  ↓  
Current Request's RequestContext
```

This allows a longer-lived bean to safely reference a shorter-lived bean.

23. Scope hierarchy — important mental model

Think of scopes in terms of lifetime:

```
ApplicationContext  
  |  
  |---- Singleton  
  |  
  |---- Prototype  
  |  
  |---- Request  
  |  
  |---- Session  
  |  
  |---- WebSocket
```

But the important problem is:

```
LONGER LIVED BEAN  
  |  
  ↓  
SHORTER LIVED DEPENDENCY
```

Example:

```
Singleton  
  ↓  
Prototype
```

or:

```
Singleton
  ↓
Request
```

The dependency cannot simply be treated as a normal object if you expect a fresh shorter-lived instance.

Solutions include:

```
ObjectProvider
Provider
@Lookup
Scoped Proxy
```

Final Interview Cheat Sheet

Singleton

```
One instance per ApplicationContext
```

Spring Singleton ≠ Singleton Pattern

```
Spring → container-managed
Pattern → class-managed
```

Prototype

```
New instance whenever requested from container
```

Request

```
One instance per HTTP request
```

Session

```
One instance per HTTP session
```

Application

One instance per ServletContext

WebSocket

One instance per WebSocket session

Singleton → Prototype

Direct injection

↓

Prototype created when singleton is created

↓

Same prototype reference reused

Need fresh prototype instances?

ObjectProvider<T>

OR

Provider<T>

OR

@Lookup

Shorter-lived web bean inside singleton?

Scoped Proxy

OR

ObjectProvider/Provider

★ Questions I'd expect immediately after this

For a **5-year Spring Boot interview**, be especially prepared for:

1. **How does Spring implement scoped proxies internally?**
2. **What happens when a prototype bean has `@PreDestroy`?**
3. **Singleton bean + request-scoped bean — how does the proxy know which request object to return?**

4. **What is `@Scope(proxyMode = ScopedProxyMode.TARGET_CLASS)` ?**
5. **JDK proxy vs CGLIB proxy?**
6. **Why can't a singleton directly hold a request-scoped object?**
7. **What is `@Lazy` ?**
8. **When are singleton beans instantiated?**
9. **How does prototype scope affect thread safety?**
10. **Can a singleton bean itself be thread-safe if it has mutable state?**
11. **What happens to prototype beans when the application context shuts down?**
12. **How does Spring resolve circular dependencies between singleton beans?**
13. **Why does constructor injection make some circular dependencies fail immediately?**
14. **What is the difference between bean scope and bean lifecycle?**

The **singleton** → **prototype** → `ObjectProvider` → **scoped proxy** chain is particularly worth mastering because it is a very common way interviewers test whether you understand Spring beyond annotations.



@Component vs @Service vs @Repository.
Is @Service technically different from @Component?
What special behavior does @Repository provide?
@Bean vs @Component.
When should you use @Bean?
@Configuration and @Bean.
Why does @Configuration matter?
What happens if you remove @Configuration?
How does component scanning work?
What packages does Spring Boot scan by default



Absolutely. This is another **high-value 5-year Spring Boot interview topic**. The key is understanding that these annotations are not interchangeable even though several of them ultimately register Spring beans.

1. @Component VS @Service VS @Repository

All three are **stereotype annotations**.

```
@Component
  ↑
  |
+— @Service
  |
+— @Repository
  |
+— @Controller
```

Conceptually, @Service, @Repository, and @Controller are specialized forms of @Component.

@Component

Generic Spring-managed component.

```
@Component
public class EmailClient {
}
```

Use it when the class doesn't naturally belong to a more specific layer.

Example:

```
EmailClient
FileProcessor
UtilityComponent
```

@Service

Used for the **service/business logic layer**.

```
@Service
public class OrderService {

    public void createOrder() {
        // business logic
    }
}
```

It communicates:

"This class contains application/business logic."

@Repository

Used for the **persistence/data-access layer**.

```
@Repository
public class OrderRepository {
}
```

It communicates:

"This class interacts with the database or persistence mechanism."

And importantly, `@Repository` has additional Spring behavior.

2. Is `@Service` technically different from `@Component` ?

For normal component registration, **there is no major special runtime behavior added by `@Service` itself.**

This:

```
@Service
public class OrderService {
```

```
}
```

and:

```
@Component
public class OrderService {
}
```

both make the class eligible for component scanning and registration as a Spring bean.

The major difference is **semantic/architectural**.

```
@Component
    ↓
Generic component

@Service
    ↓
Business/service component

@Repository
    ↓
Persistence component
```

Interview answer

`@Service` is technically a specialization of `@Component`. It is primarily used to express that the class belongs to the service/business layer. Unlike `@Repository`, it does not generally provide a distinct built-in behavior such as persistence exception translation.

That's a strong answer.

3. What special behavior does `@Repository` provide?

This is where `@Repository` differs.

Spring can apply **persistence exception translation** to repository beans.

For example, your persistence layer may encounter database-specific exceptions.

Spring can translate underlying persistence exceptions into Spring's consistent `DataAccessException` hierarchy.

Conceptually:

```
Database
  ↓
JDBC/JPA/Hibernate exception
  ↓
Spring exception translation
  ↓
DataAccessException
```

This means your service layer doesn't need to be tightly coupled to a particular persistence technology's exception types.

Example

```
@Repository
public class OrderRepository {

    public Order findById(Long id) {
        // database operation
    }
}
```

The `@Repository` annotation tells Spring this is a persistence component and allows the relevant exception-translation infrastructure to apply.

Interview answer

`@Repository` is a specialization of `@Component` for persistence components. Its important additional behavior is exception translation, where persistence-specific exceptions can be translated into Spring's `DataAccessException` hierarchy.

4. `@Bean` vs `@Component`

This is another very common question.

`@Component`

Put it on the **class**:

```
@Component
public class PaymentService {
}
```

Spring discovers it through component scanning.

@Bean

Put it on a **method**:

```
@Configuration
public class AppConfig {

    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }
}
```

The object returned by the method becomes a Spring bean.

5. When should you use @Bean?

Use @Bean when you want **explicit control over bean creation/configuration**, especially when:

1. You don't own the class

For example, a third-party library class:

```
@Bean
public ObjectMapper objectMapper() {
    return new ObjectMapper();
}
```

You can't add:

```
@Component
```

to a class from an external library.

2. You need custom construction

```
@Bean
public PaymentClient paymentClient() {

    return new PaymentClient(
        "https://payment.example.com",
        timeout
    );
}
```

```
);  
}
```

3. You need configuration

```
@Bean  
public DataSource dataSource() {  
  
    HikariDataSource ds = new HikariDataSource();  
  
    ds.setJdbcUrl(databaseUrl);  
    ds.setUsername(username);  
  
    return ds;  
}
```

4. You want conditional bean creation

For example:

```
@Bean  
@ConditionalOnProperty(  
    name = "payment.enabled",  
    havingValue = "true"  
)  
public PaymentClient paymentClient() {  
    return new PaymentClient();  
}
```

6. @Component vs @Bean — interview comparison

@Component	@Bean
Applied to class	Applied to method
Component scanning discovers it	Configuration method explicitly creates it
Usually used for application classes	Excellent for third-party/configured objects
Less explicit construction	Explicit construction
Spring instantiates the component	You control object creation

Easy memory trick

@Component

↓

"I want Spring to discover this class."

@Bean

↓

"I want to explicitly tell Spring how to create this object."

7. @Configuration and @Bean

Consider:

```
@Configuration
public class AppConfig {

    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }

    @Bean
    public OrderService orderService() {
        return new OrderService(paymentService());
    }
}
```

Here:

```
@Configuration
↓
configuration class
↓
@Bean methods
↓
Spring beans
```

8. Why does @Configuration matter?

This is a **good 5-year question**.

@Configuration tells Spring:

This class contains bean definitions/configuration that should be processed by the container.

But there's a deeper point.

Spring can enhance configuration classes so that calls between `@Bean` methods can respect Spring's bean lifecycle and singleton semantics.

Consider:

```
@Configuration
public class AppConfig {

    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }

    @Bean
    public OrderService orderService() {
        return new OrderService(paymentService());
    }
}
```

You might think:

```
paymentService()
```

simply means:

```
new PaymentService()
```

But with a full `@Configuration` class, Spring can intercept that call and return the managed bean rather than creating an unmanaged second instance.

Conceptually:

```
orderService()
  ↓
paymentService()
  ↓
Spring intercepts
  ↓
existing managed PaymentService
```

So the singleton semantics are preserved.

9. What happens if you remove `@Configuration`?

This question requires a little nuance.

Suppose:

```
@Component
public class AppConfig {

    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }

    @Bean
    public OrderService orderService() {
        return new OrderService(paymentService());
    }
}
```

Spring can still process `@Bean` methods when the class is registered as a configuration component.

But the class is no longer treated as a **full configuration class** in the same way. In particular, direct calls between its `@Bean` methods are not necessarily intercepted to return the managed bean.

This is called **lite configuration** behavior.

So:

```
@Configuration
    ↓
Full configuration

@Component + @Bean
    ↓
Lite configuration
```

Why does that matter?

With full configuration:

```
@Configuration
class Config {
```

```

@Bean
A a() {
    return new A();
}

@Bean
B b() {
    return new B(a());
}
}

```

`a()` can be intercepted so `b()` gets the container-managed `A`.

Without full configuration semantics, the direct method call can behave like a normal Java method call and create another `A`.

Strong interview answer

`@Configuration` marks a class as a full configuration class. Spring enhances it so calls between `@Bean` methods can be intercepted and resolved through the container, preserving the expected bean semantics. Without `@Configuration`, a class containing `@Bean` methods may operate in lite configuration mode, where direct calls between those methods are not intercepted in the same way.

10. `@Configuration(proxyBeanMethods = false)`

You may see this in modern Spring Boot code:

```

@Configuration(proxyBeanMethods = false)
public class AppConfig {
}

```

This tells Spring not to use the method-interception behavior for `@Bean` methods.

This is useful when your `@Bean` methods don't call one another and you don't need full configuration proxy semantics.

For example:

```

@Configuration(proxyBeanMethods = false)
public class AppConfig {

    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }
}

```

```

    }

    @Bean
    public EmailClient emailClient() {
        return new EmailClient();
    }
}

```

Each bean method is independent.

Interview follow-up

Why would Spring Boot use `proxyBeanMethods = false` ?

Good answer:

When configuration methods are independent, disabling bean-method interception can reduce proxying overhead and simplify configuration processing. But if one `@Bean` method directly calls another and relies on container-managed semantics, full configuration behavior may be needed.

11. How does Component Scanning work?

Suppose:

```

@SpringBootApplication
public class Application {
}

```

and:

```

@Service
public class OrderService {
}

```

Spring scans configured packages looking for classes annotated with component stereotypes such as:

```

@Component
@Service
@Repository
@Controller

```

```
@RestController
@Configuration
```

Conceptually:

```
Application
  |
  ↓
Component Scan
  |
+---- OrderService
+---- PaymentService
+---- OrderRepository
+---- OrderController
```

Spring then registers appropriate bean definitions.

12. What exactly happens during component scanning?

At a high level:

```
@SpringBootApplication
  ↓
Component scanning
  ↓
Find candidate classes
  ↓
Read annotations/metadata
  ↓
Create BeanDefinitions
  ↓
Register BeanDefinitions
  ↓
Later instantiate beans
```

Important distinction:

Scanning ≠ creating all objects immediately

Scanning primarily discovers and registers **bean definitions**.

Bean creation happens later according to Spring's lifecycle and bean scope/lazy configuration.

13. What packages does Spring Boot scan by default?

This is a **very common interview question**.

Suppose your main class is:

```
package com.company.order;

@SpringBootApplication
public class OrderApplication {
}
```

By default, component scanning starts from:

```
com.company.order
```

and scans its subpackages.

For example:

```
com.company.order
|
├── controller
|   └── OrderController
|
├── service
|   └── OrderService
|
├── repository
|   └── OrderRepository
|
└── config
    └── AppConfig
```

All of these can be discovered.

But:

```
com.company.payment
```

is **not automatically included** merely because it shares `com.company`.

14. Best package structure

A common structure is:

```
com.company.order
|
```

```
|— OrderApplication.java
|
|— controller
|— service
|— repository
|— entity
|— dto
|— config
```

Put the main Spring Boot class at the **root package**:

```
com.company.order
```

This allows Spring Boot's default component scanning to cover the application.

15. What if my component is outside the scanned package?

Suppose:

```
com.company.order
```

is your root package.

But your service is:

```
com.company.shared.PaymentService
```

It may not be discovered by default.

You have several options.

Option 1 — Move it under the root package

```
com.company.order
|— shared
|   |— PaymentService
```

Often the cleanest solution.

Option 2 — Explicit component scan

```
@SpringBootApplication
@ComponentScan({
```

```

    "com.company.order",
    "com.company.shared"
  })
  public class Application {
  }

```

Option 3 — Explicit `@Bean`

```

@Configuration
public class PaymentConfig {

    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }
}

```

16. What is `@SpringBootApplication` actually doing?

This is another excellent follow-up.

Conceptually:

```
@SpringBootApplication
```

combines:

```

@SpringBootApplication
@EnableAutoConfiguration
@ComponentScan

```

So:

```

@SpringBootApplication
|
+-- @SpringBootApplication
|
+-- @EnableAutoConfiguration
|
+-- @ComponentScan

```

This explains why putting the main class in the root package is so important.

🔥 17. @Component VS @Service VS @Repository

Memorize this:

```
@Component
↓
Generic Spring component

@Service
↓
Business/service layer

@Repository
↓
Persistence/data-access layer
↓
Exception translation
```

🔥 18. @Component VS @Bean

Memorize:

```
@Component
↓
Put annotation on CLASS
↓
Spring discovers it

@Bean
↓
Put annotation on METHOD
↓
You explicitly create/configure object
```

🔥 19. @Configuration mental model

```
@Configuration
↓
"I have bean definitions here"
↓
@Bean methods
↓
Spring processes them
↓
```

Full configuration semantics



Bean method calls can be intercepted

And:

`@Configuration(proxyBeanMethods = false)`



No configuration-method interception



Good when `@Bean` methods are independent



20. Component Scanning mental model

`@SpringBootApplication`



`@ComponentScan`



Starting package



All subpackages



Find:

`@Component`

`@Service`

`@Repository`

`@Controller`

`@Configuration`



BeanDefinitions



Spring creates/manages beans

★ One-liner interview answers

Q: Is `@Service` technically different from `@Component` ?

It is a specialization of `@Component` ; its main purpose is to express service-layer semantics rather than provide a fundamentally different bean-registration mechanism.

Q: What special behavior does `@Repository` provide?

It marks persistence components and enables Spring's persistence exception translation mechanism.

Q: When do you use `@Bean` ?

When I need explicit control over object creation/configuration, especially for third-party classes or objects requiring custom construction.

Q: Why does `@Configuration` matter?

It defines a full configuration class and enables Spring to intercept `@Bean` method calls so container-managed bean semantics are preserved.

Q: What package does Spring Boot scan by default?

Component scanning starts from the package of the `@SpringBootApplication` class and scans its subpackages by default. Therefore, the main application class is normally placed at the root package.



What is Spring Boot?
Spring Framework vs Spring Boot.
Why was Spring Boot introduced?
What problem does Spring Boot solve?
What is auto-configuration?
How does auto-configuration work?
What is @SpringBootApplication?
Explain:
@SpringBootApplication
What is starter dependency?
What is spring-boot-starter-web?
What is spring-boot-starter-data-jpa?
What is embedded Tomcat?
Can Spring Boot run without Tomcat?
How do you change the embedded server?
What is Spring Boot Actuator?
Why is Actuator useful in production



For a **5-year Java/Spring Boot interview**, don't stop at "Spring Boot reduces configuration." The interviewer may ask **what it actually does internally, how auto-configuration decides what to configure, what starters contain, and how you would use Actuator in production.**

1. What is Spring Boot?

Spring Boot is a framework built on top of the Spring Framework that simplifies the development and deployment of Spring applications.

It provides:

- Auto-configuration
- Starter dependencies
- Embedded servers
- Production-ready features
- Opinionated defaults
- Externalized configuration
- Easy application setup

Without Boot, you may have to configure many Spring components manually.

With Boot:

```
@SpringBootApplication
public class OrderApplication {

    public static void main(String[] args) {
        SpringApplication.run(OrderApplication.class, args);
    }
}
```

You can run the application directly.

Conceptually:

```
Spring Framework
  ↑
  |
Spring Boot
  |
  +— Auto-configuration
  +— Starters
  +— Embedded server
  +— Actuator
  +— External configuration
```

Interview answer

Spring Boot is built on top of the Spring Framework and provides conventions, auto-configuration, starter dependencies, embedded servers, and production-ready features to reduce boilerplate configuration and simplify deployment.

2. Spring Framework vs Spring Boot

This is one of the most frequently asked questions.

Spring Framework	Spring Boot
Core framework	Built on Spring
More manual configuration possible	Convention and auto-configuration
Doesn't inherently provide starter dependencies	Provides starters

Spring Framework

Spring Boot

Traditionally external server deployment was common

Embedded server support

More configuration decisions

Opinionated defaults

Production features require additional setup

Actuator provides production features

More setup

Faster application development

Simple example

Traditional Spring application might require configuring:

```
Spring MVC
DataSource
JPA
Transaction manager
Servlet infrastructure
JSON conversion
etc.
```

Spring Boot tries to provide sensible defaults based on what you have on the classpath and your configuration.

3. Why was Spring Boot introduced?

The problem was not that Spring was incapable.

The problem was **configuration complexity**.

A typical Spring application could require a significant amount of:

```
XML configuration
Bean configuration
Server configuration
Dependency configuration
Infrastructure configuration
```

Spring Boot introduced a convention-over-configuration approach.

Instead of asking developers to configure every common component manually:

```
"What should I configure?"
```

Boot tries to answer:

"What dependencies and configuration have you provided?"

↓

"Based on that, what sensible configuration can I automatically provide?"

4. What problem does Spring Boot solve?

Think of Spring Boot as reducing **setup and infrastructure boilerplate**.

Without Boot

You may have to manually configure:

```
Spring MVC
Servlet container
Jackson
DataSource
JPA
Transaction management
Bean definitions
```

With Boot

You can add:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

and Boot can configure much of the common web infrastructure automatically.

So the major problems solved are:

```
Dependency management
+
Auto-configuration
+
Embedded server
+
External configuration
+
Production monitoring
```

```
=  
Less boilerplate
```

5. What is Auto-Configuration?

🔥 **Very important interview question.**

Auto-configuration means Spring Boot attempts to automatically configure Spring components based on:

- Dependencies available on the classpath
- Existing beans
- Application properties
- Conditional annotations
- Application type

For example, if you add:

```
spring-boot-starter-web
```

Boot detects web-related classes and configures the necessary web infrastructure.

Similarly, if you add JPA-related dependencies and configure a database, Boot can configure things such as:

```
DataSource  
EntityManagerFactory  
Transaction infrastructure
```

depending on the environment and available configuration.

6. How does Auto-Configuration work?

This is where you should answer at a **5-year level**.

Suppose you add:

```
spring-boot-starter-data-jpa
```

Spring Boot's auto-configuration system looks at the classpath and conditions.

Conceptually:

```
Application starts
  ↓
@EnableAutoConfiguration
  ↓
Discover auto-configuration classes
  ↓
Evaluate conditions
  ↓
Is required class present?
  ↓
Is required bean missing?
  ↓
Does configuration property match?
  ↓
YES
  ↓
Register configuration/beans
```

For example, conceptually:

```
@Configuration
@ConditionalOnClass(EntityManager.class)
@ConditionalOnMissingBean(...)
public class SomeJpaAutoConfiguration {
    // configuration
}
```

The actual auto-configuration classes are more extensive, but this demonstrates the mechanism.

7. What does "conditional" mean?

Suppose Boot has an auto-configuration for something.

It may say conceptually:

```
IF
    class X exists
AND
    bean Y doesn't already exist
AND
    property Z is enabled
THEN
    configure bean
```

Spring Boot has many conditional annotations, including:

```
@ConditionalOnClass
@ConditionalOnMissingBean
@ConditionalOnBean
@ConditionalOnProperty
@ConditionalOnWebApplication
@ConditionalOnResource
```

This is the core idea behind auto-configuration.

8. What happens if I define my own bean?

This is another excellent interview question.

Suppose Boot would normally configure a particular bean.

But you define your own:

```
@Bean
public ObjectMapper objectMapper() {
    return customObjectMapper();
}
```

Some Boot auto-configurations use conditions such as:

```
@ConditionalOnMissingBean
```

So if your bean already exists:

```
Boot auto-configuration
    ↓
Is bean already present?
    ↓
YES
    ↓
Don't create default bean
```

This is why Boot is often described as:

convention over configuration, but still allowing customization.

9. What is `@SpringBootApplication`?

This annotation is extremely important.

```
@SpringBootApplication
public class Application {
}
```

Conceptually, it combines three important annotations:

```
@SpringBootApplication
|
+— @SpringBootConfiguration
|
+— @EnableAutoConfiguration
|
+— @ComponentScan
```

So you can remember:

```
@SpringBootApplication
=
Configuration
+
Auto-configuration
+
Component scanning
```

10. Explain `@SpringBootApplication`

Let's break it down.

`@SpringBootConfiguration`

Indicates that the class is a Spring Boot configuration class.

It is essentially a specialized form of Spring's configuration mechanism.

`@EnableAutoConfiguration`

Tells Spring Boot to enable its auto-configuration mechanism.

Conceptually:

```
Look at classpath
↓
Look at environment
↓
Evaluate conditions
```

↓

Configure appropriate infrastructure

`@ComponentScan`

Tells Spring to scan for components.

For example:

```
@Service
@Repository
@Component
@Controller
@RestController
@Configuration
```

By default, scanning starts from the package containing the application class and includes its subpackages.

11. What is a Starter Dependency?

A **Spring Boot starter** is a convenient dependency descriptor that brings together a set of dependencies commonly needed for a particular type of application.

Instead of manually adding:

```
Spring MVC
Jackson
Tomcat
Validation-related dependencies
etc.
```

you can use:

```
spring-boot-starter-web
```

The starter itself is mainly about **dependency management**.

Important distinction:

Starter dependency ≠ auto-configuration.

They work together.

```
Starter
  ↓
Brings dependencies
  ↓
Those dependencies appear on classpath
  ↓
Auto-configuration detects them
  ↓
Boot configures appropriate infrastructure
```

12. What is `spring-boot-starter-web` ?

It is the standard starter for building traditional Spring web/MVC applications.

It brings the dependencies needed for common web application development, including Spring MVC and an embedded servlet container such as Tomcat under the standard setup.

Conceptually:

```
spring-boot-starter-web
|
+— Spring MVC
+— Embedded Tomcat
+— JSON support
+— Web infrastructure
+— Related dependencies
```

Therefore, after adding it, you can build:

```
@RestController
@RequestMapping("/orders")
public class OrderController {
}
```

and run the application directly.

13. What is `spring-boot-starter-data-jpa` ?

This starter provides dependencies needed for using **Spring Data JPA**.

Conceptually:

```
spring-boot-starter-data-jpa
|
+— Spring Data JPA
```

- +— Spring ORM/JPA infrastructure
- +— Hibernate (commonly the default JPA provider)
- +— Transaction-related support

Then you can write:

```
@Entity
public class Order {
}
```

and:

```
public interface OrderRepository
    extends JpaRepository<Order, Long> {
}
```

Spring Boot can configure much of the JPA infrastructure based on the dependencies and database configuration.

14. Starter vs Dependency

A common confusion:

```
spring-boot-starter-web
```

isn't itself the web server.

It is a **starter that pulls in a set of dependencies**.

For example:

```
starter-web
  ↓
dependencies
  ↓
Spring MVC
Tomcat
Jackson
etc.
```

Then Boot's auto-configuration uses those available libraries to configure the application.

15. What is Embedded Tomcat?

Normally, a traditional Java web application could be packaged as a WAR and deployed to an externally managed servlet container.

With Spring Boot, the servlet container can be packaged with your application.

For example:

```
order-service.jar
|
+— Application code
+— Dependencies
+— Embedded Tomcat
```

When you run:

```
java -jar order-service.jar
```

the application starts the embedded server.

Conceptually:

```
JVM
|
+— Spring Boot
   |
   +— Spring ApplicationContext
      |
      +— Embedded Tomcat
```

You don't need to separately install and deploy to a Tomcat server for the usual executable-jar setup.

16. Can Spring Boot run without Tomcat?

Yes.

Spring Boot doesn't fundamentally require Tomcat.

The server depends on your application type and dependencies.

For servlet-based applications, you can replace Tomcat with another supported servlet container.

For example:

```
Tomcat
Jetty
```

Undertow

And for reactive applications:

Spring WebFlux

↓

Reactive server such as Reactor Netty

17. How do you change the embedded server?

For a servlet-based application, the common approach is to exclude Tomcat and add another server starter.

For example, using Jetty:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
  <exclusions>
    <exclusion>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-tomcat</artifactId>
    </exclusion>
  </exclusions>
</dependency>
```

Then add the Jetty starter appropriate to your Spring Boot version.

Conceptually:

starter-web

↓

Tomcat

becomes:

starter-web

↓

exclude Tomcat

↓

add Jetty

Interview answer

For a servlet-based Spring Boot application, I can exclude the Tomcat starter and add a supported alternative such as Jetty. Spring Boot then auto-configures the selected server.

18. What is Spring Boot Actuator?

Spring Boot Actuator provides production-oriented endpoints and instrumentation for monitoring and managing a Spring Boot application.

You typically add:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

It can expose endpoints such as:

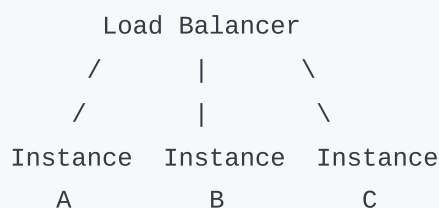
```
/actuator/health
/actuator/info
/actuator/metrics
```

depending on your configuration and Spring Boot version.

19. Why is Actuator useful in production?

This is where you should give a practical answer rather than just listing endpoints.

Suppose you have:



Your operations/monitoring system needs to know:

```
Is instance healthy?
How much memory is it using?
How many requests are coming?
What are response metrics?
Are there problems with the application?
```

Actuator provides useful application-level observability and management endpoints.

Health

```
/actuator/health
```

Can provide health information and can integrate with infrastructure health checking, depending on configuration.

Conceptually:

```
Load Balancer / Monitoring
      ↓
/actuator/health
      ↓
UP / DOWN / status details
```

Metrics

```
/actuator/metrics
```

Provides application metrics.

Depending on configuration and instrumentation, metrics can cover areas such as:

```
HTTP requests
JVM
memory
GC
threads
CPU-related metrics
database connection pools
```

For production observability, metrics are commonly exported to a monitoring system through Micrometer integrations.

20. Actuator + Micrometer

This is a great **5-year follow-up**.

Think:

```
Application
      ↓
Micrometer
      ↓
Metrics
```

↓
Prometheus / monitoring backend
↓
Grafana / dashboards

Actuator exposes/uses Spring Boot's management and observability infrastructure, while Micrometer provides a vendor-neutral metrics facade and integrations with monitoring systems.

For example:

HTTP request
↓
Spring Boot
↓
Micrometer records metric
↓
Prometheus
↓
Grafana

21. Is Actuator safe to expose publicly?

🔥 Important production question.

You should not blindly expose all actuator endpoints to the public internet.

Actuator endpoints can expose operational information.

A production architecture might be:

Internet
|
↓
API
|
↓
Application

Monitoring Network
|
↓
Actuator

You should:

- Expose only required endpoints
- Secure sensitive endpoints
- Use authentication/authorization where appropriate
- Restrict management access at the network layer where appropriate
- Avoid exposing sensitive environment/configuration information publicly

For example, you might only expose health information needed by your infrastructure and keep more sensitive management endpoints restricted.

22. Production Scenario

Interviewer:

Your Spring Boot application is deployed across 10 instances. One instance starts failing intermittently. How would Actuator help?

A strong answer:

I would expose and secure the appropriate health and metrics endpoints. The health endpoint can help infrastructure determine whether an instance is healthy, while metrics can help identify JVM, HTTP, thread, GC, connection-pool, and other application-level issues. I would combine this with centralized logs, distributed tracing, and a monitoring system rather than relying on Actuator alone.

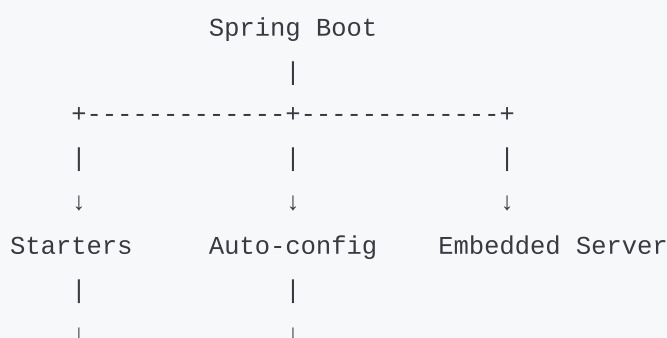
That's much stronger than:

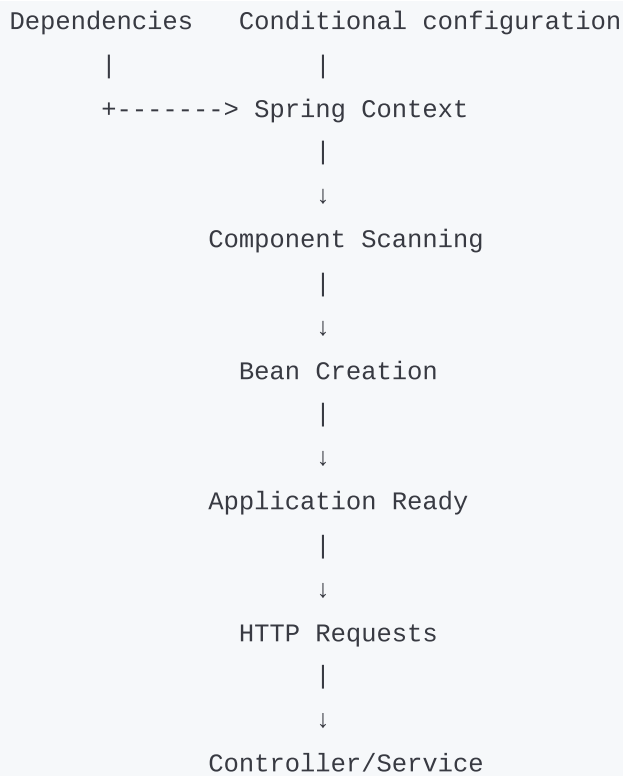
"I use `/actuator/health`."



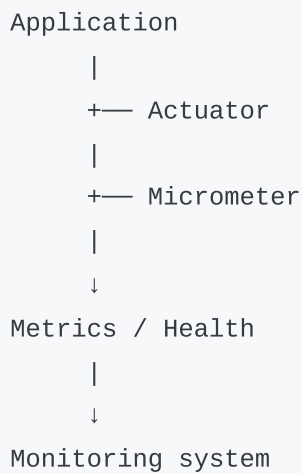
Complete Spring Boot Mental Model

Remember this flow:





And for production:



★ 5-Year Interview Answers — Rapid Fire

What is Spring Boot?

A framework built on Spring that simplifies application development through auto-configuration, starters, embedded servers, externalized configuration, and production-ready features.

Why Spring Boot?

To reduce configuration and infrastructure boilerplate while still allowing customization.

What is auto-configuration?

Spring Boot automatically configures application infrastructure based on classpath dependencies, existing beans, properties, and conditional rules.

What is `@SpringBootApplication` ?

It combines `@SpringBootConfiguration` , `@EnableAutoConfiguration` , and `@ComponentScan` .

What is a starter?

A convenient dependency descriptor that brings together commonly required dependencies for a particular application capability.

What is `spring-boot-starter-web` ?

A starter for servlet-based Spring web/MVC applications, bringing Spring MVC and commonly an embedded Tomcat server along with related web dependencies.

What is embedded Tomcat?

A servlet container packaged with the application so the Spring Boot application can run as an executable JAR without requiring a separately installed application server.

Can Boot run without Tomcat?

Yes. For servlet applications, Tomcat can be replaced with another supported servlet container such as Jetty; reactive applications can use a reactive server such as Reactor Netty.

What is Actuator?

A Spring Boot module providing production-oriented monitoring, health, metrics, and management capabilities.

Why Actuator in production?

It provides application-level health and observability information that can be consumed by infrastructure and monitoring systems, helping with availability monitoring, diagnosis, and operational visibility.



What is auto-configuration?
How does Spring Boot decide what to configure?
What is @ConditionalOnClass?
@ConditionalOnMissingBean.
@ConditionalOnProperty.
How do you disable an auto-configuration?
How would you create your own auto-configuration?
What happens if you add/remove a starter dependency?
answer above question only don't add anything in it



1. What is Auto-Configuration?

Auto-configuration is a Spring Boot feature that automatically configures Spring beans and application infrastructure based on the application's dependencies, existing beans, properties, and environment.

For example, if you add:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

Spring Boot can automatically configure things such as:

DataSource
EntityManagerFactory
Transaction infrastructure
JPA repositories

provided the required classes and configuration are available.

The important point is:

Auto-configuration does not blindly create everything. It uses conditions to decide whether a particular configuration should be applied.

2. How does Spring Boot decide what to configure?

Spring Boot primarily uses:

1. **Classpath dependencies**
2. **Existing beans**
3. **Application properties**
4. **Conditional annotations**
5. **Application type/environment**

Conceptually:

```
Application starts
  ↓
@EnableAutoConfiguration
  ↓
Find auto-configuration classes
  ↓
Evaluate conditions
  ↓
Required classes present?
  ↓
Required bean already exists?
  ↓
Required property enabled?
  ↓
YES
  ↓
Apply configuration
```

For example:

```
spring-boot-starter-data-jpa
  ↓
JPA classes available
  ↓
JPA auto-configuration considered
  ↓
Database configuration available?
  ↓
Existing custom beans?
  ↓
Configure required JPA infrastructure
```

Spring Boot uses conditional annotations extensively to make these decisions.

Example:

```
@Configuration
public class MyAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean
    public PaymentClient paymentClient() {
        return new PaymentClient();
    }
}
```

Suppose the application doesn't define `PaymentClient`.

```
PaymentClient exists?
    ↓
    NO
    ↓
Create default PaymentClient
```

But if the application defines:

```
@Bean
public PaymentClient paymentClient() {
    return new CustomPaymentClient();
}
```

then:

```
PaymentClient exists?
    ↓
    YES
    ↓
Don't create default bean
```

This is one of the key mechanisms that makes Spring Boot **customizable**.

5. What is `@ConditionalOnProperty` ?

`@ConditionalOnProperty` allows configuration to be enabled or disabled based on an application property.

Example:

```
@Configuration
@ConditionalOnProperty(
    name = "payment.enabled",
    havingValue = "true"
)
public class PaymentAutoConfiguration {
}
```

Then:

```
payment.enabled=true
```

means:

```
Property = true
    ↓
Configuration enabled
```

Whereas:

```
payment.enabled=false
```

means:

```
Property = false
    ↓
Configuration skipped
```

A practical example:

```
@Bean
@ConditionalOnProperty(
    name = "notification.enabled",
    havingValue = "true"
)
public NotificationClient notificationClient() {
    return new NotificationClient();
}
```

This lets users control auto-configuration through application properties without changing Java code.

6. How do you disable an auto-configuration?

There are several ways.

Using `exclude` with `@SpringBootApplication`

```
@SpringBootApplication(  
    exclude = DataSourceAutoConfiguration.class  
)  
public class Application {  
}
```

This tells Spring Boot not to apply that auto-configuration.

Using `spring.autoconfigure.exclude`

You can also configure:

```
spring.autoconfigure.exclude=\norg.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
```

This is useful when you want to control the exclusion through configuration rather than the application annotation.

Why would you disable auto-configuration?

For example, suppose Spring Boot tries to configure a database, but your application doesn't use a database.

You could exclude:

```
DataSourceAutoConfiguration.class
```

so Spring Boot doesn't attempt to configure the DataSource infrastructure.

7. How would you create your own auto-configuration?

At a high level:

Step 1 — Create your configuration

```
@AutoConfiguration  
@ConditionalOnClass(PaymentClient.class)  
public class PaymentAutoConfiguration {  
  
    @Bean
```

```

@ConditionalOnMissingBean
public PaymentClient paymentClient() {
    return new PaymentClient();
}
}

```

Here:

```

@Configuration
↓
This is an auto-configuration class

@ConditionalOnClass
↓
Only if required class exists

@ConditionalOnMissingBean
↓
Only if user hasn't defined their own bean

@Bean
↓
Create default bean

```

Step 2 — Register the auto-configuration

For modern Spring Boot versions, register it in:

```
META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
```

with:

```
com.example.PaymentAutoConfiguration
```

Spring Boot can then discover the auto-configuration.

Overall flow

```

Custom starter/library
↓
AutoConfiguration class
↓
AutoConfiguration.imports
↓

```

```
Spring Boot discovers it
    ↓
Conditions evaluated
    ↓
Beans created if conditions match
```

8. What happens if you add/remove a starter dependency?

When you add a starter

Suppose you add:

```
spring-boot-starter-data-jpa
```

The starter brings required dependencies onto the classpath.

Then:

```
Add starter
    ↓
Dependencies added
    ↓
Classes become available
    ↓
Relevant auto-configurations become candidates
    ↓
Conditions evaluated
    ↓
Matching configuration applied
```

For example:

```
Add JPA starter
    ↓
JPA/Hibernate classes available
    ↓
JPA auto-configuration considered
    ↓
Conditions satisfied
    ↓
JPA infrastructure configured
```

When you remove a starter

Suppose you remove:

```
spring-boot-starter-data-jpa
```

Then the JPA-related dependencies may no longer be available.

Conceptually:

```
Remove starter
  ↓
JPA dependencies removed
  ↓
JPA classes unavailable
  ↓
@ConditionalOnClass fails
  ↓
JPA auto-configuration skipped
```

As a result, functionality that depended on those dependencies may no longer be available.

Important

Removing a starter doesn't simply mean:

"Spring removes some beans."

It can change the **classpath**, and that changes which auto-configurations are eligible to run.



application.properties vs application.yml.
How do you define custom properties?
@Value vs @ConfigurationProperties.
Profiles.
application-dev.yml.
application-prod.yml.
How do you activate a profile?
Environment variables.
Command-line arguments.
Configuration precedence.
How would you manage secrets?
Should passwords be stored in application.yml?
How would you externalize configuration in production?
What is Spring Cloud Config?
What is Consul configuration



1. application.properties VS application.yml

Both are used for **externalized configuration** in Spring Boot.

application.properties

```
server.port=8080
spring.datasource.url=jdbc:postgresql://localhost:5432/orderdb
spring.datasource.username=admin
app.payment.timeout=5000
```

application.yml

```
server:
  port: 8080

spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/orderdb
    username: admin

app:
  payment:
    timeout: 5000
```

Both represent the same configuration.

Main difference

`YAML` is usually more readable for **hierarchical/nested configuration**, while `.properties` is simple and straightforward.

There is no fundamental runtime advantage where one is inherently "faster" or "better."

2. How do you define custom properties?

You can define application-specific properties:

```
app.payment.timeout=5000
app.payment.retry-count=3
app.payment.enabled=true
```

Then access them using `@Value` :

```
@Value("${app.payment.timeout}")
private int timeout;
```

Or preferably for groups of related configuration, use `@ConfigurationProperties` .

3. `@Value` VS `@ConfigurationProperties`

`@Value`

```
@Value("${app.payment.timeout}")
private int timeout;
```

Good for injecting a **small number of individual properties**.

Example:

```
@Value("${app.name}")
private String appName;
```

`@ConfigurationProperties`

Suppose:

```
app:
  payment:
    url: https://payment-service
```

```
timeout: 5000
retry-count: 3
```

You can create:

```
@ConfigurationProperties(prefix = "app.payment")
public class PaymentProperties {

    private String url;
    private int timeout;
    private int retryCount;

    // getters/setters
}
```

Then register it appropriately, for example:

```
@ConfigurationPropertiesScan
@SpringBootApplication
public class Application {
}
```

Now Spring binds:

```
app.payment.url
      ↓
url

app.payment.timeout
      ↓
timeout

app.payment.retry-count
      ↓
retryCount
```

Why is `@ConfigurationProperties` generally preferred for grouped configuration?

It gives you:

- Type-safe configuration
- Centralized configuration

- Better organization
- Easier validation
- Easier testing
- Better support for large configuration structures

Interview answer

I use `@Value` for a few simple properties and `@ConfigurationProperties` when I have a group of related configuration values that should be represented as a typed configuration object.

4. What are Spring Profiles?

Profiles allow you to maintain **environment-specific configuration and beans**.

Common profiles:

```
dev
test
qa
prod
```

For example:

```
application.yml
application-dev.yml
application-prod.yml
```

You can have different database configurations for different environments.

5. `application-dev.yml`

Example:

```
server:
  port: 8081

spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/order_dev
```

And:

```
application-prod.yml
```

```
server:  
  port: 8080  
  
spring:  
  datasource:  
    url: jdbc:postgresql://prod-db:5432/order
```

The application can load the configuration corresponding to the active profile.

6. How do you activate a profile?

There are several common ways.

```
application.properties
```

```
spring.profiles.active=dev
```

Command line

```
java -jar app.jar --spring.profiles.active=prod
```

Environment variable

```
SPRING_PROFILES_ACTIVE=prod
```

The latter two are particularly useful for deployment because you don't have to modify the application artifact.

7. Environment Variables

Spring Boot can read configuration from environment variables.

For example:

```
SERVER_PORT=8080  
SPRING_PROFILES_ACTIVE=prod
```

Configuration naming generally maps hierarchical property names to environment-variable style names.

For example:

```
spring.datasource.url
```

can be represented as:

```
SPRING_DATASOURCE_URL
```

And:

```
app.payment.timeout
```

can be represented as:

```
APP_PAYMENT_TIMEOUT
```

This is especially useful in containers and cloud environments.

8. Command-Line Arguments

Spring Boot can accept configuration through command-line arguments.

Example:

```
java -jar app.jar --server.port=9090
```

or:

```
java -jar app.jar --spring.profiles.active=prod
```

These arguments become configuration properties and can override lower-priority configuration sources.

9. Configuration Precedence

This is important because the same property can exist in multiple places.

For example:

```
application.yml
  ↓
Environment variable
  ↓
Command-line argument
```

If all three define:

```
server.port
```

the higher-precedence source wins.

A simplified mental model is:

```
Lowest priority
  ↓
application.yml / application.properties
  ↓
profile-specific configuration
  ↓
environment variables
  ↓
command-line arguments
  ↓
Highest priority
```

Spring Boot has a more extensive and version-specific property-source ordering, so in an interview it is better to say:

Spring Boot has a defined property-source precedence, with external sources such as environment variables and command-line arguments generally taking precedence over values packaged in the application.

10. How would you manage secrets?

Do not hard-code secrets into source code or commit them into Git.

Avoid:

```
spring:
  datasource:
    username: admin
    password: MySecretPassword123
```

in a repository.

Instead, use a secret-management mechanism such as:

```
AWS Secrets Manager
HashiCorp Vault
Kubernetes Secrets
```

Azure Key Vault
GCP Secret Manager

depending on the infrastructure.

The application can receive secrets through:

Environment variables
Secret files
Secret-management integrations
External configuration systems

11. Should passwords be stored in `application.yml` ?

For production:

No, not as plaintext committed to source control.

This is bad:

```
spring:
  datasource:
    password: myProductionPassword
```

Especially if:

```
Git
  ↓
Repository
  ↓
Everyone with repository access
```

Instead:

```
Secret Manager
  ↓
Runtime
  ↓
Application
  ↓
Database
```

For local development, developers may use local configuration that isn't committed, but production credentials should be managed through an appropriate secret-management solution.

12. How would you externalize configuration in production?

I would separate:

```
Application artifact
      +
Environment-specific configuration
      +
Secrets
```

For example:

```
app.jar
  |
  +---- application defaults
  |
  ↓
Production environment
  |
  +---- Environment variables
  +---- External configuration
  +---- Secret manager
```

The same application artifact can then be deployed to:

```
DEV
QA
PROD
```

without rebuilding the application for each environment.

For example:

```
SPRING_PROFILES_ACTIVE=prod
DB_HOST=prod-db
DB_PORT=5432
```

while sensitive credentials come from a secret manager.

Strong interview answer

I prefer keeping environment-specific configuration outside the application artifact and injecting it at deployment/runtime. Non-sensitive configuration can come from environment

variables or an external configuration system, while credentials and secrets should come from a dedicated secret manager.

13. What is Spring Cloud Config?

Spring Cloud Config provides centralized external configuration management for distributed applications.

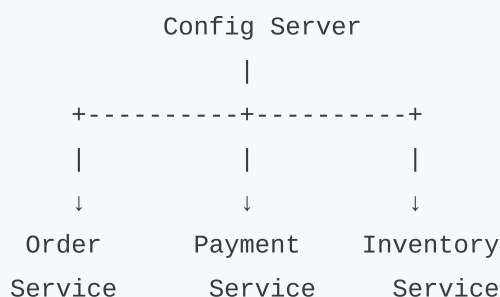
Instead of maintaining configuration independently inside every microservice:

```
Order Service
└─ application.yml

Payment Service
└─ application.yml

Inventory Service
└─ application.yml
```

you can centralize configuration:



A common architecture is:

```
Git Repository
  ↓
Config Server
  ↓
Microservices
```

The Config Server retrieves configuration from a backing repository/source and serves configuration to client applications.

14. Why use Spring Cloud Config?

Imagine you have:

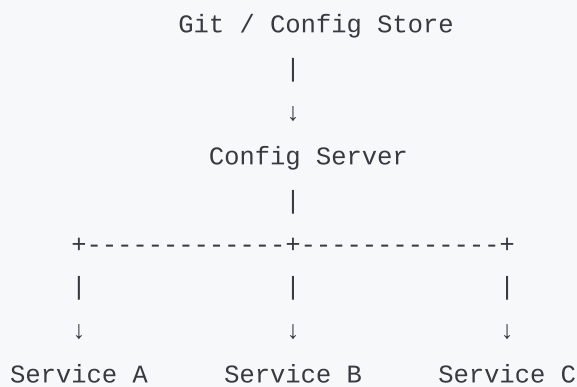
50 microservices

and each service needs:

database URLs
feature flags
timeouts
external service URLs
configuration values

Managing all of that independently becomes difficult.

With centralized configuration:



Benefits include:

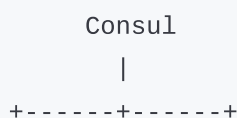
- Centralized configuration
- Environment-specific configuration
- Versioning through a configuration repository
- Separation of configuration from application code
- Easier configuration management across microservices

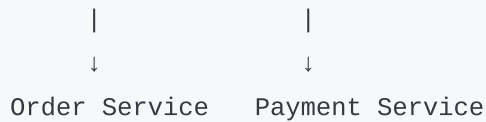
15. What is Consul Configuration?

Consul can provide centralized configuration storage and service-discovery capabilities.

For configuration, applications can store key-value configuration in Consul's KV store.

Conceptually:





Configuration might conceptually look like:

```
config/  
  order-service/  
    timeout = 5000  
    feature.enabled = true  
  
  payment-service/  
    timeout = 3000
```

Applications can retrieve configuration from Consul rather than keeping all configuration inside their packaged JAR.

16. Spring Cloud Config vs Consul Configuration

They solve a similar problem—**externalized/centralized configuration**—but they have different architectures and strengths.

Spring Cloud Config

Common architecture:

```
graph TD; A[Git] --> B[Spring Cloud Config Server]; B --> C[Spring Boot applications];
```

Good when configuration is maintained in a version-controlled repository and you want Spring-oriented centralized configuration.

Consul

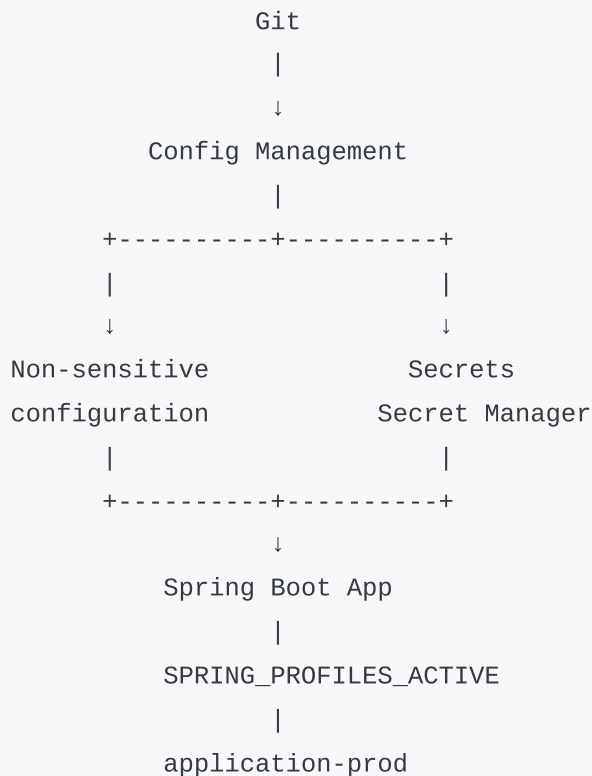
Common architecture:

```
graph TD; A[Consul KV] --> B[Spring Boot applications];
```

Consul is also well known for **service discovery**, in addition to its KV configuration capabilities.

17. Production Configuration Architecture

A strong 5-year answer would look like:



The important separation is:

Configuration
≠
Secrets

Normal configuration can be centrally managed.

Secrets should generally be managed by a dedicated secrets-management solution.

Quick Interview Answers

`application.properties` vs `application.yml` ?

Both provide externalized configuration. YAML is often more readable for hierarchical configuration, while properties is simpler key-value syntax.

`@Value` vs `@ConfigurationProperties` ?

`@Value` is convenient for individual properties; `@ConfigurationProperties` is better for groups of related, type-safe configuration.

What are profiles?

Profiles allow environment-specific beans and configuration, such as dev, QA, and production.

How activate a profile?

Using `spring.profiles.active`, commonly through environment variables or command-line arguments in production.

Should passwords be stored in `application.yml` ?

Not as plaintext in source control. Production secrets should come from a dedicated secret-management mechanism.

How externalize configuration in production?

Keep non-sensitive configuration outside the application artifact and inject it at runtime using environment variables or centralized configuration; use a secret manager for sensitive values.

What is Spring Cloud Config?

A centralized configuration solution for distributed applications that allows microservices to obtain environment-specific configuration from a central configuration source.

What is Consul configuration?

Consul can store configuration in its KV store and provide it to applications. It can also provide service discovery, making it useful in microservice environments.



Controllers

@Controller vs @RestController.

@RequestMapping.

@GetMapping.

@PostMapping.

@PutMapping.

@PatchMapping.

@DeleteMapping.

@PathVariable vs @RequestParam.

@RequestBody.

@RequestHeader.

@CookieValue.

give one line definition



- `@Controller` — Marks a Spring MVC class that typically returns a **view/page** such as HTML.
- `@RestController` — Combination of `@Controller` + `@ResponseBody`; typically returns **data such as JSON/XML directly in the HTTP response**.
- `@RequestMapping` — Maps HTTP requests to a controller or method based on **URL path, HTTP method, headers, parameters, etc.**
- `@GetMapping` — Maps **HTTP GET** requests to a controller method, typically for retrieving data.
- `@PostMapping` — Maps **HTTP POST** requests, typically for creating/submitting data.
- `@PutMapping` — Maps **HTTP PUT** requests, typically for completely updating/replacing a resource.
- `@PatchMapping` — Maps **HTTP PATCH** requests, typically for partially updating a resource.
- `@DeleteMapping` — Maps **HTTP DELETE** requests, typically for deleting a resource.
- `@PathVariable` — Extracts a value directly from the **URL path**.
- `@RequestParam` — Extracts a value from **query parameters** in the URL.

- `@RequestBody` — Converts the **HTTP request body** (commonly JSON) into a Java object.
- `@RequestHeader` — Extracts a value from an **HTTP request header**.
- `@CookieValue` — Extracts a value from an **HTTP cookie**.



GET vs POST.
PUT vs PATCH.
PUT vs POST.
What is idempotency?
Which HTTP methods are idempotent?
HTTP status codes.
200 vs 201.
400 vs 404.
401 vs 403.
409 Conflict.
301
300
415
422
429
500
501
502
503
504
give one line definition



- **GET vs POST** — `GET` retrieves data and should not modify server state; `POST` submits data, commonly to create a resource or trigger an operation.
- **PUT vs PATCH** — `PUT` typically replaces the complete resource; `PATCH` partially updates a resource.
- **PUT vs POST** — `PUT` is generally idempotent and targets a known resource/URI; `POST` is generally non-idempotent and often lets the server determine the new resource URI.
- **Idempotency** — An HTTP operation is idempotent when making the same request multiple times has the same intended server-side effect as making it once.
- **Idempotent HTTP methods** — `GET`, `HEAD`, `PUT`, `DELETE`, and `OPTIONS` are defined as idempotent; `POST` and `PATCH` are not inherently idempotent.

HTTP Status Codes

- **200 OK** — The request was successful.
- **201 Created** — The request successfully created a new resource.
- **300 Multiple Choices** — The request has multiple possible representations/resources to choose from.
- **301 Moved Permanently** — The requested resource has permanently moved to a new URL.
- **400 Bad Request** — The server cannot process the request because of invalid client input/request syntax.
- **401 Unauthorized** — Authentication is required or the provided authentication credentials are invalid.
- **403 Forbidden** — The server understood the request but refuses to authorize it.
- **404 Not Found** — The requested resource could not be found.
- **409 Conflict** — The request conflicts with the current state of the resource.
- **415 Unsupported Media Type** — The server does not support the request's media/content type.
- **422 Unprocessable Content** — The request format is valid, but the contained data fails semantic/validation rules.
- **429 Too Many Requests** — The client has sent too many requests within a given period/rate limit.
- **500 Internal Server Error** — The server encountered an unexpected condition while processing the request.
- **501 Not Implemented** — The server does not support the functionality required to fulfill the request.
- **502 Bad Gateway** — A gateway/proxy received an invalid response from an upstream server.
- **503 Service Unavailable** — The server is temporarily unable to handle the request, commonly due to overload or maintenance.

- **504 Gateway Timeout** — A gateway/proxy did not receive a timely response from an upstream server.



How do you handle exceptions globally?
@ExceptionHandler.
@ControllerAdvice.
@RestControllerAdvice.
How do you return a standard error response?
How do you handle validation errors?
How do you handle unexpected exceptions?
Should you return stack traces to clients?
How do you map business exceptions to HTTP status codes?
How do you log exceptions



1. How do you handle exceptions globally?

Use `@RestControllerAdvice` + `@ExceptionHandler` to centralize exception handling across controllers.

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(OrderNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleOrderNotFound(
        OrderNotFoundException ex) {

        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body(new ErrorResponse("ORDER_NOT_FOUND", ex.getMessage()));
    }
}
```

2. @ExceptionHandler

`@ExceptionHandler` defines a method that handles a **specific exception type** thrown by controller processing.

```
@ExceptionHandler(OrderNotFoundException.class)
public ResponseEntity<ErrorResponse> handle(OrderNotFoundException ex) {
    // return error response
}
```

3. @ControllerAdvice

@ControllerAdvice is a Spring mechanism for applying **exception handling and other controller-related behavior globally** across controllers.

```
@ControllerAdvice
public class GlobalExceptionHandler {
}
```

For REST APIs, @RestControllerAdvice is generally more convenient.

4. @RestControllerAdvice

@RestControllerAdvice combines the global-controller-advice behavior with response-body semantics, making it suitable for **global REST API exception handling**.

```
@RestControllerAdvice
public class GlobalExceptionHandler {
}
```

5. How do you return a standard error response?

Create a consistent error DTO:

```
public record ErrorResponse(
    String code,
    String message,
    String path,
    Instant timestamp
) {}
```

Then return the same structure for different errors:

```
{
  "code": "ORDER_NOT_FOUND",
  "message": "Order not found",
  "path": "/orders/101",
  "timestamp": "2026-09-01T10:30:00Z"
}
```

This gives clients a predictable error format.

6. How do you handle validation errors?

Use Bean Validation annotations:

```
public class CreateOrderRequest {

    @NotBlank
    private String productId;

    @Min(1)
    private int quantity;
}
```

Controller:

```
@PostMapping("/orders")
public Order create(
    @Valid @RequestBody CreateOrderRequest request) {
    // ...
}
```

Then handle validation exceptions globally:

```
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<ErrorResponse> handleValidation(
    MethodArgumentNotValidException ex) {

    // Extract field validation errors
    // Return 400 Bad Request
}
```

For validation failures, the typical HTTP status is:

```
400 Bad Request
```

7. How do you handle unexpected exceptions?

Have a final generic handler:

```
@ExceptionHandler(Exception.class)
public ResponseEntity<ErrorResponse> handleUnexpected(
    Exception ex) {

    log.error("Unexpected error", ex);

    return ResponseEntity
        .status(HttpStatus.INTERNAL_SERVER_ERROR)
        .body(new ErrorResponse(
```

```
        "INTERNAL_ERROR",
        "An unexpected error occurred"
    ));
}
```

The client receives a safe generic message, while the actual exception is logged internally.

8. Should you return stack traces to clients?

No, generally not in production.

Don't expose:

```
{
  "error": "NullPointerException",
  "stackTrace": "com.company...."
}
```

because stack traces can reveal:

- Internal class names
- Database details
- Infrastructure information
- File paths
- Implementation details

Instead return:

```
{
  "code": "INTERNAL_ERROR",
  "message": "An unexpected error occurred"
}
```

and keep the detailed stack trace in your internal logs.

9. How do you map business exceptions to HTTP status codes?

Create meaningful business exceptions and map them to appropriate HTTP statuses.

Example:

```
public class OrderNotFoundException extends RuntimeException {
}
```

Map it:

```
@ExceptionHandler(OrderNotFoundException.class)
public ResponseEntity<ErrorResponse> handle(
    OrderNotFoundException ex) {

    return ResponseEntity
        .status(HttpStatus.NOT_FOUND)
        .body(new ErrorResponse(
            "ORDER_NOT_FOUND",
            "Order was not found"
        ));
}
```

Typical mapping:

ResourceNotFoundException	→ 404
ValidationException	→ 400
AuthenticationException	→ 401
AuthorizationException	→ 403
BusinessConflictException	→ 409
UnexpectedException	→ 500

The important principle is:

HTTP status should represent the API-level meaning of the failure, not simply the Java exception class name.

10. How do you log exceptions?

For an unexpected exception, log the exception **with its stack trace** internally:

```
log.error(
    "Unexpected error while processing orderId={}",
    orderId,
    ex
);
```

For expected business exceptions, you generally don't need to log them as **ERROR** every time.

For example:

```
log.warn(
    "Order not found, orderId={}",
```

```
orderId  
);
```

Good production logging practice

Log useful context such as:

```
timestamp  
request/correlation ID  
service name  
operation  
resource ID  
exception type  
stack trace for unexpected errors
```

But avoid logging sensitive information such as:

```
passwords  
tokens  
authorization credentials  
sensitive personal data
```

A good architecture is:

```
Controller  
  ↓  
Service  
  ↓  
Exception occurs  
  ↓  
@RestControllerAdvice  
  ↓  
Map exception → HTTP status  
  ↓  
Standard ErrorResponse
```

while:

```
Exception  
  ↓  
Application logs  
  ↓  
Centralized logging/monitoring
```



What is Bean Validation?

@Valid vs @Validated.

@NotNull.

@NotBlank.

@NotEmpty.

@Size.

@Min.

@Max.

@Email.

How do you create a custom validation annotation?

How do you validate request parameters?

How do you return validation errors consistently?

How does validation work internally?



1. What is Bean Validation?

Bean Validation is a standard Java mechanism for validating object data using annotations such as `@NotNull`, `@NotBlank`, `@Size`, and `@Email`; Spring Boot commonly integrates it through Hibernate Validator.

2. @Valid vs @Validated

- `@Valid` — Triggers standard Bean Validation on an object and supports nested validation, but does not support validation groups.
- `@Validated` — Spring's variant of `@Valid` that additionally supports **validation groups** and is commonly used for method-level validation.

3. @NotNull

`@NotNull` ensures that the value is not `null`, but it allows empty strings and empty collections.

```
@NotNull
private String name;
```

4. @NotBlank

`@NotBlank` ensures a `String` is not `null`, empty, or only whitespace.

```
@NotBlank
private String name;
```

5. @NotEmpty

`@NotEmpty` ensures a string, collection, map, or array is not `null` and contains at least one element/character, but whitespace-only strings are allowed.

```
@NotEmpty
private List<String> items;
```

6. @Size

`@Size` validates the size/length of strings, collections, maps, or arrays within specified minimum and maximum limits.

```
@Size(min = 3, max = 20)
private String username;
```

7. @Min

`@Min` ensures a numeric value is greater than or equal to the specified minimum.

```
@Min(1)
private int quantity;
```

8. @Max

`@Max` ensures a numeric value is less than or equal to the specified maximum.

```
@Max(100)
private int quantity;
```

9. @Email

`@Email` validates that a string conforms to an email-like format.

```
@Email
private String email;
```

10. How do you create a custom validation annotation?

Create an annotation:

```
@Target(ElementType.FIELD)
@Retention(RetentionPolicy.RUNTIME)
@Constraint(validatedBy = StrongPasswordValidator.class)
public @interface StrongPassword {

    String message() default "Invalid password";

    Class<?>[] groups() default {};

    Class<? extends Payload>[] payload() default {};
}
```

Then create the validator:

```
public class StrongPasswordValidator
    implements ConstraintValidator<StrongPassword, String> {

    @Override
    public boolean isValid(String value, ConstraintValidatorContext context) {

        if (value == null) {
            return true;
        }

        return value.length() >= 8;
    }
}
```

Use it:

```
public class UserRequest {

    @StrongPassword
    private String password;
}
```

So the flow is:

```
@StrongPassword
    ↓
ConstraintValidator
```

```
↓  
isValid()  
↓  
true / false
```

11. How do you validate request parameters?

For request bodies:

```
@PostMapping("/users")  
public User create(  
    @Valid @RequestBody UserRequest request) {  
    // ...  
}
```

For request parameters/path variables, enable method validation and use constraints:

```
@RestController  
@Validated  
public class UserController {  
  
    @GetMapping("/users")  
    public User getUser(  
        @RequestParam  
        @Min(1)  
        Long id) {  
  
        // ...  
    }  
}
```

You can similarly validate:

```
@PathVariable  
@RequestParam  
@RequestHeader
```

depending on the requirement and validation setup.

12. How do you return validation errors consistently?

Handle validation exceptions globally using `@RestControllerAdvice`.

For example:

```

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidation(
        MethodArgumentNotValidException ex) {

        // Extract field errors
        // Build standard response

        return ResponseEntity
            .badRequest()
            .body(new ErrorResponse(
                "VALIDATION_ERROR",
                "Request validation failed"
            ));
    }
}

```

A consistent response can look like:

```

{
  "code": "VALIDATION_ERROR",
  "message": "Request validation failed",
  "errors": [
    {
      "field": "email",
      "message": "must be a valid email"
    },
    {
      "field": "quantity",
      "message": "must be greater than or equal to 1"
    }
  ]
}

```

This allows clients to handle validation failures consistently.

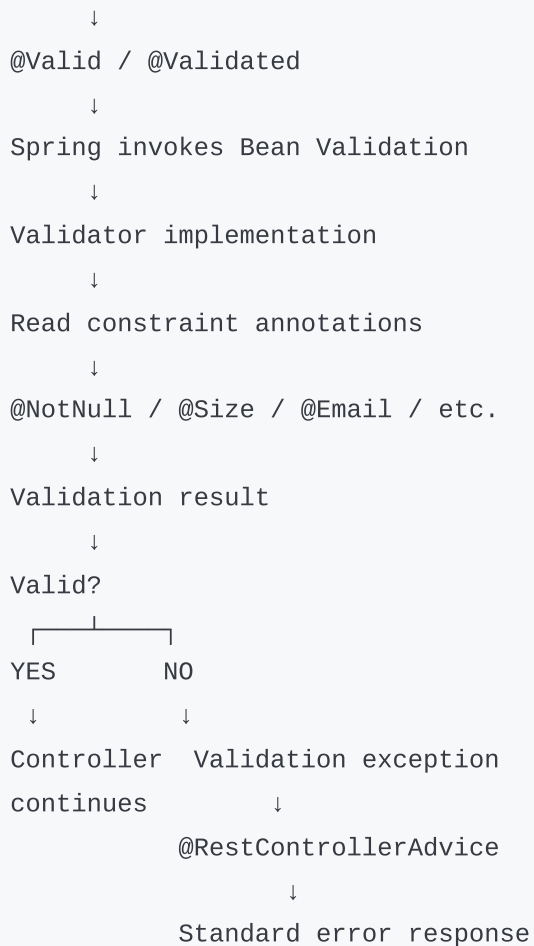
13. How does validation work internally?

The basic flow is:

```

HTTP Request
  ↓
Controller

```



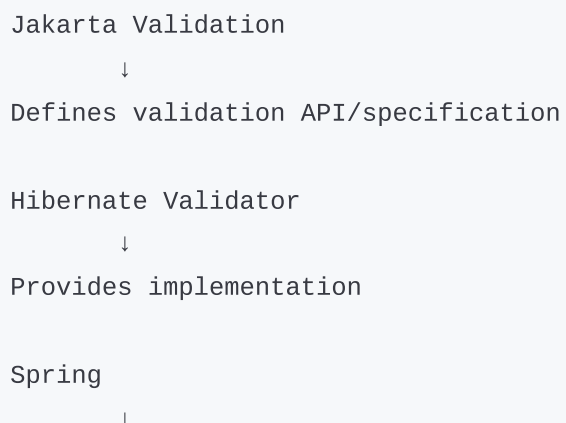
Internally, Spring integrates with the **Jakarta Bean Validation API**, while an implementation such as **Hibernate Validator** performs the actual constraint checking.

For example:

```
@Size(min = 3)
private String username;
```

The validation provider reads the constraint metadata and invokes the corresponding validator.

Important distinction



Integrates validation into controllers,
method validation, etc.



What is AOP?
Why do we need AOP?
Cross-cutting concerns.
Aspect.
Join point.
Pointcut.
Advice.
@Before.
@After.
@AfterReturning.
@AfterThrowing.
@Around.
What is a Spring proxy?
JDK dynamic proxy vs CGLIB.
How does @Transactional use AOP?
Why doesn't @Transactional work in some self-invocation cases?
What is self-invocation?
How can you fix self-invocation issues?



1. What is AOP?

AOP (Aspect-Oriented Programming) is a programming approach used to separate **cross-cutting concerns** such as logging, security, transactions, caching, and auditing from the main business logic.

2. Why do we need AOP?

AOP prevents the same infrastructure logic from being repeated across many classes.

Without AOP:

```
Service A → logging + business logic  
Service B → logging + business logic  
Service C → logging + business logic
```

With AOP:

```
Logging Aspect  
↓  
Service A → Business Logic
```

```
Service B → Business Logic  
Service C → Business Logic
```

The business code remains focused on its actual responsibility.

3. What are Cross-Cutting Concerns?

Cross-cutting concerns are functionalities that affect multiple parts/layers of an application rather than belonging to one specific business module.

Examples:

```
Logging  
Security  
Transactions  
Caching  
Auditing  
Performance monitoring  
Tracing
```

4. What is an Aspect?

An Aspect is a module/class containing logic for a cross-cutting concern and defining where and when that logic should execute.

Example:

```
@Aspect  
@Component  
public class LoggingAspect {  
}
```

5. What is a Join Point?

A Join Point is a point during program execution where an aspect can potentially be applied.

In Spring AOP, join points are primarily **method executions**.

Example:

```
orderService.createOrder();
```

The execution of `createOrder()` can be a join point.

6. What is a Pointcut?

A Pointcut is an expression that specifies which join points an aspect should intercept.

Example:

```
@Pointcut("execution(* com.example.service.*(..))")
public void serviceMethods() {
}
```

Meaning:

Apply this pointcut to methods in the service package.

Easy distinction:

Join Point → Where interception can happen

Pointcut → Which join points should be selected

7. What is Advice?

Advice is the actual code that runs when a selected join point is reached.

Examples:

```
@Before
@After
@AfterReturning
@AfterThrowing
@Around
```

Think:

Pointcut → WHERE
Advice → WHAT

8. @Before

@Before advice executes before the target method executes.

```
@Before("serviceMethods()")
public void before() {
```

```
log.info("Before method");  
}
```

Flow:

```
Before Advice  
  ↓  
Target Method
```

9. @After

@After advice executes after the target method finishes, regardless of whether it succeeds or throws an exception.

```
@After("serviceMethods()")  
public void after() {  
    log.info("Method completed");  
}
```

Flow:

```
Target Method  
  ↓  
@After
```

It is conceptually similar to a `finally` block.

10. @AfterReturning

@AfterReturning executes only when the target method completes successfully and can access its return value.

```
@AfterReturning(  
    pointcut = "serviceMethods()",  
    returning = "result"  
)  
public void afterReturning(Object result) {  
    log.info("Result: {}", result);  
}
```

Flow:

Target Method
↓
Success
↓
@AfterReturning

11. @AfterThrowing

@AfterThrowing executes when the target method throws an exception.

```
@AfterThrowing(  
    pointcut = "serviceMethods()",  
    throwing = "ex"  
)  
public void afterThrowing(Exception ex) {  
    log.error("Exception occurred", ex);  
}
```

Flow:

Target Method
↓
Exception
↓
@AfterThrowing

12. @Around

@Around advice surrounds the target method and gives you control over whether and when the target method executes.

```
@Around("serviceMethods()")  
public Object around(ProceedingJoinPoint pjp) throws Throwable {  
  
    long start = System.currentTimeMillis();  
  
    try {  
        return pjp.proceed();  
    } finally {  
        long time = System.currentTimeMillis() - start;  
        log.info("Execution time: {}", time);  
    }  
}
```

```
}  
}
```

Flow:

```
@Around  
↓  
Before logic  
↓  
pjp.proceed()  
↓  
Target Method  
↓  
After logic  
↓  
@Around
```

`@Around` is powerful because you can:

- Execute logic before the method
- Execute the method
- Execute logic after the method
- Modify arguments
- Modify return values
- Handle exceptions
- Even prevent the target method from executing

13. What is a Spring Proxy?

A **Spring proxy** is an object created by Spring that wraps a target bean and **intercepts method calls before delegating them to the actual target object.**

Conceptually:

```
Client  
|  
↓  
Proxy  
|  
+— Logging  
+— Security  
+— Transaction
```

```
+--- Caching
|
↓
Target Bean
```

For example:

```
orderService.createOrder();
```

may actually be:

```
Client
↓
OrderService Proxy
↓
Transaction interceptor
↓
Actual OrderService
↓
createOrder()
```

This proxy mechanism is fundamental to Spring AOP.

14. JDK Dynamic Proxy vs CGLIB

Spring commonly uses two proxy mechanisms.

JDK Dynamic Proxy

Creates a proxy based on **interfaces**.

```
public interface PaymentService {
    void pay();
}
```

```
public class PaymentServiceImpl
    implements PaymentService {
}
```

Conceptually:

```
Interface
↑
Proxy
```

↓

Implementation

The proxy implements the interface.

CGLIB

CGLIB-style subclass proxying creates a proxy by **subclassing the target class**.

```
PaymentService
    ↑
    |
PaymentService Proxy
```

This allows proxying classes even when they don't implement an interface.

Important modern Spring point

In modern Spring versions, class-based proxying is commonly used by default in many Spring Boot scenarios, while JDK proxies can be selected when appropriate/configured.

Key difference

```
JDK Proxy
→ Interface-based

CGLIB/class-based proxy
→ Subclass-based
```

15. How does `@Transactional` use AOP?

This is a **very important 5-year interview question**.

Suppose:

```
@Service
public class OrderService {

    @Transactional
    public void createOrder() {
        // database operations
    }
}
```

Spring creates a proxy around the service.

Conceptually:

```
Controller
  ↓
OrderService Proxy
  ↓
Transaction Interceptor
  ↓
BEGIN TRANSACTION
  ↓
Actual OrderService
  ↓
createOrder()
  ↓
COMMIT
```

If an appropriate exception causes rollback:

```
OrderService Proxy
  ↓
BEGIN TRANSACTION
  ↓
createOrder()
  ↓
Exception
  ↓
ROLLBACK
  ↓
Exception propagated
```

So `@Transactional` isn't magic inside your method.

Spring's infrastructure uses proxy-based interception to apply transaction behavior around eligible method calls.

16. Why doesn't `@Transactional` work in some self-invocation cases?

🔥 Very commonly asked.

Consider:

```
@Service
public class OrderService {
```

```

    public void processOrder() {
        createOrder();
    }

    @Transactional
    public void createOrder() {
        // database operation
    }
}

```

You might expect:

```

processOrder()
    ↓
@Transactional
    ↓
Transaction starts

```

But the call:

```
createOrder();
```

is a **direct call on the same object**.

It does not go through the Spring proxy.

Conceptually:

```

External caller
    ↓
Spring Proxy
    ↓
processOrder()
    ↓
this.createOrder()
    ↓
Actual object

```

The second call bypasses the proxy.

Therefore the transaction interceptor associated with the proxy doesn't get a chance to intercept that self-invocation.

17. What is Self-Invocation?

Self-invocation is when a method in a class calls another method of the same class directly.

Example:

```
public void methodA() {  
    methodB();  
}
```

Conceptually:

```
this.methodB();
```

The important point is:

```
External call  
  ↓  
Proxy  
  ↓  
Target
```

versus:

```
Target  
  ↓  
this.methodB()  
  ↓  
Target directly
```

The second one bypasses proxy-based AOP interception.

18. How can you fix self-invocation issues?

Solution 1 — Move the transactional method to another bean

Usually the cleanest solution.

```
@Service  
public class OrderService {  
  
    private final PaymentService paymentService;  
  
    public OrderService(PaymentService paymentService) {  
        this.paymentService = paymentService;  
    }  
}
```

```

    }

    public void processOrder() {
        paymentService.createPayment();
    }
}

```

```

@Service
public class PaymentService {

    @Transactional
    public void createPayment() {
        // transaction
    }
}

```

Now:

```

OrderService
    ↓
PaymentService Proxy
    ↓
@Transactional

```

The call crosses a Spring bean boundary and goes through the proxy.

Solution 2 — Call through the Spring proxy

You can obtain/use the proxied bean rather than calling the method directly on `this`.

However, this approach is generally less clean than restructuring the service.

Solution 3 — Self-injection

Some applications inject the proxied bean into itself:

```

@Service
public class OrderService {

    private final OrderService self;

    public OrderService(OrderService self) {
        this.self = self;
    }
}

```

```

public void process() {
    self.createOrder();
}

@Transactional
public void createOrder() {
}
}

```

The call:

```
self.createOrder();
```

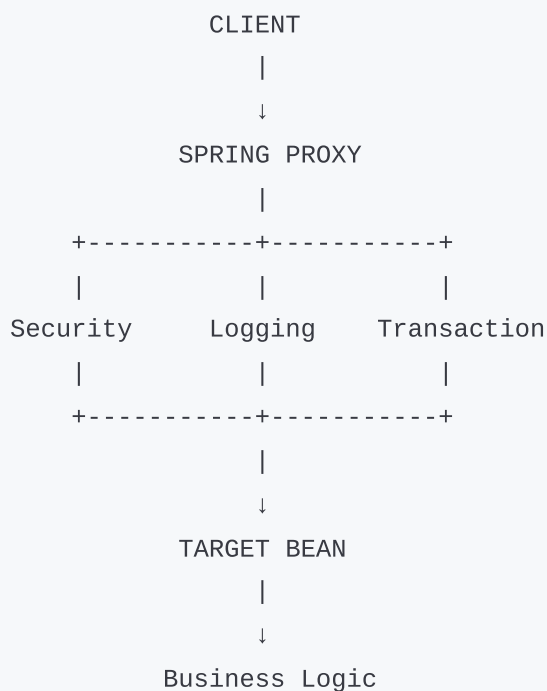
can go through the proxy.

But this can make the design confusing, so **moving the operation to another service is usually preferable**.



The Most Important AOP Mental Model

For your interview, remember this:



And the self-invocation problem:

External call:

```

Client
↓

```

Proxy



@Transactional method



Transaction works

Self invocation:

Method A



this.methodB()



Target directly



Proxy bypassed



@Transactional interception may not happen

One-line interview answer for the entire topic:

Spring AOP generally works through proxies; when a method is called through the proxy, Spring can apply cross-cutting behavior such as transactions, security, logging, and caching, but direct self-invocation bypasses the proxy and therefore bypasses proxy-based advice.



What is @Transactional?
How does @Transactional work internally?
Transaction propagation.
Transaction isolation.
Rollback behavior.
What exceptions trigger rollback by default?
Checked vs unchecked exception rollback.
rollbackFor.
noRollbackFor.
REQUIRED.
REQUIRES_NEW.
NESTED.
SUPPORTS.
MANDATORY.
NOT_SUPPORTED.
NEVER.
What happens when a transaction calls another transactional method?
Transaction boundary.
Why should transactions usually be at the service layer?
Can @Transactional be applied to private methods?
Can @Transactional work on final methods?
What happens if an exception is caught inside a transaction?
How do you manually mark a transaction rollback-only?
Very important scenario

Method A is @Transactional and calls Method B with REQUIRES_NEW. Method B fails. What happens to A's transaction?



1. What is @Transactional ?

@Transactional tells Spring to execute a method within a **database transaction**, automatically handling transaction begin, commit, and rollback.

2. How does @Transactional work internally?

Spring generally uses **AOP proxies** to intercept calls to transactional methods.

Controller



```
Spring Proxy
  ↓
Transaction Interceptor
  ↓
BEGIN TRANSACTION
  ↓
Service Method
  ↓
COMMIT / ROLLBACK
```

When the method is called through the Spring proxy, the transaction interceptor checks the `@Transactional` configuration and manages the transaction accordingly.

3. Transaction Propagation

Propagation defines what should happen when a transactional method calls another transactional method.

For example:

```
Method A → Method B
```

If both are transactional, propagation determines whether B:

- Joins A's transaction
- Creates a new transaction
- Runs without a transaction
- Requires an existing transaction

4. Transaction Isolation

Isolation defines how one transaction can see changes made by other concurrent transactions.

Common levels:

```
READ_UNCOMMITTED
READ_COMMITTED
REPEATABLE_READ
SERIALIZABLE
```

Higher isolation generally provides stronger consistency but can reduce concurrency/performance.

5. Rollback Behavior

When a transactional method fails with an exception that triggers rollback, Spring marks the transaction for rollback.

```
@Transactional
  ↓
BEGIN
  ↓
Database operations
  ↓
Exception
  ↓
ROLLBACK
```

If the method completes successfully:

```
@Transactional
  ↓
BEGIN
  ↓
Database operations
  ↓
Success
  ↓
COMMIT
```

6. What exceptions trigger rollback by default?

By default, Spring rolls back transactions for:

```
RuntimeException
Error
```

It does **not automatically roll back for checked exceptions**.

7. Checked vs Unchecked Exception Rollback

Unchecked exception

```
throw new RuntimeException();
```

Default behavior:

RuntimeException



ROLLBACK

Checked exception

```
throw new IOException();
```

Default behavior:

Checked Exception



No automatic rollback by default

You can change this using `rollbackFor`.

8. `rollbackFor`

`rollbackFor` tells Spring which exceptions should cause rollback.

```
@Transactional(rollbackFor = IOException.class)
public void process() throws IOException {
}
```

Now:

IOException



ROLLBACK

This is useful when your business operation throws a checked exception but the entire transaction should be undone.

9. `noRollbackFor`

`noRollbackFor` tells Spring **not to roll back** for a particular exception.

```
@Transactional(noRollbackFor = BusinessException.class)
public void process() {
}
```

If that exception occurs:

BusinessWarningException



No rollback

assuming no other rollback-triggering condition occurs.

10. REQUIRED

REQUIRED is the default propagation mode.

```
@Transactional(propagation = Propagation.REQUIRED)
```

Behavior:

Transaction exists?



+--+--+

YES NO



Join

Create

Example:

A @Transactional



B @Transactional(REQUIRED)

B joins A's transaction.

A Transaction



+— A



+— B

11. REQUIRES_NEW

REQUIRES_NEW always executes using a new transaction.

If A already has a transaction:

A Transaction



B

↓
REQUIRES_NEW

Spring:

A Transaction
↓
SUSPEND A
↓
START B Transaction
↓
B completes
↓
COMMIT/ROLLBACK B
↓
RESUME A

So A and B have independent transaction boundaries.

12. NESTED

NESTED executes within the existing transaction using a nested transaction mechanism, typically implemented with a database savepoint.

Conceptually:

A Transaction
↓
Savepoint
↓
B operation

If B fails and the nested mechanism supports it:

Rollback to savepoint

rather than necessarily rolling back the entire outer transaction.

Important: **NESTED** depends on the transaction manager and database support for nested/savepoint behavior.

13. SUPPORTS

SUPPORTS uses the existing transaction if one exists; otherwise it executes without a transaction.

Transaction exists?	
+--+--+	
YES	NO
Join	No transaction

14. MANDATORY

MANDATORY **requires an existing transaction and throws an exception if none exists.**

Transaction exists?	
+--+--+	
YES	NO
Join	Exception

15. NOT_SUPPORTED

NOT_SUPPORTED **executes without a transaction and suspends an existing transaction if one exists.**

```

A Transaction
  ↓
Suspend A
  ↓
B NOT_SUPPORTED
  ↓
No transaction
  ↓
Resume A

```

16. NEVER

NEVER **requires that no transaction exists and throws an exception if a transaction is already active.**

Transaction exists?	
+--+--+	
NO	YES

17. What happens when a transaction calls another transactional method?

It depends on the propagation setting.

For the default `REQUIRED` :

```
A @Transactional
    ↓
B @Transactional(REQUIRED)
```

B joins A's transaction.

There is effectively one transaction:

```
BEGIN
↓
A
↓
B
↓
COMMIT
```

With `REQUIRES_NEW` :

```
BEGIN A
↓
A
↓
SUSPEND A
↓
BEGIN B
↓
B
↓
COMMIT/ROLLBACK B
↓
RESUME A
↓
COMMIT/ROLLBACK A
```

18. What is a Transaction Boundary?

A **transaction boundary** defines where a transaction starts and where it ends.

For:

```
@Transactional
public void createOrder() {
    // operations
}
```

the transaction boundary is approximately:

```
BEGIN
  ↓
createOrder()
  ↓
COMMIT / ROLLBACK
```

The transaction boundary is especially important when multiple database operations must succeed or fail together.

19. Why should transactions usually be at the service layer?

Because the **service layer represents a business operation** and often coordinates multiple repository/database operations.

Example:

```
createOrder()
|
+— saveOrder()
+— updateInventory()
+— createPaymentRecord()
```

You usually want these operations to participate in one business transaction.

```
@Transactional
public void createOrder() {
    orderRepository.save(...);
    inventoryRepository.update(...);
    paymentRepository.save(...);
}
```

So:

```
Controller
  ↓
Service ← Transaction boundary
  ↓
Repository
```

This keeps transaction management aligned with business operations rather than individual database calls.

20. Can `@Transactional` be applied to private methods?

Generally, no—not with Spring's normal proxy-based transaction management.

Example:

```
@Transactional
private void saveOrder() {
}
```

Calling this method internally does not go through the Spring proxy.

Therefore the transactional advice won't be applied as expected.

21. Can `@Transactional` work on final methods?

With **proxy-based Spring AOP**, a `final` method cannot be overridden by a subclass proxy, so class-based proxying cannot advise it.

Therefore:

`@Transactional` should generally not be placed on final methods when relying on Spring's proxy-based transaction management.

22. What happens if an exception is caught inside a transaction?

Example:

```
@Transactional
public void process() {

    try {
        saveOrder();
    }
}
```

```

    } catch (Exception e) {
        log.error("Error", e);
    }

    updateInventory();
}

```

If the exception is caught and not rethrown, Spring may see the method as completing successfully.

Therefore, the transaction can potentially **commit**, even though an operation inside it failed.

Conceptually:

```

BEGIN
  ↓
saveOrder()
  ↓
Exception
  ↓
caught
  ↓
continue
  ↓
method returns normally
  ↓
COMMIT

```

If you want rollback despite catching the exception, explicitly mark the transaction rollback-only.

23. How do you manually mark a transaction rollback-only?

Using Spring's `TransactionAspectSupport`:

```

TransactionAspectSupport
    .currentTransactionStatus()
    .setRollbackOnly();

```

Example:

```

@Transactional
public void process() {

```

```

try {
    saveOrder();
} catch (Exception e) {

    TransactionAspectSupport
        .currentTransactionStatus()
        .setRollbackOnly();

    log.error("Processing failed", e);
}
}

```

Now when the method completes:

```

Transaction marked rollback-only
    ↓
Method returns
    ↓
Spring attempts commit
    ↓
Transaction rolls back

```

🔥 24. Very Important Scenario

Method A is `@Transactional` and calls Method B with `REQUIRES_NEW`. Method B fails. What happens to A's transaction?

Suppose:

```

@Transactional
public void methodA() {
    // A operations

    methodB();

    // more A operations
}

```

and B:

```

@Transactional(propagation = Propagation.REQUIRES_NEW)
public void methodB() {

```

```
// B operations  
}
```

Assuming B is invoked through the Spring proxy:

Step 1 — A starts

```
A  
↓  
BEGIN Transaction A
```

Step 2 — A calls B

Because B uses:

```
REQUIRES_NEW
```

Spring suspends A:

```
Transaction A  
↓  
SUSPEND
```

Step 3 — B starts a new transaction

```
Transaction B  
↓  
B operations
```

Step 4 — B fails

Suppose B throws a rollback-triggering exception:

```
Transaction B  
↓  
Exception  
↓  
ROLLBACK B
```

Step 5 — A resumes

```
ROLLBACK B
```

↓

```
RESUME A
```

What happens to A?

A's transaction is not automatically rolled back merely because B's independent `REQUIRES_NEW` transaction rolled back.

A can continue:

```
Transaction A
```

↓

```
more A operations
```

↓

```
COMMIT A
```

So the final result can be:

```
Transaction A → COMMIT
```

```
Transaction B → ROLLBACK
```

But there is an important caveat

If A allows B's exception to propagate and A's transaction itself encounters a rollback-triggering exception, then **A can also roll back**.

For example:

```
A
```

↓

```
B (REQUIRES_NEW)
```

↓

```
B fails → B rolls back
```

↓

```
exception propagates to A
```

↓

```
A doesn't handle it
```

↓

```
A rolls back
```

So the key principle is:

`REQUIRES_NEW` gives *B* an independent transaction. *B*'s rollback does not inherently roll back *A*, but how the exception is propagated and handled can still cause *A*'s transaction to roll back.



JPA vs Hibernate.
What is ORM?
Entity.
@Entity.
@Id.
@GeneratedValue.
Entity lifecycle.
Transient state.
Persistent state.
Detached state.
Removed state.
Persistence Context.
First-level cache.
Dirty checking.
Flush vs commit.
save() vs saveAndFlush().
Lazy vs eager loading.
N+1 query problem.
How do you solve N+1?
Cascade types.
orphanRemoval.
mappedBy.
Owning side of relationship.
One-to-One.
One-to-Many.
Many-to-One.
Many-to-Many.



1. JPA vs Hibernate

- **JPA** — A Java specification/API that defines how Java applications should perform ORM and persistence.
- **Hibernate** — A popular implementation of the JPA specification that provides the actual ORM functionality.

JPA



Specification / API

Hibernate

↓
Implementation

2. What is ORM?

ORM (Object-Relational Mapping) is a technique that maps Java objects/classes to database tables and Java fields to database columns.

Java Object		Database
↓		↓
Order	↔	orders
id	↔	id
amount	↔	amount

3. What is an Entity?

An Entity is a Java class whose objects are managed and persisted by JPA/Hibernate, usually corresponding to a database table.

```
@Entity
public class Order {
}
```

4. @Entity

@Entity marks a Java class as a JPA entity that can be persisted to a database.

```
@Entity
public class Order {
}
```

5. @Id

@Id marks the field/property that represents the primary key of the entity.

```
@Id
private Long id;
```

6. @GeneratedValue

@GeneratedValue tells JPA that the entity's primary-key value should be generated automatically.

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
```

7. Entity Lifecycle

An entity generally moves through these states:

```
Transient
  ↓
Persistent
  ↓
Detached
  ↓
Removed
```

8. Transient State

A transient entity is a newly created Java object that is not currently associated with a persistence context and has not been persisted.

```
Order order = new Order();
```

At this point:

```
Java Object
  ↓
Not managed by JPA
```

9. Persistent State

A persistent entity is attached to the persistence context and is managed by JPA/Hibernate.

```
entityManager.persist(order);
```

Now Hibernate tracks changes to the entity.

```
Persistence Context
  ↓
Order
```

↓
Hibernate tracks it

10. Detached State

A detached entity was previously persistent but is no longer associated with the current persistence context.

For example:

```
entityManager.detach(order);
```

or when the persistence context is closed.

```
Persistence Context
  ↓
Order

close context
  ↓
Order
  ↓
Detached
```

Changes to a detached entity are not automatically tracked by that persistence context.

11. Removed State

A removed entity is a managed entity that has been marked for deletion from the database.

```
entityManager.remove(order);
```

The actual SQL `DELETE` is generally executed when the persistence context is flushed.

12. What is Persistence Context?

Persistence Context is a managed set of entity instances maintained by JPA's `EntityManager`, where each database identity normally corresponds to a single managed entity instance within that context.

Conceptually:

```
EntityManager
  ↓
```

Persistence Context

|
+— Order#1
+— Order#2
+— Customer#5

It tracks:

- Managed entities
- Entity identity
- Changes to entities
- Insert/update/delete operations
- First-level cache

13. First-Level Cache

The first-level cache is the persistence-context-level cache maintained by the `EntityManager` and is enabled by default.

Example:

```
Order order1 = entityManager.find(Order.class, 1L);  
Order order2 = entityManager.find(Order.class, 1L);
```

Within the same persistence context, Hibernate can return the same managed entity instance rather than issuing another database query.

```
First find  
  ↓  
DB query  
  ↓  
Persistence Context  
  
Second find  
  ↓  
Persistence Context  
  ↓  
Existing entity
```

It is **not a global application-wide cache**.

14. Dirty Checking

Dirty checking is Hibernate's mechanism for detecting changes made to managed entities and automatically generating SQL updates during flush.

Example:

```
@Transactional
public void updateOrder() {

    Order order = repository.findById(1L).get();

    order.setAmount(500);
}
```

You don't necessarily need:

```
repository.save(order);
```

because the entity is managed.

Conceptually:

```
Load entity
  ↓
Original state tracked
  ↓
Change entity
  ↓
Flush
  ↓
Hibernate detects difference
  ↓
UPDATE SQL
```

15. Flush vs Commit

Flush

Flush synchronizes changes from the persistence context to the database by executing required SQL statements, but it does not itself commit the transaction.

```
Persistence Context
  ↓
Flush
  ↓
```

SQL executed



Database

Commit

Commit permanently completes the database transaction and makes its changes durable according to the database's transaction semantics.

Transaction



Commit



Transaction completed

Important

Flush ≠ Commit

A flush can happen before the transaction commits.

16. `save()` vs `saveAndFlush()`

In Spring Data JPA:

- `save()` — Saves the entity through the repository, but the SQL may not be sent to the database immediately; synchronization normally occurs at flush/transaction completion.
- `saveAndFlush()` — Saves the entity and explicitly flushes the persistence context immediately.

Conceptually:

`save()`



Persistence Context



Later flush



DB

versus:

```
saveAndFlush()  
↓  
Persistence Context  
↓  
Immediate flush  
↓  
DB
```

Flush still does not mean commit.

17. Lazy vs Eager Loading

Lazy Loading

Lazy loading means the related entity/collection is loaded only when it is accessed.

```
@OneToMany(fetch = FetchType.LAZY)  
private List<Order> orders;
```

Conceptually:

```
Load Customer  
↓  
Orders NOT loaded  
  
customer.getOrders()  
↓  
Load Orders
```

Eager Loading

Eager loading means the relationship is intended to be loaded immediately along with the owning entity.

```
@ManyToOne(fetch = FetchType.EAGER)  
private Customer customer;
```

Important

Lazy loading is generally preferred for many relationships because eager loading can unnecessarily load large object graphs.

18. What is the N+1 Query Problem?

N+1 occurs when one query loads N parent records and then additional queries are executed to load related data for each parent.

Example:

```
1 query:
SELECT * FROM orders;

Then:

N queries:
SELECT * FROM customer WHERE id = ?
SELECT * FROM customer WHERE id = ?
SELECT * FROM customer WHERE id = ?
...
```

Total:

```
1 + N queries
```

For 100 orders:

```
1 + 100 = 101 queries
```

This can significantly hurt performance.

19. How do you solve N+1?

Common approaches include:

JOIN FETCH

```
@Query("""
    SELECT o
    FROM Order o
    JOIN FETCH o.customer
""")
List<Order> findOrdersWithCustomer();
```

This can fetch the required relationship in one query.

Entity Graph

```
@EntityGraph(attributePaths = "customer")
List<Order> findAll();
```

Batch fetching

Configure Hibernate to fetch related entities in batches rather than one at a time.

DTO/projection queries

Fetch exactly the data required by the API rather than loading an entire entity graph.

Important

Don't blindly change everything to `EAGER` to fix $N+1$; that can create different performance problems.

20. Cascade Types

Cascade defines which persistence operations performed on a parent entity are propagated to its related entities.

Common cascade types:

```
PERSIST
MERGE
REMOVE
REFRESH
DETACH
ALL
```

Example:

```
@OneToMany(cascade = CascadeType.ALL)
private List<OrderItem> items;
```

Now certain lifecycle operations on the parent can cascade to its children.

21. `CascadeType.PERSIST`

`PERSIST` **propagates the persist operation from the parent to the associated entity.**

```
persist(Order)
↓
```

```
persist(OrderItem)
```

22. `CascadeType.MERGE`

MERGE **propagates the merge operation from the parent to associated entities.**

```
merge(Order)
  ↓
merge(OrderItem)
```

23. `CascadeType.REMOVE`

REMOVE **propagates deletion from the parent to associated entities.**

```
delete(Order)
  ↓
delete(OrderItem)
```

Use this carefully because deleting the parent can cause related records to be deleted.

24. `CascadeType.ALL`

ALL **enables all JPA cascade operations for the relationship.**

```
cascade = CascadeType.ALL
```

Equivalent conceptually to:

```
PERSIST
MERGE
REMOVE
REFRESH
DETACH
```

25. `orphanRemoval`

`orphanRemoval = true` **tells JPA to delete a child entity when it is removed from the parent's relationship/collection and becomes an orphan.**

Example:

```
@OneToMany(
    cascade = CascadeType.ALL,
```

```

        orphanRemoval = true
    )
    private List<OrderItem> items;

```

If:

```

order.getItems().remove(item);

```

the orphaned `OrderItem` can be deleted from the database during synchronization.

Cascade vs orphanRemoval

```

Cascade REMOVE
    ↓
Parent deleted
    ↓
Child deleted

orphanRemoval
    ↓
Child removed from relationship
    ↓
Child deleted

```

26. `mappedBy`

`mappedBy` **tells JPA that the relationship is mapped by the other entity's field and that this side is not the owning side of the relationship.**

Example:

```

@OneToMany(mappedBy = "order")
private List<OrderItem> items;

```

Here:

```

Order
    ↓
items

```

is the inverse side, while:

```

@ManyToOne

```

```
private Order order;
```

is the owning side.

27. What is the Owning Side?

The owning side is the side of a bidirectional relationship that controls the database relationship mapping, typically through the foreign-key column.

Example:

```
@Entity
class Order {

    @OneToMany(mappedBy = "order")
    private List<OrderItem> items;
}
```

```
@Entity
class OrderItem {

    @ManyToOne
    @JoinColumn(name = "order_id")
    private Order order;
}
```

Here:

```
Order
  ↓
inverse side

OrderItem
  ↓
owning side
  ↓
order_id foreign key
```

A key interview point:

`mappedBy` points to the field on the owning side.

28. One-to-One

One-to-One means one entity instance is associated with exactly one instance of another entity.

Example:

User
↓
Passport

```
@OneToOne
@JoinColumn(name = "passport_id")
private Passport passport;
```

29. One-to-Many

One-to-Many means one entity is associated with multiple entities.

Example:

Order
↓
OrderItem
OrderItem
OrderItem

```
@OneToMany(mappedBy = "order")
private List<OrderItem> items;
```

30. Many-to-One

Many-to-One means multiple entities are associated with one entity.

Example:

OrderItem —┐
OrderItem —┴─> Order
OrderItem —┐

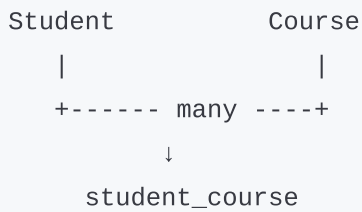
```
@ManyToOne
@JoinColumn(name = "order_id")
private Order order;
```

This is very common because the many-side usually contains the foreign key.

31. Many-to-Many

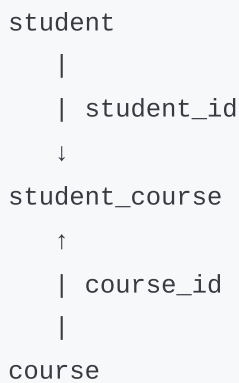
Many-to-Many means multiple instances on each side can be associated with multiple instances on the other side, typically represented using a join table.

Example:



```
@ManyToMany
@JoinTable(
    name = "student_course",
    joinColumns = @JoinColumn(name = "student_id"),
    inverseJoinColumns = @JoinColumn(name = "course_id")
)
private Set<Course> courses;
```

Database structure:



The join table stores the associations.



What is Spring Data JPA?
JpaRepository vs CrudRepository.
How does Spring generate repository implementations?
Derived query methods.
@Query.
JPQL vs native SQL.
Pagination.
Sorting.
Page vs Slice.
Projections.
Specifications.
Dynamic queries.
save() behavior.
Transaction boundaries.
Auditing.
@CreatedDate.
@LastModifiedDate.
@CreatedBy.
@LastModifiedBy.



1. What is Spring Data JPA?

Spring Data JPA is a Spring module that simplifies database access using JPA by providing repository abstractions, query generation, pagination, sorting, and more.

2. JpaRepository VS CrudRepository

- `CrudRepository` — Provides basic CRUD operations such as `save()`, `findById()`, `findAll()`, and `delete()`.
- `JpaRepository` — Extends `PagingAndSortingRepository` / CRUD functionality and adds JPA-specific features such as flushing and batch-related operations.

```
public interface UserRepository
    extends JpaRepository<User, Long> {
}
```

Interview answer: Use `JpaRepository` when you are specifically working with JPA and need its additional functionality; use `CrudRepository` when only basic CRUD abstraction is required.

3. How does Spring generate repository implementations?

Spring Data creates a runtime proxy implementation for your repository interface and uses repository infrastructure to execute CRUD operations and queries.

```
UserRepository interface
    ↓
Spring Data JPA
    ↓
Runtime Proxy
    ↓
EntityManager
    ↓
Hibernate
    ↓
Database
```

You normally don't implement:

```
findByEmail(...)
```

yourself.

Spring Data interprets the method name and creates the query behavior.

4. Derived Query Methods

Derived query methods are repository methods where Spring Data derives the query from the method name.

```
List<User> findByName(String name);

Optional<User> findByEmail(String email);

List<User> findByAgeGreaterThan(int age);
```

For:

```
findByEmail(String email)
```

Spring derives a query conceptually similar to:

```
SELECT * FROM users WHERE email = ?
```

5. @Query

@Query allows you to explicitly define a JPQL or native SQL query on a repository method.

```
@Query("SELECT u FROM User u WHERE u.email = :email")
Optional<User> findUser(
    @Param("email") String email);
```

Use it when a derived method name becomes too complex or when you need precise query control.

6. JPQL vs Native SQL

JPQL

JPQL queries entities and their fields rather than database tables and columns.

```
@Query("""
    SELECT u
    FROM User u
    WHERE u.email = :email
""")
```

It uses:

```
User → Entity
email → Entity field
```

Native SQL

Native SQL executes database-specific SQL directly against tables and columns.

```
@Query(
    value = "SELECT * FROM users WHERE email = :email",
    nativeQuery = true
)
```

Key difference

```
JPQL
↓
Entity-oriented
↓
```

Database independent to a greater extent

Native SQL

↓

Table-oriented

↓

Can use database-specific SQL features

7. Pagination

Pagination retrieves data in smaller pages instead of loading the entire result set at once.

Repository:

```
Page<User> findAll(Pageable pageable);
```

Usage:

```
Pageable pageable =  
    PageRequest.of(0, 20);  
  
Page<User> users =  
    repository.findAll(pageable);
```

Meaning:

```
page = 0  
size = 20
```

This is important when a table contains thousands or millions of records.

8. Sorting

Sorting allows query results to be returned according to one or more fields in ascending or descending order.

```
Sort sort =  
    Sort.by("name").ascending();  
  
List<User> users =  
    repository.findAll(sort);
```

With pagination:

```
Pageable pageable =
    PageRequest.of(
        0,
        20,
        Sort.by("createdDate").descending()
    );
```

9. Page VS Slice

Page

Page contains the requested data plus pagination metadata such as total elements and total pages.

```
Page<User>
```

It may require an additional count query to determine the total number of records.

Slice

Slice tells you whether another slice/page exists but does not calculate the total number of elements/pages.

```
Slice<User>
```

Key difference

Page

↓

Content

Total elements

Total pages

Has next

Slice

↓

Content

Has next

Use Slice when you don't need the total count and want to avoid the count-query overhead.

10. Projections

Projections allow you to retrieve only selected fields instead of loading the complete entity.

Example interface projection:

```
public interface UserView {  
    String getName();  
    String getEmail();  
}
```

Repository:

```
List<UserView> findByAge(int age);
```

Instead of:

```
User  
├─ id  
├─ name  
├─ email  
├─ address  
└─ ...
```

you can retrieve only:

```
name  
email
```

This can reduce unnecessary data loading.

11. Specifications

Spring Data JPA Specifications provide a programmatic way to build reusable and dynamic database predicates using the JPA Criteria API.

Example:

```
public class UserSpecification {  
  
    public static Specification<User> hasAge(int age) {  
        return (root, query, cb) ->  
            cb.equal(root.get("age"), age);  
    }  
}
```

```
}  
}
```

Repository:

```
public interface UserRepository  
    extends JpaRepository<User, Long>,  
        JpaSpecificationExecutor<User> {  
  
}
```

Then:

```
repository.findAll(  
    UserSpecification.hasAge(25)  
);
```

12. Dynamic Queries

Dynamic queries are queries whose filtering conditions are built at runtime based on the user's/search criteria.

For example, a search API may receive:

```
name = Manoj  
age = 25  
city = Delhi  
status = ACTIVE
```

But another request might provide only:

```
city = Delhi  
status = ACTIVE
```

Specifications are commonly used:

```
Search Criteria  
  ↓  
Build predicates dynamically  
  ↓  
Specification  
  ↓  
JPA query  
  ↓  
Database
```

Other approaches include Criteria API, QueryDSL, and custom repository implementations.

13. `save()` Behavior

`save()` **either persists a new entity or merges an existing/detached entity based on how Spring Data JPA determines whether the entity is new.**

Conceptually:

```
save(entity)
  ↓
Is entity new?
  /      \
YES       NO
  ↓       ↓
persist() merge()
```

For a new entity:

```
repository.save(newUser);
```

typically results in `EntityManager.persist()`.

For an existing/detached entity, Spring Data may use:

```
EntityManager.merge()
```

Important:

`save()` *does not necessarily mean an immediate SQL `INSERT / UPDATE` ; SQL execution is normally synchronized with the persistence context during flush/transaction processing.*

14. Transaction Boundaries

A transaction boundary defines the scope within which database operations are executed atomically.

Typically:

```
@Service
public class OrderService {

    @Transactional
    public void createOrder() {
        orderRepository.save(order);
    }
}
```

```
        inventoryRepository.update(inventory);
    }
}
```

Conceptually:

```
BEGIN TRANSACTION
    ↓
save Order
    ↓
update Inventory
    ↓
COMMIT
```

If a rollback-triggering failure occurs:

```
BEGIN
    ↓
Order
    ↓
Inventory
    ↓
Exception
    ↓
ROLLBACK
```

For business operations involving multiple repository calls, the **service layer** is commonly the appropriate transaction boundary.

15. Auditing

Spring Data JPA auditing automatically records metadata about when an entity was created/modified and, when configured, who created/modified it.

Example:

```
@Entity
@EntityListeners(AuditingEntityListener.class)
public class User {

    @CreatedDate
    private Instant createdDate;

    @LastModifiedDate
    private Instant lastModifiedDate;
```

```
@CreatedBy
private String createdBy;

@LastModifiedBy
private String lastModifiedBy;
}
```

You also enable auditing:

```
@EnableJpaAuditing
```

16. @CreatedDate

@CreatedDate automatically stores the date/time when the entity is initially created.

```
@CreatedDate
private Instant createdDate;
```

Example:

```
createdDate = 2026-09-01T10:30:00Z
```

17. @LastModifiedDate

@LastModifiedDate automatically stores the date/time when the entity is modified.

```
@LastModifiedDate
private Instant lastModifiedDate;
```

Conceptually:

```
Create
↓
createdDate = T1
lastModifiedDate = T1

Update
↓
createdDate = T1
lastModifiedDate = T2
```

18. @CreatedBy

`@CreatedBy` automatically stores the identity of the user/principal who created the entity, provided an `AuditorAware` implementation is configured.

```
@CreatedBy
private String createdBy;
```

For example:

```
createdBy = "user123"
```

19. @LastModifiedBy

`@LastModifiedBy` automatically stores the identity of the user/principal who most recently modified the entity, provided auditing is configured with `AuditorAware`.

```
@LastModifiedBy
private String lastModifiedBy;
```

Conceptually:

```
Created:
createdBy = userA

Modified later:
lastModifiedBy = userB
```

Auditing flow

```
Entity
↓
Spring Data Auditing
↓
CreateDate / LastModifiedDate
CreatedBy / LastModifiedBy
↓
Entity persisted
↓
Database
```

`@CreatedBy` and `@LastModifiedBy` require Spring to know the current auditor, commonly through an `AuditorAware<T>` implementation.



Authentication vs authorization.
What is Spring Security?
Security filter chain.
What is SecurityFilterChain?
JWT authentication.
OAuth2.
Access token vs refresh token.
Role vs authority.
401 vs 403.
How does JWT authentication work?
Where should JWT be validated?
How do you secure specific endpoints?
Method-level security.
@PreAuthorize.
Password hashing.
Why should passwords never be encrypted using reversible encryption?
CORS.
Session-based vs stateless authentication.
How do microservices validate JWTs



1. Authentication vs Authorization

- **Authentication** — Verifies **who the user is**.
- **Authorization** — Determines **what the authenticated user is allowed to do**.

Login

↓

Authentication → Who are you?

↓

Authorization → What can you access?

2. What is Spring Security?

Spring Security is a Spring framework that provides authentication, authorization, protection against common web attacks, and security integration for applications.

3. Security Filter Chain

Security Filter Chain is a sequence of security filters through which an incoming HTTP request passes before reaching the controller.

```
HTTP Request
  ↓
Security Filters
  ↓
Authentication
  ↓
Authorization
  ↓
Controller
```

Examples of security responsibilities include:

- JWT authentication
- Username/password authentication
- CORS/security headers
- Authorization checks
- CSRF protection

4. What is `SecurityFilterChain` ?

`SecurityFilterChain` **is the Spring Security configuration that defines which security filters apply and how requests should be authenticated/authorized.**

Example:

```
@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {

    http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/public/**").permitAll()
            .requestMatchers("/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated()
        );

    return http.build();
}
```

5. JWT Authentication

JWT authentication uses a signed JSON Web Token containing claims about the authenticated user, which the client sends with subsequent requests.

Typical header:

```
Authorization: Bearer <JWT>
```

6. OAuth2

OAuth 2.0 is an authorization framework that allows an application to obtain limited access to protected resources without sharing the user's credentials with the resource server.

Example:

```
User
  ↓
Authorization Server
  ↓
Access Token
  ↓
Client
  ↓
Resource Server
```

OAuth2 is primarily an **authorization framework**, while authentication is commonly handled through related protocols such as **OpenID Connect (OIDC)**.

7. Access Token vs Refresh Token

- **Access Token** — Short-lived token used to access protected APIs.
- **Refresh Token** — Longer-lived credential used to obtain a new access token without requiring the user to authenticate again.

```
Refresh Token
  ↓
Authorization Server
  ↓
New Access Token
  ↓
API
```

8. Role vs Authority

- **Role** — A higher-level permission grouping, commonly represented as `ROLE_ADMIN`.
- **Authority** — A specific permission granted to a user, such as `ORDER_READ` or `ORDER_DELETE`.

Example:

```
Role:
ROLE_ADMIN

Authorities:
ORDER_READ
ORDER_CREATE
ORDER_DELETE
USER_READ
```

A role can therefore be used as a convenient grouping of authorities.

9. 401 vs 403

- **401 Unauthorized** — The request lacks valid authentication credentials.
- **403 Forbidden** — The user is authenticated but does not have sufficient permission.

```
No valid authentication
    ↓
    401

Authenticated
    ↓
Insufficient permission
    ↓
    403
```

10. How does JWT authentication work?

Typical flow:

```
1. User logs in
    ↓
2. Authentication Server verifies credentials
    ↓
3. JWT/access token generated
```

↓

4. Client sends JWT with API request
- ↓
- Authorization: Bearer <JWT>
- ↓
5. Spring Security filter extracts token
- ↓
6. Token signature/claims/expiration validated
- ↓
7. Authentication placed in SecurityContext
- ↓
8. Authorization rules checked
- ↓
9. Controller executes

11. Where should JWT be validated?

JWT should be validated at the security boundary before protected business logic executes, typically in the Spring Security filter chain/resource server layer.

Validation should generally include:

- Signature
- Expiration
- Issuer
- Audience
- Required claims

The controller should not normally contain JWT validation logic.

12. How do you secure specific endpoints?

Configure authorization rules in `SecurityFilterChain`.

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/public/**").permitAll()
    .requestMatchers("/admin/**").hasRole("ADMIN")
    .requestMatchers("/orders/**").hasAuthority("ORDER_READ")
    .anyRequest().authenticated()
)
```

Conceptually:

```
/public/**      → Anyone
/admin/**       → ADMIN
/orders/**      → ORDER_READ
Everything else → Authenticated
```

13. Method-Level Security

Method-level security allows authorization rules to be applied directly to service/controller methods rather than only URL endpoints.

Enable it:

```
@EnableMethodSecurity
```

Then:

```
@PreAuthorize("hasRole('ADMIN')")
public void deleteUser(Long id) {
}
```

This is useful when authorization depends on business-level rules.

14. @PreAuthorize

@PreAuthorize checks an authorization expression before the method executes.

Example:

```
@PreAuthorize("hasRole('ADMIN')")
public void deleteUser(Long id) {
}
```

You can also check authorities:

```
@PreAuthorize("hasAuthority('ORDER_DELETE')")
public void deleteOrder(Long id) {
}
```

You can use method parameters in more advanced authorization expressions as well.

15. Password Hashing

Password hashing converts a password into a one-way representation so the original password is not stored.

Use password-hashing algorithms designed for passwords, such as:

Argon2
bcrypt
PBKDF2
scrypt

Example concept:

Password
↓
Password Hashing Algorithm
↓
Stored Hash

During login:

Entered Password
↓
Password Encoder
↓
Compare with Stored Hash

16. Why should passwords never be encrypted using reversible encryption?

Encryption is reversible when the key is available, whereas passwords should be stored using a password-hashing algorithm so the original password cannot simply be decrypted from the database.

Bad:

Password
↓
Encryption
↓
Encrypted password
↓
Decrypt
↓
Password

Preferred:

```
Password
  ↓
Password Hash
  ↓
Stored Hash
```

Even if the database is compromised, attackers should not receive plaintext passwords or a reversible encrypted form.

17. CORS

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that controls whether a web page from one origin can make requests to another origin.

Example:

```
Frontend:
https://app.example.com

Backend:
https://api.example.com
```

The backend can specify which origins, HTTP methods, and headers are allowed.

Important:

*CORS is primarily a **browser-enforced policy**; it is not a mechanism for authenticating API clients.*

18. Session-Based vs Stateless Authentication

Session-Based

The server maintains authentication state, typically using a session identifier stored by the client.

```
Client
  ↓
Session ID
  ↓
Server Session
  ↓
User information
```

Stateless

The server does not maintain per-user login session state and instead receives the authentication information with each request, commonly through an access token such as a JWT.

```
graph TD
    Client --> JWT
    JWT --> Server[Server validates JWT]
    Server --> Request[Request authorized]
```

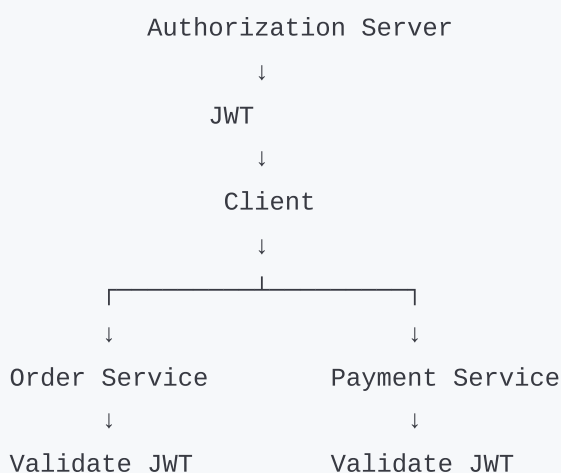
Key difference

Session-based → Authentication state maintained server-side

Stateless → Authentication state represented in the request/token

19. How do microservices validate JWTs?

In a typical microservices architecture:



Each protected microservice can act as a **resource server** and validate the token.

Typical validation:

```
graph TD
    JWT[JWT received] --> Check[Check signature]
    Check --> Next[ ]
```

```
graph TD; A[Check expiration] --> B[Check issuer]; B --> C[Check audience where applicable]; C --> D[Extract claims]; D --> E[Create Authentication]; E --> F[Check authorities/roles]; F --> G[Allow or reject request];
```

A common production approach is for services to obtain the authorization server's public keys through its standard key-discovery mechanism and validate JWT signatures locally, rather than calling the authorization server for every request.

The important microservices principle is:

Authentication can be centralized, but each resource service should enforce authorization for the resources it owns.



What is Actuator?
Health endpoint.
Metrics.
Info.
Environment.
Beans.
Loggers.
Readiness vs liveness.
Why should Actuator endpoints be secured?
How do you expose custom health checks?
How do you monitor API latency?
How do you monitor error rates?
What is Micrometer?
Micrometer vs Actuator.
How would you integrate Prometheus/Grafana?



1. What is Actuator?

Spring Boot Actuator provides production-ready endpoints and features for monitoring and managing a Spring Boot application.

2. Health Endpoint

The health endpoint shows whether the application and its important dependencies are healthy.

```
/actuator/health
```

Example:

```
{  
  "status": "UP"  
}
```

It can also report dependency health such as database, Redis, Kafka, etc., when appropriate health indicators are configured.

3. Metrics

The metrics endpoint exposes application and JVM metrics such as request counts, latency, memory, CPU, and database-related metrics.

```
/actuator/metrics
```

4. Info

The info endpoint exposes application information configured by the application.

```
/actuator/info
```

For example:

```
Application version  
Build information  
Git information
```

5. Environment

The environment endpoint shows Spring's configuration environment, including properties and active profiles, and should be carefully secured because it may expose sensitive configuration details.

```
/actuator/env
```

6. Beans

The beans endpoint shows information about beans managed by the Spring IoC container.

```
/actuator/beans
```

It is mainly useful for debugging and understanding the application context.

7. Loggers

The loggers endpoint allows you to inspect and, where permitted/configured, change logging levels at runtime.

```
/actuator/loggers
```

For example:

```
com.example.service → DEBUG
```

8. Readiness vs Liveness

Liveness

Liveness indicates whether the application instance is alive and should continue running.

```
Liveness = "Should the platform restart this application?"
```

A liveness failure may cause a platform such as Kubernetes to restart the instance.

Readiness

Readiness indicates whether the application is ready to receive traffic.

```
Readiness = "Should this instance receive traffic?"
```

For example:

```
Application starting
  ↓
Liveness → UP
Readiness → DOWN
  ↓
Application initialization completes
  ↓
Readiness → UP
  ↓
Traffic allowed
```

This distinction prevents a temporarily unready application from unnecessarily being restarted.

9. Why should Actuator endpoints be secured?

Actuator endpoints can expose sensitive application and infrastructure information, so they should not normally be publicly accessible.

For example:

```
/env
/beans
/loggers
```

may reveal configuration or internal implementation details.

Production practice:

```
Internet
  ↓
Authentication / Authorization
  ↓
Actuator
```

Usually expose only the endpoints that are actually required.

10. How do you expose custom health checks?

Implement a custom `HealthIndicator`.

```
@Component("paymentService")
public class PaymentHealthIndicator
    implements HealthIndicator {

    @Override
    public Health health() {

        boolean healthy = checkPaymentService();

        if (healthy) {
            return Health.up().build();
        }

        return Health.down()
            .withDetail("reason", "Payment service unavailable")
            .build();
    }
}
```

Then:

```
/actuator/health
```

can include the custom health information.

11. How do you monitor API latency?

Use **Micrometer metrics** integrated with Spring Boot Actuator.

Spring Boot can automatically record HTTP server request metrics.

Conceptually:

```
API Request
  ↓
Spring Boot
  ↓
Micrometer
  ↓
Timer / Histogram
  ↓
Monitoring system
```

You can monitor metrics such as:

```
Request count
Request duration
Percentiles
HTTP status
Endpoint
```

For example, you might monitor:

```
p50 latency
p95 latency
p99 latency
```

p95 = 95% of requests completed within that latency value.

12. How do you monitor error rates?

Use HTTP request metrics and group requests by status/outcome.

Conceptually:

```
Total requests = 10,000
5xx errors      = 200

Error rate = 200 / 10,000
           = 2%
```

You would typically monitor:

```
4xx rate
5xx rate
```

```
Exception count
Failed requests
```

A particularly useful production metric is:

```
5xx error rate over time
```

because it helps detect server-side failures.

13. What is Micrometer?

Micrometer is a vendor-neutral metrics instrumentation library that provides an API for collecting application metrics and exporting them to monitoring systems.

Conceptually:

```
Application
  ↓
Micrometer
  ↓
Metrics
  ↓
Prometheus / Datadog / CloudWatch / etc.
```

It is often described as:

SLF4J for metrics

because it provides an abstraction over different monitoring systems, similar to how SLF4J abstracts logging implementations.

14. Micrometer vs Actuator

	Actuator	Micrometer
Purpose	Production monitoring/management features	Metrics instrumentation
Provides endpoints	Yes	No, by itself
Provides metrics API	Uses Micrometer	Yes
Health checks	Yes	Not its primary purpose
JVM/application metrics	Exposes them	Collects/instruments them

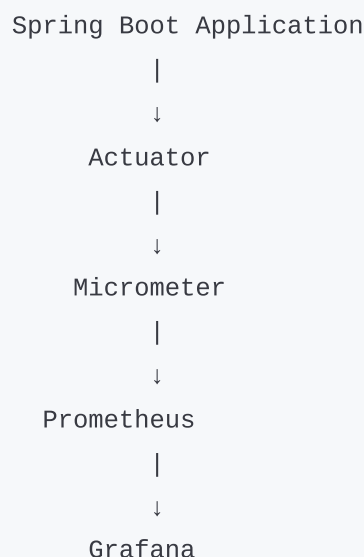
	Actuator	Micrometer
Export to monitoring systems	Through integrations	Provides registry/export support

Simple interview answer:

Actuator exposes operational endpoints and application information, while Micrometer provides the metrics instrumentation and collection abstraction that Actuator uses for metrics.

15. How would you integrate Prometheus/Grafana?

Typical architecture:



Step 1 — Add Actuator

```
spring-boot-starter-actuator
```

Step 2 — Add Prometheus registry

Use the Micrometer Prometheus registry dependency appropriate for your Spring Boot version.

Step 3 — Expose Prometheus endpoint

Configure Actuator to expose the required endpoint, commonly:

```
/actuator/prometheus
```

Step 4 — Configure Prometheus

Prometheus periodically **scrapes** the application's metrics endpoint.

```
Prometheus
  |
  | HTTP scrape
  ↓
/actuator/prometheus
```

Step 5 — Configure Grafana

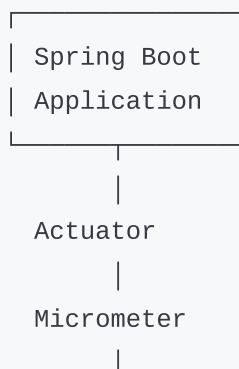
Grafana uses Prometheus as a data source and creates dashboards.

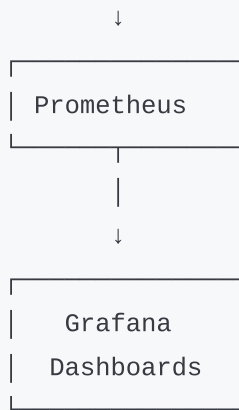
```
Spring Boot
  ↓
Micrometer
  ↓
Prometheus
  ↓
Grafana Dashboard
```

You can then build dashboards for:

```
API request rate
p95/p99 latency
5xx error rate
JVM memory
CPU
GC activity
Thread counts
Database connection pool
```

Production monitoring picture





5-year interview answer:

"I would expose only the required Actuator endpoints, secure them, use Micrometer for application and business metrics, expose Prometheus-compatible metrics, let Prometheus scrape them, and use Grafana for dashboards and alert visualization. For API monitoring, I'd track request rate, p95/p99 latency, and 4xx/5xx error rates."



SLF4J vs Logback.
Why use SLF4J?
Log levels.
DEBUG vs INFO vs WARN vs ERROR.
How should logging be designed in production?
What should never be logged?
Correlation ID.
Request ID.
Distributed tracing.
How would you trace one request across 5 microservices



1. SLF4J vs Logback

- **SLF4J** — Logging **abstraction/API** that lets application code use a common logging interface.
- **Logback** — A concrete **logging implementation** that actually writes/processes log messages.

```
Application
  ↓
SLF4J
  ↓
Logback
  ↓
Console / File / Log System
```

2. Why use SLF4J?

SLF4J decouples application code from the logging implementation, allowing you to change the underlying logging framework without changing application logging code.

Example:

```
private static final Logger log =
    LoggerFactory.getLogger(OrderService.class);

log.info("Order created: {}", orderId);
```

3. Log Levels

Common log levels:

```
TRACE  
DEBUG  
INFO  
WARN  
ERROR
```

From least to most severe:

```
TRACE → DEBUG → INFO → WARN → ERROR
```

4. DEBUG vs INFO vs WARN vs ERROR

DEBUG

Detailed information useful mainly during development or troubleshooting.

```
log.debug("Processing order {}", orderId);
```

INFO

Normal application events that are useful for understanding application behavior.

```
log.info("Order {} created successfully", orderId);
```

WARN

Something unexpected or potentially problematic occurred, but the application can continue.

```
log.warn("Payment retry attempt {}", attempt);
```

ERROR

A significant failure occurred that requires investigation or attention.

```
log.error("Failed to process order {}", orderId, exception);
```

5. How should logging be designed in production?

Production logging should be structured, meaningful, searchable, and should provide enough context to troubleshoot problems without exposing sensitive information.

A good production log should contain things such as:

```
timestamp
log level
service name
environment
class/logger
message
correlation/request ID
trace ID
important business/request identifier
exception information when relevant
```

Example:

```
2026-09-01 14:30:22
ERROR
order-service
traceId=abc123
requestId=req456
orderId=789
Failed to process payment
```

Prefer structured logs such as JSON when logs are consumed by centralized logging systems.

6. What should never be logged?

Never log sensitive information such as passwords, authentication tokens, private keys, session credentials, or other secrets.

Avoid logging:

- ✗ Passwords
- ✗ JWT/access tokens
- ✗ Refresh tokens
- ✗ API keys
- ✗ Database passwords
- ✗ Private keys
- ✗ Sensitive personal information
- ✗ Full payment/card information

Also avoid unnecessarily logging entire request/response bodies when they may contain sensitive data.

7. Correlation ID

A correlation ID is an identifier used to associate all logs belonging to the same logical request or business operation across services.

Example:

```
Correlation-ID: abc-123
```

Request:

```
Client
  ↓
Service A
  ↓
Service B
  ↓
Service C
```

All services can include:

```
correlationId=abc-123
```

in their logs.

This allows you to search centralized logs for:

```
correlationId=abc-123
```

and see the complete request flow.

8. Request ID

A request ID uniquely identifies a particular request received by a service or application.

Example:

```
Request 1 → requestId=req-001
Request 2 → requestId=req-002
```

It helps correlate logs belonging to that individual request.

9. Correlation ID vs Request ID

They can be used similarly, but conceptually:

```
Request ID
  ↓
Identifies one request

Correlation ID
  ↓
Correlates related operations across services
```

For example:

```
Client
  ↓
Order Service
  requestId = A
  correlationId = X
  ↓
Payment Service
  requestId = B
  correlationId = X
  ↓
Notification Service
  requestId = C
  correlationId = X
```

The individual request IDs differ, while the correlation ID connects the overall business/request flow.

10. Distributed Tracing

Distributed tracing tracks a request as it travels across multiple services and records the time spent in each operation/service.

Example:

```
Client
  ↓
API Gateway
  ↓
Order Service
  ↓
Payment Service
```

↓
Inventory Service

A trace represents the overall request:

Trace
├─ Gateway span
├─ Order span
├─ Payment span
└─ Inventory span

Each **span** represents an individual operation within the trace.

Common ecosystem:

OpenTelemetry
↓
Tracing backend
↓
Jaeger / Tempo / Zipkin / etc.

11. How would you trace one request across 5 microservices?

This is a very important microservices interview question.

Suppose:

Client
↓
API Gateway
↓
Order Service
↓
Payment Service
↓
Inventory Service
↓
Notification Service

Step 1 — Request enters Gateway

Generate or accept a trace context.

Trace ID = ABC123

The gateway creates the first span:

Trace ABC123
└─ Gateway span

Step 2 — Propagate trace context

When Gateway calls Order Service, it propagates the tracing context through HTTP headers. With modern distributed tracing, this is commonly done using **W3C Trace Context**, such as the `traceparent` header.

Gateway
↓
trace context
↓
Order Service

Step 3 — Order Service creates its span

Trace ABC123
├─ Gateway
└─ Order Service

Step 4 — Continue through remaining services

Trace ABC123

Gateway
↓
Order
↓
Payment
↓
Inventory
↓
Notification

Each service creates its own span while preserving the same trace context.

Step 5 — Collect traces centrally

```
Microservices
  ↓
OpenTelemetry
  ↓
Collector / tracing pipeline
  ↓
Tracing backend
  ↓
Trace visualization
```

You can then see something like:

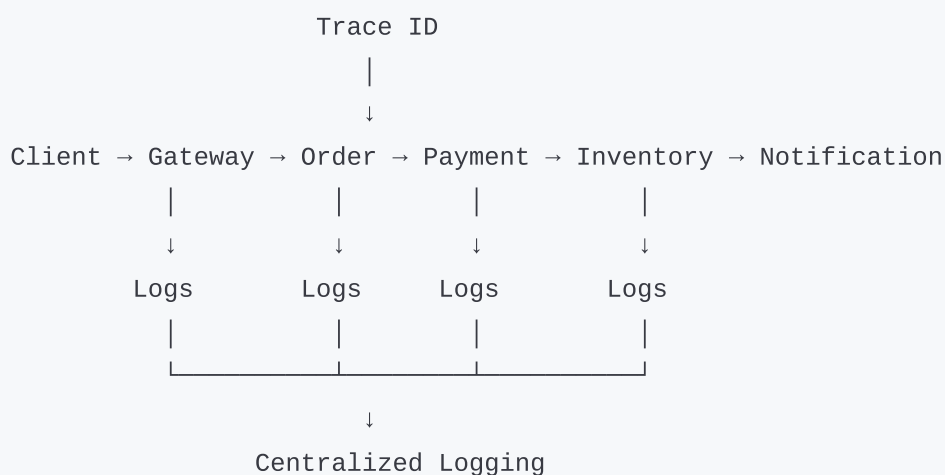
```
Trace ABC123

Gateway      20 ms
  ↓
Order       100 ms
  ↓
Payment     800 ms
  ↓
Inventory   50 ms
  ↓
Notification 30 ms
```

Now you immediately know that **Payment Service consumed most of the request time.**

Correlation ID + Distributed Tracing in Production

A strong production design is:



+

Distributed Tracing

Interview answer:

"I would use OpenTelemetry-based distributed tracing with trace-context propagation across all five services. Each service creates spans for its operations and propagates the trace context to downstream services. Logs would include the trace ID and request/correlation ID, so I can jump from a trace to the corresponding logs and identify exactly where latency or failures occurred."



What is Spring Cache abstraction?
@Cacheable.
@CachePut.
@CacheEvict.
@Caching.
Cache key.
TTL.
Cache invalidation.
Cache stampede.
Cache penetration.
Cache avalanche.
Redis caching.
Local cache vs Redis.
When should you NOT cache



1. What is Spring Cache Abstraction?

Spring Cache abstraction provides a common caching API that lets you cache method results without coupling your business code to a specific cache implementation.

```
Application
  ↓
@Cacheable
  ↓
Spring Cache Abstraction
  ↓
Redis / Caffeine / Other Cache
```

2. @Cacheable

@Cacheable stores a method's result in the cache and returns the cached value on subsequent calls instead of executing the method again.

```
@Cacheable("users")
public User getUser(Long id) {
    return userRepository.findById(id).orElseThrow();
}
```

First call



DB



Cache

Second call



Cache



Return

3. @CachePut

@CachePut always executes the method and then updates the cache with its returned value.

```
@CachePut(value = "users", key = "#user.id")
public User updateUser(User user) {
    return userRepository.save(user);
}
```

Difference:

@Cacheable → Cache hit can skip method execution

@CachePut → Method always executes and cache is updated

4. @CacheEvict

@CacheEvict removes an entry from the cache, usually when cached data becomes invalid.

```
@CacheEvict(value = "users", key = "#id")
public void deleteUser(Long id) {
    userRepository.deleteById(id);
}
```

For clearing the entire cache:

```
@CacheEvict(value = "users", allEntries = true)
```

5. @Caching

@Caching groups multiple cache operations on a single method.

```
@Caching(  
    put = {  
        @CachePut(value = "users", key = "#user.id")  
    },  
    evict = {  
        @CacheEvict(value = "userList", allEntries = true)  
    }  
)  
public User update(User user) {  
    return userRepository.save(user);  
}
```

6. Cache Key

A cache key uniquely identifies the cached value.

Example:

```
@Cacheable(value = "users", key = "#id")  
public User getUser(Long id) {  
}
```

If:

```
id = 101
```

the key could conceptually be:

```
users::101
```

A good key should be:

- Unique for the data
- Deterministic
- Stable
- Efficient to construct

For multiple parameters:

```
@Cacheable(  
    value = "products",  
    key = "#category + ':' + #page"  
)
```

7. TTL

TTL (Time To Live) defines how long a cached entry remains valid before it expires automatically.

Example:

```
User cached  
  ↓  
TTL = 10 minutes  
  ↓  
10 minutes  
  ↓  
Entry expires
```

TTL is especially important for Redis caches where data can become stale.

8. Cache Invalidation

Cache invalidation means removing or updating cached data when the underlying data changes.

Example:

```
Database  
  ↓  
User updated  
  ↓  
Old cache value becomes stale  
  ↓  
Evict/update cache
```

Common strategies:

```
Cache-aside  
Write-through  
Write-behind
```

Invalidate on update
TTL-based expiration

With Spring Cache, a common approach is:

```
@CacheEvict(value = "users", key = "#user.id")
```

or:

```
@CachePut(value = "users", key = "#user.id")
```

9. Cache Stampede

Cache stampede occurs when a popular cache entry expires and many requests simultaneously query the underlying database to rebuild the same cache entry.

Example:

```
Popular key expires
    ↓
1000 requests
    ↓
1000 DB queries
    ↓
Database overloaded
```

Common solutions:

```
Locking / request coalescing
Jittered TTL
Background refresh
Refresh-ahead
Pre-warming
```

10. Cache Penetration

Cache penetration occurs when requests repeatedly ask for data that does not exist, causing every request to hit the database because nothing valid can be cached.

Example:

```
Request userId = 999999
    ↓
```

```
Cache miss
  ↓
DB
  ↓
User doesn't exist
  ↓
No cached value
  ↓
Next request repeats
```

Solutions include:

```
Negative caching
Input validation
Bloom filters
```

For example, temporarily cache "user not found" for an appropriate short TTL.

11. Cache Avalanche

Cache avalanche occurs when many cache entries expire or become unavailable at roughly the same time, causing a large surge of requests to the underlying systems.

Example:

```
10,000 cached entries
  ↓
All expire around same time
  ↓
Huge DB traffic
  ↓
Database overloaded
```

Solutions:

```
Randomized TTL
Staggered expiration
Cache warming
High availability
Rate limiting
Fallback mechanisms
```

12. Redis Caching

Redis is an in-memory distributed data store commonly used as a shared cache between multiple application instances.

Architecture:



With Redis, multiple application instances can share the same cached data.

Typical flow:



13. Local Cache vs Redis

	Local Cache	Redis
Location	Application JVM	External service
Sharing	Not shared between instances	Shared
Latency	Very low	Low

	Local Cache	Redis
Scalability	Per-instance	Distributed
Consistency	Harder across instances	Easier to share
Infrastructure	Simple	Requires Redis
Example	Caffeine	Redis

Local cache

```
Instance 1 → Cache 1
Instance 2 → Cache 2
Instance 3 → Cache 3
```

Each instance has its own cache.

Redis

```
Instance 1 └─┐
Instance 2 ──┴─→ Redis
Instance 3 └─┐
```

All instances can access the shared cache.

14. When should you NOT cache?

Don't cache data when the cost of stale/inconsistent data is greater than the performance benefit or when the data is not reused enough to justify caching.

Avoid or carefully consider caching when:

1. Data changes extremely frequently

```
Stock price
Real-time status
```

if stale values are unacceptable.

2. Data is highly sensitive

Avoid caching sensitive information unless the security, isolation, and expiration requirements are properly designed.

3. Data has very low reuse

If every request asks for different data:

```
Request 1 → ID 1  
Request 2 → ID 2  
Request 3 → ID 3  
...
```

the cache may provide little benefit.

4. Data is very large

Caching huge objects can consume significant memory.

5. Database query is already extremely cheap

If caching adds complexity without meaningful performance improvement, don't cache.

6. Strong consistency is mandatory

If users must always see the latest database state, caching may be inappropriate unless the cache consistency strategy is designed carefully.

7. Cache invalidation is too difficult

A classic production rule:

If you cannot reliably define when cached data becomes stale and how it will be invalidated, caching can create more problems than it solves.

Strong 5-year interview answer

"I use caching for frequently read, relatively stable data where the database or downstream service is expensive to query. I first define the cache key, TTL, invalidation strategy, and consistency requirements. For a single-instance application, a local cache such as Caffeine can be sufficient; for multiple instances, Redis is useful because the cache is shared. I also consider stampede, penetration, and avalanche scenarios before introducing caching."



Monolith vs microservices.
When should you choose microservices?
Advantages/disadvantages of microservices.
Service discovery.
API Gateway.
Load balancing.
Client-side vs server-side load balancing.
OpenFeign.
RestClient/WebClient.
Synchronous vs asynchronous communication.
Circuit breaker.
Retry.
Timeout.
Bulkhead.
Rate limiting
Distributed tracing.
Centralized configuration.
Distributed transactions.
Saga pattern.
Event-driven architecture.
Kafka integration.
Idempotency.
Dead-letter queue.
Eventual consistency.



1. Monolith vs Microservices

- **Monolith** — The application is built and deployed as a single unit.
- **Microservices** — The application is split into independently deployable services, each owning a specific business capability.

Monolith:

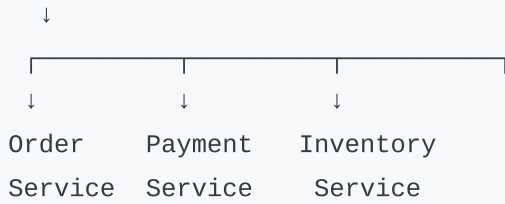
Client → Application → Database

Microservices:

Client



API Gateway



2. When should you choose Microservices?

Choose microservices when the system has sufficiently large/complex domains, requires independent scaling or deployment, and the organization can handle the operational complexity.

Good reasons:

- Independent scaling
- Independent deployments
- Clear business boundaries
- Different teams owning different services
- Fault isolation
- Different technology requirements

Don't choose microservices simply because they are popular; a small application may be better as a monolith.

3. Advantages of Microservices

- **Independent deployment** — Services can be deployed independently.
- **Independent scaling** — Scale only the service that needs more capacity.
- **Fault isolation** — Failure in one service can potentially be contained.
- **Team autonomy** — Different teams can own different services.
- **Clear business boundaries** — Services can represent business capabilities.

4. Disadvantages of Microservices

- Distributed-system complexity
- Network failures
- Distributed transactions

- More deployment/monitoring infrastructure
- Service-to-service latency
- Data consistency challenges
- More difficult debugging
- Distributed tracing requirements
- More operational overhead

5. Service Discovery

Service discovery allows services to dynamically find the network location of other service instances instead of hardcoding their addresses.

Example:

```
Order Service
  ↓
Service Discovery
  ↓
Payment Service
  ↓
10.0.2.15:8080
```

Common approaches include:

```
Service Registry
Eureka
Consul
Kubernetes Service/DNS
```

6. API Gateway

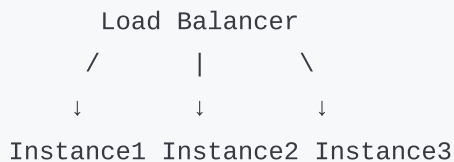
API Gateway is an entry point for client requests that can perform routing, authentication, rate limiting, TLS termination, and other cross-cutting concerns.

```
Client
  ↓
API Gateway
  ├──> Order Service
  ├──> Payment Service
  └──> User Service
```

It can prevent clients from needing to know every internal service address.

7. Load Balancing

Load balancing distributes incoming requests across multiple healthy instances of a service.



Benefits:

- Better availability
- Better resource utilization
- Horizontal scaling
- Failure handling

8. Client-Side vs Server-Side Load Balancing

Client-side

The client discovers service instances and selects which instance to call.

```
graph TD; OS[Order Service] --> SR[Service Registry]; SR --> IL[Instance list]; IL --> CSI[Client selects instance];
```

The diagram shows a vertical flow: 'Order Service' points to 'Service Registry', which points to 'Instance list', which finally points to 'Client selects instance'.

Server-side

The client sends the request to a load balancer, which selects the service instance.

```
graph TD; OS[Order Service] --> LB[Load Balancer];
```

The diagram shows a vertical flow: 'Order Service' points to 'Load Balancer'.

↓

Instance 1 / 2 / 3

Difference

Client-side → Client chooses destination

Server-side → Load balancer chooses destination

9. OpenFeign

OpenFeign is a declarative HTTP client that allows you to define service-to-service HTTP calls using Java interfaces.

Example:

```
@FeignClient(name = "payment-service")
public interface PaymentClient {

    @GetMapping("/payments/{id}")
    Payment getPayment(@PathVariable Long id);
}
```

Instead of manually constructing HTTP requests, you define the API contract and the framework creates the client implementation.

10. RestClient vs WebClient

RestClient

RestClient is Spring's synchronous HTTP client API for making blocking HTTP calls.

```
Request
  ↓
Wait
  ↓
Response
```

WebClient

WebClient is Spring's reactive, non-blocking HTTP client designed for asynchronous/reactive communication.

```
Request
  ↓
Do other work
  ↓
Response arrives
```

For a traditional Spring MVC application, `RestClient` is often simpler; `WebClient` is particularly useful in reactive applications or when non-blocking I/O is important.

11. Synchronous vs Asynchronous Communication

Synchronous

The caller waits for the downstream service response before continuing.

```
Order
  ↓
Payment
  ↓
Response
  ↓
Order continues
```

Example:

```
REST
Feign
RestClient
```

Asynchronous

The caller sends a message/event and does not need to wait for the consumer to finish processing it.

```
Order Service
  ↓
Kafka
  ↓
Payment Service
```

Key difference

Synchronous → Request/response
Asynchronous → Message/event based

12. Circuit Breaker

A circuit breaker prevents repeated calls to a failing downstream service and gives that service time to recover.

States:

```
CLOSED
  ↓ failures exceed threshold
OPEN
  ↓ after wait period
HALF_OPEN
  ↓
Success → CLOSED
Failure → OPEN
```

Example:

```
Payment Service is down

Order
  ↓
Circuit Breaker
  ↓
Don't repeatedly call Payment
  ↓
Fallback / error
```

Common Spring implementation:

Resilience4j

13. Retry

Retry automatically repeats a failed operation when the failure is considered temporary.

Example:

```
Request
  ↓
Payment Service
  ↓
Timeout
  ↓
Retry
  ↓
Payment Service
  ↓
Success
```

Use retry carefully.

Good candidates:

```
Temporary network failure
Transient service unavailability
Temporary timeout
```

Bad candidates:

```
Invalid request
Authentication failure
Permanent business validation failure
```

Retries should normally use a limit and backoff, often exponential backoff with jitter.

14. Timeout

A timeout defines the maximum amount of time a service will wait for a downstream operation before failing it.

Without a timeout:

```
Service A
  ↓
Service B hangs
  ↓
Thread waits indefinitely
  ↓
Resources exhausted
```

With timeout:

```
Service A
  ↓
Service B
  ↓
Timeout
  ↓
Fail fast / fallback
```

Timeouts are essential for preventing slow downstream services from consuming resources indefinitely.

15. Bulkhead

Bulkhead isolates resources so that failure or overload in one operation does not consume all resources of the application.

Named after ship bulkheads: one damaged section should not sink the entire ship.

Example:

```
Application
├── Payment Pool
├── Inventory Pool
└── Notification Pool
```

If Payment becomes slow:

```
Payment Pool → Exhausted
```

```
Inventory Pool → Still available
```

This prevents one dependency from taking down unrelated functionality.

16. Rate Limiting

Rate limiting restricts how many requests a client or service can make within a specified period.

Example:

```
Client
  ↓
Rate Limiter
```

↓
100 requests/minute

Request 101:

Rate limit exceeded
↓
HTTP 429

Common algorithms include:

Token Bucket
Leaky Bucket
Fixed Window
Sliding Window

17. Distributed Tracing

Distributed tracing tracks a request across multiple services using a trace ID and spans.

Client
↓
Gateway
↓
Order
↓
Payment
↓
Inventory

Conceptually:

Trace ID: ABC123

Gateway	— Span 1
Order	— Span 2
Payment	— Span 3
Inventory	— Span 4

This helps identify:

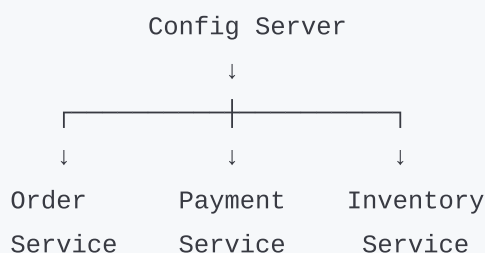
- Which service is slow

- Where an exception occurred
- End-to-end latency
- Downstream dependency problems

OpenTelemetry is commonly used for instrumentation and context propagation.

18. Centralized Configuration

Centralized configuration stores application configuration in a central system so services don't need to maintain configuration independently in every deployment.



Examples:

Spring Cloud Config
Consul
Kubernetes ConfigMap/Secret
Cloud-native configuration services

Benefits:

- Centralized management
- Environment-specific configuration
- Easier configuration changes
- Reduced duplication

Secrets should be handled through appropriate secret-management systems rather than ordinary configuration files.

19. Distributed Transactions

A distributed transaction is a business operation that spans multiple services/databases and therefore cannot usually be handled by a single local database transaction.

Example:

```
Order Service → Order DB
    ↓
Payment Service → Payment DB
    ↓
Inventory Service → Inventory DB
```

You cannot simply use one normal local transaction across all these independent databases.

Common approaches include:

```
Saga
Outbox pattern
Compensating transactions
```

20. Saga Pattern

Saga breaks a distributed business transaction into a sequence of local transactions, with compensating actions used when later steps fail.

Example:

```
Create Order
    ↓
Reserve Inventory
    ↓
Process Payment
    ↓
Order Confirmed
```

If payment fails:

```
Create Order
    ↓
Reserve Inventory
    ↓
Payment FAILED
    ↓
Release Inventory
    ↓
Cancel Order
```

The compensating actions undo the business effects of earlier successful steps.

Two common Saga styles:

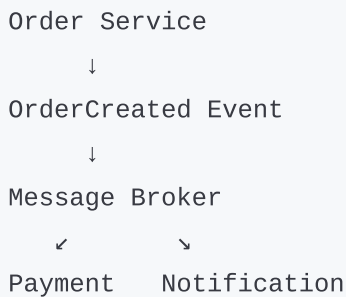
Choreography → Services react to events

Orchestration → Central Saga orchestrator coordinates steps

21. Event-Driven Architecture

Event-driven architecture uses events to communicate that something happened, allowing other components to react asynchronously.

Example:



The producer doesn't necessarily need to know which consumers process the event.

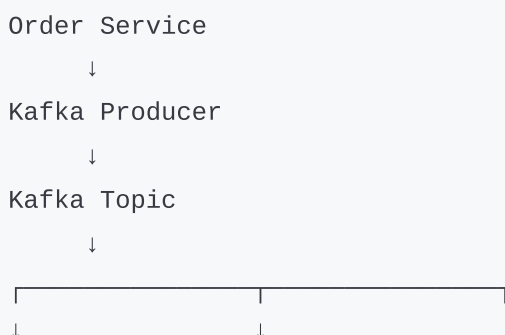
Benefits:

- Loose coupling
- Asynchronous processing
- Scalability
- Easy addition of new consumers

22. Kafka Integration

Kafka is a distributed event-streaming platform commonly used to publish, store, and consume events between services.

Typical architecture:



Payment	Notification
Consumer	Consumer

Example event:

```
OrderCreated
{
  "orderId": 123,
  "customerId": 456
}
```

Important Kafka concepts:

Producer
Consumer
Topic
Partition
Offset
Consumer Group
Broker

23. Idempotency

An operation is idempotent when performing it multiple times produces the same intended business result as performing it once.

Example:

```
POST payment request
  ↓
Network timeout
  ↓
Client retries
  ↓
Same payment request arrives again
```

Without idempotency:

Payment 1
Payment 2 ❌

With an idempotency key:

Idempotency-Key: ABC123

First request:

ABC123 → Process payment

Retry:

ABC123 → Return existing result

This is especially important for payment/order APIs and message consumers.

24. Dead-Letter Queue

A Dead-Letter Queue (DLQ) stores messages that cannot be successfully processed after configured retry attempts or because they are otherwise considered unprocessable.

```
Kafka / Queue
  ↓
Consumer
  ↓
Processing fails
  ↓
Retry
  ↓
Retry
  ↓
Still fails
  ↓
DLQ
```

The DLQ allows engineers to inspect and reprocess problematic messages without continuously blocking normal processing.

25. Eventual Consistency

Eventual consistency means different services or replicas may temporarily have different data, but they converge to a consistent state after successful asynchronous processing.

Example:

```
Order Service
  ↓
Order = CONFIRMED
```

↓
OrderCreated event
↓
Inventory Service
↓
Inventory updated shortly afterward

There can be a short period:

Order DB → CONFIRMED
Inventory DB → old state

Later:

Order DB → CONFIRMED
Inventory DB → updated

This is common in event-driven microservices because services often own separate databases and communicate asynchronously.

🔥 Important 5-Year Scenario

"Order → Payment → Inventory. Payment is slow. How would you design it?"

A strong answer would be:

"I would first define whether payment must be synchronous for the user flow. For synchronous calls, I would use a strict timeout, limited retry with exponential backoff and jitter for transient failures, and a circuit breaker to prevent cascading failures. I'd use bulkheads to isolate payment resources from other operations. If the business permits asynchronous processing, I'd publish an order/payment event through Kafka and use an event-driven Saga for the overall workflow. I'd make payment operations idempotent so retries don't cause duplicate payments, use a DLQ for messages that repeatedly fail, and use distributed tracing with correlation/trace IDs to monitor the complete flow."



What is API Gateway?
Why do we need an API Gateway?
Gateway vs Load Balancer.
What is service discovery?
Eureka vs Consul.
What happens when service discovery is unavailable?
How does OpenFeign work?
Feign timeout configuration.
Retry configuration.
Circuit breaker with Resilience4j.
What is fallback?
Retry vs circuit breaker.
Why can retries sometimes make a production outage worse?
What is bulkhead isolation?
How would you prevent cascading failures?



1. What is API Gateway?

API Gateway is a single entry point for clients that routes requests to backend microservices and can handle cross-cutting concerns such as authentication, rate limiting, routing, and monitoring.

```
Client
  ↓
API Gateway
  ├── User Service
  ├── Order Service
  └── Payment Service
```

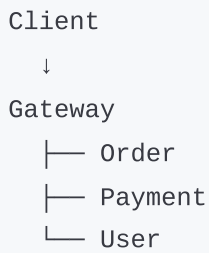
2. Why do we need an API Gateway?

We use an API Gateway to hide internal service topology from clients and centralize common concerns such as authentication, routing, rate limiting, CORS, logging, and request transformation.

Without gateway:

```
Client → Order Service
Client → Payment Service
Client → User Service
```

With gateway:



3. Gateway vs Load Balancer

API Gateway

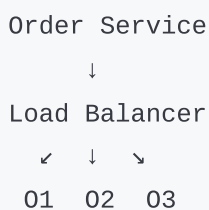
API Gateway operates mainly at the application/API level and understands routes, APIs, authentication, and policies.

Example:

```
/api/orders/**    → Order Service
/api/payments/**  → Payment Service
```

Load Balancer

Load balancer distributes traffic across healthy instances of a service or application.



Key difference

API Gateway
→ Which service should handle this request?

Load Balancer
→ Which healthy instance of that service should handle it?

They can be used together:



```
API Gateway
  ↓
Load Balancer
  ↓
Order-1 / Order-2 / Order-3
```

4. What is Service Discovery?

Service discovery allows services to dynamically find the network locations of other service instances.

Instead of hardcoding:

```
http://10.10.1.20:8080
```

a service asks a discovery mechanism:

```
Where is payment-service?
```

and receives available instances:

```
payment-service
├─ 10.10.1.20:8080
├─ 10.10.1.21:8080
└─ 10.10.1.22:8080
```

5. Eureka vs Consul

	Eureka	Consul
Created by	Netflix	HashiCorp
Service discovery	Yes	Yes
Health checks	Yes	Yes
Key/value configuration	Limited compared with Consul	Yes
Multi-platform ecosystem	Primarily Java/Spring ecosystem	Broad
Common Spring integration	Spring Cloud Netflix	Spring Cloud Consul

Simple interview answer

Eureka is primarily focused on service discovery within the Netflix/Spring ecosystem, while Consul provides service discovery plus health checking, service networking features, and a distributed key-value store.

6. What happens when Service Discovery is unavailable?

It depends on the architecture and whether services have cached service information, but new discovery operations may fail and services may be unable to locate newly registered instances.

For example:

```
Order Service
  ↓
Service Discovery ❌
  ↓
Cannot discover Payment
```

If the client has cached previously discovered instances:

```
Service Discovery ❌
  ↓
Cached instance list
  ↓
Continue calling known instances
```

But this can become problematic because:

- New instances may not be discovered
- Removed/unhealthy instances may remain in stale cache
- Scaling information may become outdated
- New deployments may not be visible

Production design should avoid making service discovery a single point of failure and should use redundancy/caching appropriate to the discovery technology.

7. How does OpenFeign work?

OpenFeign provides a declarative HTTP client where you define an interface and Spring creates the client implementation that performs HTTP calls.

Example:

```
@FeignClient(name = "payment-service")
public interface PaymentClient {

    @GetMapping("/payments/{id}")
    Payment getPayment(@PathVariable Long id);
}
```

Then:

```
Payment payment =
    paymentClient.getPayment(100);
```

Conceptually:

```
Order Service
  ↓
Feign Proxy
  ↓
Service Discovery
  ↓
Payment Instance
  ↓
HTTP
```

Feign handles the HTTP client plumbing so the application code can work with an interface.

8. Feign Timeout Configuration

Feign timeouts define how long the client waits for connection establishment and response processing before failing the call.

Conceptually:

```
spring:
  cloud:
    openfeign:
      client:
        config:
          payment-service:
            connectTimeout: 2000
            readTimeout: 3000
```

Meaning:

Connection timeout → 2 seconds
Read/response timeout → 3 seconds

The exact configuration properties can vary by Spring Cloud version, so the project's version-specific documentation should be followed.

9. Retry Configuration

Retry configuration determines whether and how failed calls are attempted again.

Example concept:

```
Request
  ↓
Payment
  ↓
Timeout
  ↓
Retry #1
  ↓
Timeout
  ↓
Retry #2
  ↓
Failure
```

A production retry policy should define:

```
Maximum attempts
Backoff
Jitter
Retryable exceptions/statuses
Overall timeout/deadline
```

Don't blindly retry every failure.

10. Circuit Breaker with Resilience4j

Resilience4j Circuit Breaker monitors failures and temporarily stops calls to an unhealthy dependency when failure thresholds are exceeded.

Example:

```
@CircuitBreaker(
    name = "paymentService",
```

```

        fallbackMethod = "paymentFallback"
    )
    public Payment makePayment() {
        return paymentClient.pay();
    }

```

Flow:

```

CLOSED
  ↓
Failures increase
  ↓
Threshold reached
  ↓
OPEN
  ↓
Calls fail fast
  ↓
After wait duration
  ↓
HALF_OPEN
  ↓
Test calls

```

11. What is Fallback?

Fallback is an alternative response or action executed when a protected operation fails or the circuit breaker is open.

Example:

```

public Payment paymentFallback(
    Throwable ex) {

    return Payment.pending();
}

```

Conceptually:

```

Payment Service
  ↓
  FAIL
  ↓
Circuit Breaker
  ↓

```

Fallback



Return alternative response

A fallback should be **business-safe**; it should not hide serious failures or incorrectly report that an operation succeeded.

12. Retry vs Circuit Breaker

Retry

Retry says: "The failure may be temporary, so try again."

Call → Failure → Retry → Retry → Success

Circuit Breaker

Circuit breaker says: "The dependency is currently unhealthy, so stop calling it for a while."

Failures



Circuit opens



Fail fast

They can work together

Request



Circuit Breaker



Retry



Payment Service

But they must be configured carefully.

13. Why can retries sometimes make a production outage worse?

Retries can multiply traffic against an already overloaded or failing service, causing a retry storm and making the outage worse.

Suppose:

```
100 requests/sec
```

Each request retries twice:

```
100 original  
+ 100 retry 1  
+ 100 retry 2  
= up to 300 attempts/sec
```

Now imagine 10 services doing similar retries:

```
Service A  
  ↓  
Service B  
  ↓  
Service C
```

The extra traffic can propagate through the system.

This can cause:

```
Service overloaded  
  ↓  
Requests become slow  
  ↓  
Timeouts  
  ↓  
Retries  
  ↓  
More requests  
  ↓  
More overload  
  ↓  
Cascading failure
```

Therefore use:

```
Limited retries  
Exponential backoff  
Jitter
```

Timeouts
Circuit breakers
Rate limits

14. What is Bulkhead Isolation?

Bulkhead isolation limits the resources available to one dependency or workload so that its failure cannot consume all application resources.

Example:

Application
├─ Payment calls → 20 threads
├─ Inventory calls → 20 threads
└─ Notification calls → 10 threads

If Payment becomes slow:

Payment pool → Exhausted
Inventory → Still has resources
Notification → Still has resources

Without isolation:

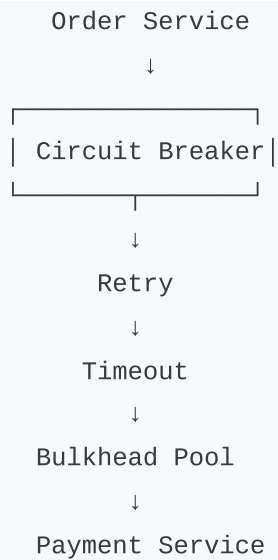
Payment calls consume all threads
↓
No threads left
↓
Entire application becomes unhealthy

15. How would you prevent cascading failures?

I would combine timeouts, limited retries with backoff/jitter, circuit breakers, bulkheads, rate limiting, bounded queues/thread pools, graceful degradation, and proper monitoring/tracing.

A typical design:

```
graph TD
    Client --> API_Gateway[API Gateway]
    API_Gateway --> Rate_Limiter[Rate Limiter]
    Rate_Limiter --> 
```



Example production failure

Suppose Payment Service becomes slow.

Without protection:

```
Payment slow
↓
Order threads wait
↓
Order thread pool exhausted
↓
Order API becomes slow
↓
Gateway requests pile up
↓
More services affected
↓
Cascading failure
```

With resilience patterns:

```
Payment slow
↓
Timeout
↓
Limited retry
↓
Circuit opens
↓
Bulkhead protects Order resources
```

↓

Fallback / controlled failure

↓

Order Service remains available

Strong 5-year interview answer

"I would first set realistic timeouts so calls don't wait indefinitely. For transient failures I'd use limited retries with exponential backoff and jitter, but only for operations that are safe to retry. I'd put a circuit breaker around unstable dependencies so repeated failures don't continuously consume resources. I'd use bulkheads to isolate thread pools or concurrency for critical downstream services, rate limiting to control incoming load, and bounded queues to prevent unbounded resource consumption. Finally, I'd use distributed tracing, metrics, and alerts to detect latency, error rates, and dependency failures early."



Unit test vs integration test.

JUnit 5.

Mockito.

@Mock.

@InjectMocks.

@MockBean / modern Spring test alternatives.

@SpringBootTest.

@WebMvcTest.

@DataJpaTest.

MockMvc.

Testcontainers.

How do you test REST controllers?

How do you test service classes?

How do you test repositories?

How do you test external API calls?

What should be mocked and what shouldn't?

Unit test vs integration test trade-offs.



1. Unit Test vs Integration Test

- **Unit Test** — Tests one class/component in isolation by mocking its dependencies.
- **Integration Test** — Tests how multiple real components work together, often including Spring context, database, or external infrastructure.

Unit:

Service → Mock Repository

Integration:

Controller → Service → Repository → Database

2. JUnit 5

JUnit 5 is a Java testing framework used to write and execute automated tests using annotations such as `@Test`, `@BeforeEach`, and `@AfterEach`.

```
@Test
void shouldCreateUser() {
    // arrange
    // act
```

```
// assert  
}
```

Common annotations:

```
@Test  
@BeforeEach  
@AfterEach  
@BeforeAll  
@AfterAll  
@ParameterizedTest
```

3. Mockito

Mockito is a mocking framework used to create and configure test doubles for dependencies in unit tests.

Example:

```
when(userRepository.findById(1L))  
    .thenReturn(Optional.of(user));
```

Then verify:

```
verify(userRepository).findById(1L);
```

4. @Mock

@Mock creates a Mockito mock object that behaves as a fake dependency during a unit test.

```
@Mock  
private UserRepository userRepository;
```

It does **not** create the real repository implementation.

5. @InjectMocks

@InjectMocks creates the class under test and injects the Mockito mocks into its dependencies.

```
@Mock  
private UserRepository repository;
```

```
@InjectMocks
private UserService service;
```

Conceptually:

```
UserService
  ↓
Mock UserRepository
```

6. @MockBean / Modern Spring Test Alternatives

`@MockBean` historically added a Mockito mock to the Spring `ApplicationContext` and replaced a matching bean, but in newer Spring Boot versions it has been deprecated in favor of Spring Framework's `@MockitoBean`.

Modern example:

```
@MockitoBean
private UserService userService;
```

Use Spring's mock-bean support when you are testing with a Spring context and need to replace a real bean with a mock.

7. @SpringBootTest

`@SpringBootTest` loads a broad/full Spring Boot application context and is mainly used for integration-style testing.

```
@SpringBootTest
class OrderIntegrationTest {
}
```

Conceptually:

```
Spring Boot Context
  ↓
Controller
  ↓
Service
  ↓
Repository
  ↓
Database
```

It is powerful but generally slower than focused test slices.

8. `@WebMvcTest`

`@WebMvcTest` **loads only the Spring MVC/web layer and is commonly used to test controllers without loading the entire application.**

```
@WebMvcTest(UserController.class)
class UserControllerTest {
}
```

Typically:

```
Controller
  ↓
Mock Service
```

It is useful for testing:

- Request mapping
- Validation
- HTTP status
- JSON request/response
- Exception handling
- Controller behavior

9. `@DataJpaTest`

`@DataJpaTest` **loads a focused JPA test context for testing repositories and entity persistence behavior.**

```
@DataJpaTest
class UserRepositoryTest {
}
```

It typically configures:

```
Repository
EntityManager
JPA
Database/Test database
```

It is useful for testing:

- Repository queries
- Entity mappings
- Relationships
- JPQL/native queries
- Persistence behavior

10. MockMvc

MockMvc allows you to test Spring MVC controllers by sending simulated HTTP requests without starting a real web server.

Example:

```
mockMvc.perform(  
    get("/users/1")  
)  
    .andExpect(status().isOk());
```

You can test:

```
HTTP method  
URL  
Headers  
Request body  
Status code  
Response JSON  
Validation
```

11. Testcontainers

Testcontainers runs real infrastructure components such as databases, Kafka, or Redis in disposable containers during tests.

Example:

```
JUnit Test  
  ↓  
Testcontainers  
  ↓  
PostgreSQL Container
```

↓

Repository

This is useful when an embedded database does not accurately represent production behavior.

For example:

Production → PostgreSQL

Test → PostgreSQL container

instead of:

Production → PostgreSQL

Test → H2

12. How do you test REST Controllers?

I would use `@WebMvcTest` and `MockMvc`, mocking the service layer and verifying HTTP behavior.

Example:

```
@WebMvcTest(UserController.class)
class UserControllerTest {

    @Autowired
    MockMvc mockMvc;

    @MockitoBean
    UserService userService;

    @Test
    void shouldGetUser() throws Exception {

        when(userService.getUser(1L))
            .thenReturn(new UserDto(1L, "Manoj"));

        mockMvc.perform(get("/users/1"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.id").value(1))
            .andExpect(jsonPath("$.name").value("Manoj"));
```

```
}  
}
```

You should test:

```
Request mapping  
HTTP status  
Request validation  
Request/response JSON  
Exception handling  
Authorization rules where appropriate
```

13. How do you test Service Classes?

I would use a pure JUnit + Mockito unit test and mock repositories/external dependencies.

```
@ExtendWith(MockitoExtension.class)  
class UserServiceTest {  
  
    @Mock  
    UserRepository repository;  
  
    @InjectMocks  
    UserService service;  
  
    @Test  
    void shouldReturnUser() {  
  
        User user = new User(1L, "Manoj");  
  
        when(repository.findById(1L))  
            .thenReturn(Optional.of(user));  
  
        User result = service.getUser(1L);  
  
        assertEquals("Manoj", result.getName());  
  
        verify(repository).findById(1L);  
    }  
}
```

Test:

Business logic
Validation
Exception handling
Branching
Repository interaction
Business rules

14. How do you test Repositories?

I would use `@DataJpaTest` and test the repository against a real test database, preferably the same database technology as production when database-specific behavior matters.

Example:

```
@DataJpaTest
class UserRepositoryTest {

    @Autowired
    UserRepository repository;

    @Test
    void shouldFindByEmail() {

        User user = new User();
        user.setEmail("test@example.com");

        repository.save(user);

        Optional<User> result =
            repository.findByEmail("test@example.com");

        assertTrue(result.isPresent());
    }
}
```

For production-like database behavior:

```
@DataJpaTest
    ↓
Testcontainers
    ↓
PostgreSQL
```

15. How do you test External API Calls?

I would avoid calling the real external service in normal tests and use a mock HTTP server such as WireMock or MockWebServer to simulate the external API.

Example:

```
Service Under Test
    ↓
Mock HTTP Server
    ↓
Fake response
```

Test scenarios such as:

```
200 → Success
400 → Bad request
401 → Unauthorized
404 → Not found
500 → Server error
Timeout
Slow response
Malformed response
```

For a small unit test, mock the HTTP client itself; for an integration-style client test, prefer a mock HTTP server so the HTTP serialization, headers, status handling, and client configuration are actually exercised.

16. What should be mocked and what shouldn't?

Mock

Mock external dependencies whose behavior is not the subject of the unit test.

Examples:

```
Repository
External REST client
Kafka producer
Third-party service
Email service
Payment gateway
```

For a service unit test:

```
UserService
  ↓
Mock Repository
  ↓
Mock External API
```

Don't mock unnecessarily

Don't mock the class you are actually testing or simple domain objects/value objects.

For repository integration tests, don't mock the JPA repository itself—you are specifically testing it.

For controller tests, mock the service because the controller is the subject.

17. Unit Test vs Integration Test Trade-offs

	Unit Test	Integration Test
Speed	Very fast	Slower
Scope	Small	Larger
Dependencies	Mocked	Real/partially real
Debugging	Easier	More complex
Confidence in integration	Lower	Higher
Cost	Low	Higher
Typical tool	JUnit + Mockito	Spring Boot Test/Testcontainers

Unit test

```
Service
  ↓
Mocks
```

Advantages:

- Fast
- Isolated

- Easy to run frequently
- Easy to identify business-logic failures

Disadvantages:

- Can miss integration problems
- Mocks may behave differently from real dependencies

Integration test

```
Controller
  ↓
Service
  ↓
Repository
  ↓
Real/Test Database
```

Advantages:

- Tests real integration
- Detects mapping/query/configuration issues
- Higher confidence in component interaction

Disadvantages:

- Slower
- More setup
- More infrastructure
- Potentially more difficult to diagnose

Strong 5-year interview answer

"I prefer a testing pyramid: many fast unit tests for business logic, focused integration tests for database and framework integration, and a smaller number of end-to-end tests for critical user flows. I don't mock everything; I mock dependencies when testing a unit in isolation, but I use real infrastructure or Testcontainers when the integration itself is what I need to verify."